Appunti teoria SO

Evoluzione Hardware e Multiprogrammazione

> **Contesto Storico**
>
> Agli albori dell'informatica, quando i calcolatori funzionavano ancora a valvole ed erano estremamente costosi e lenti, la presenza di un operatore umano era indispensabile per risparmiare tempo e gestire manualmente l'esecuzione.

Nuove Funzioni Hardware Con l'evoluzione dei sistemi di elaborazione, sono state introdotte nuove funzionalità a livello hardware per consentire al **Monitor** (il predecessore dell'odierno sistema operativo) di gestire meglio le risorse:

- **Protezione della memoria:** Funzionalità che impedisce ai programmi utente di accedere ad aree di memoria non autorizzate.
- **Timer:** Meccanismo che previene la monopolizzazione della CPU da parte di un singolo processo.
- **Istruzioni privilegiate:** Particolari istruzioni di controllo che possono essere eseguite solamente dal Monitor.
- **Interruzioni:** Eventi che permettono al sistema di sospendere l'esecuzione corrente per gestire operazioni asincrone.

> **Approfondimento: Violazione di Memoria e Interruzioni**
>
> In certe situazioni è strettamente necessario poter "violare" o accedere alla memoria in modo controllato durante l'esecuzione. Le **interruzioni** permettono al sistema di compiere un salto verso un'istruzione specifica, che non fa parte delle normali istruzioni utente ma del linguaggio del Monitor. Ad esempio, se un processo richiede troppo tempo per l'esecuzione (*too much time to execute*), l'interruzione restituisce il controllo al sistema.

Il Problema dell'Overhead L'utilizzo del Monitor ha però generato il problema dell'**Overhead** (sovraccarico di sistema):

- Una parte significativa della memoria centrale deve essere costantemente occupata dal Monitor stesso.
- Una frazione del tempo macchina (CPU) viene consumata dal Monitor per il suo stesso funzionamento, sottraendo tempo all'esecuzione effettiva dei programmi.

Terza Generazione (1965–1980) La terza generazione di calcolatori è segnata dall'introduzione dei **circuiti integrati**.

> **Nota sui Circuiti Integrati**
>
> I circuiti integrati sono costituiti da piastre di silicio che contengono migliaia di transistor. Questo ha permesso di costruire computer molto più potenti e decisamente meno dispendiosi in termini di energia.

La Multiprogrammazione L'innovazione software chiave di questo periodo è la **multiprogrammazione**. È nata per risolvere l'enorme spreco di tempo della CPU durante le

operazioni di Input/Output.

Ecco un esempio di quanto fosse inefficace l'esecuzione sequenziale:

- Lettura di un record: 15 micros
- Esecuzione di 100 istruzioni: 1 micros
- Scrittura di un record: 15 micros
- **Totale:** 31 micros
- **Utilizzo reale della CPU:** 1/31 = 3.2%

Uniprogramming vs Multiprogramming

- **Uniprogramming:** I processi vengono eseguiti uno alla volta, in modo strettamente sequenziale.
- **Multiprogramming:** Si caricano più job contemporaneamente in memoria e si assegna a turno la CPU ai vari processi.

> **Dinamica di Assegnazione delle Risorse**
>
> Immaginiamo 3 job che necessitano di risorse diverse. Nella multiprogrammazione, il sistema li carica tutti in memoria contemporaneamente, purché lo spazio sia sufficiente e periferiche come disco e stampante non debbano essere utilizzate nello stesso esatto momento. Se il primo job si blocca in attesa di un I/O e non usa la CPU, questa risorsa non rimane inattiva, ma viene assegnata al processo successivo pronto per l'esecuzione.

I Problemi della Multiprogrammazione Questo approccio ha introdotto nuove sfide architetturali e di gestione:

1. **Gestione della memoria:** Nell'esempio precedente la memoria richiesta era sufficiente, ma nella realtà, se si attivano numerosi processi, questi potrebbero non entrare fisicamente nella memoria disponibile, richiedendo tecniche di gestione avanzate.
2. **Scheduling:** Determinare come, quando e a chi assegnare il processore. Quando la CPU passa dal primo processo agli altri a ruota, il sistema operativo deve "togliere" il processo corrente dal processore. Questo genera il problema di dover salvare i dati di stato del processo uscente, per poi poterli ripristinare in modo trasparente quando il processo otterrà nuovamente il turno di esecuzione.

Il Time-Sharing Il **Time-sharing** (condivisione del tempo) è un paradigma sviluppato per supportare la gestione di più lavori interattivi:

- Permette a calcolatori potenti di supportare più utenti contemporaneamente sulla stessa macchina.
- Il processore viene condiviso in modo fluido tra tutti gli utenti attivi.
- L'accesso avviene simultaneamente attraverso l'uso di terminali.
- Il sistema operativo alterna raPIDamente l'esecuzione di ciascun programma utente assegnando un **quanto di elaborazione** (un breve periodo di tempo).
- Esempio storico notevole: il sistema **CTSS** (Compatible Time-Sharing System).

Multiprogrammazione vs Time-Sharing Pur condividendo principi di esecuzione concorrente, hanno finalità diverse:

Caratteristica	Multiprogrammazione	Time-Sharing
Obiettivo Principale	Massimizzare l'uso del processore (massimizzare i lavori completati)	Minimizzare il tempo di risposta percepito dall'utente

Caratteristica	Multiprogrammazione	Time-Sharing
Istruzioni al SO	**JCL** (Job Control Language): istruzioni preparate in anticipo e fornite insieme al job	Comandi interattivi inseriti direttamente dall'utente al terminale

Quarta Generazione (1980 - 1990)

- Nasce con l'introduzione della tecnologia **LSI** (Integrazione su larga scala).
- Vede la nascita e la diffusione di massa dei **Personal Computer**.
- È caratterizzata dallo sviluppo di un'**interfaccia amichevole** (user-friendly) per avvicinare l'informatica al grande pubblico.

Quinta Generazione (1990 - oggi)

- Dominata dall'avvento di dispositivi mobili come gli **smartphone**.
- Impone nuovi vincoli fisici e progettuali:
  - Gestione dell'**energia limitata**.
  - Integrazione e gestione di **nuovi dispositivi**.

> **Nuovi Dispositivi**
>
> I moderni sistemi operativi mobili devono tenere costantemente conto dell'energia utilizzata, dei forti limiti della **memoria** a disposizione e della presenza di innumerevoli componenti hardware aggiuntive, come i **sensori**, che richiedono driver e gestioni specifiche.

Funzionamento di un sistema di elaborazione

- *CPU:* costituita dai registri (es. Program counter che contiene l'indirizzo dell'istruzione successiva da eseguire o l'instruction register che contiene l'istruzione da eseguire,...)
- *Memoria principale:* vengono caricati i programmi che sono costituiti da istruzioni (quelli che la CPU può eseguire) e dati (locazioni di memoria in cui mettiamo le nostre informazioni).
- *Moduli I/O:* relativi a tastiera, monitor e dischi

Nei sistemi moderni ci sono più processori (cores) su un singolo chip (socket) . Ognuno di questi può svolgere contemporaneamente agli altri delle istruzioni/programmi. Meno si accede alla memoria principale e più è veloce la computazione => per questo esiste la memoria cache (unità di memoria piccola così da essere veloce e poco costosa) dalla quale la CPU prende un piccolo numero di dati e informazioni. I dati contenuti nella cache sono quelli usati di recente (molto probabilmente li riuseremo). Poi di solito c'è un processore grafico (GPU). Questo è in grado di applicare la stessa istruzione a tutti i dati di un array (in un tempo ridotto). Usati principalmente per aspetti grafici (es. Pixel). La tecnica usata è la *Single-instruction multiple-data* (SIMD) e si può usare anche per applicazioni che non sono grafiche. Esiste poi un processore di segnale digitale (DSP) che codifica/decodifica segnali audio e video e garantisce un supporto crittografico. Ad oggi tutto ciò si può trovare su un solo chip (Soc) => funzionale per i cellulari

Ciclo di estrazione ed esecuzione: L'istruzione viene estratta, viene eseguita e, se il processo è terminato, si torna in dietro e si ricomincia dall'inizio. Tipi di istruzione riguardano:

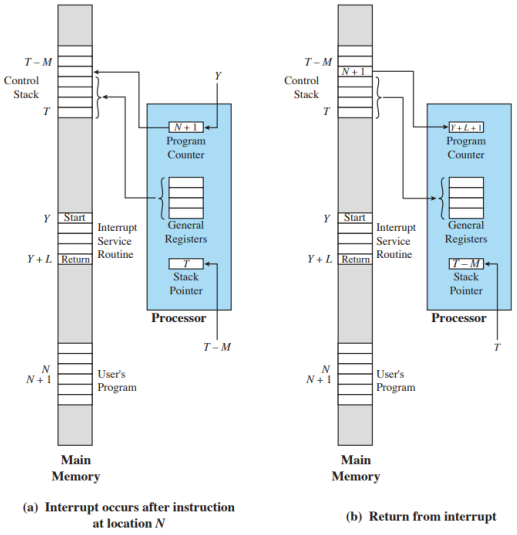
- Memoria
- I/O
- Elaborazione dati (es. Operazioni logiche o aritmetiche)

- Controllo (es. Goto)

> **Tip**
>
> RICORDA:
>
> - PC = memorizza l'indirizzo di memoria della prossima istruzione da prelevare. Durante la fase di fetch il valore contenuto nel PC viene inviato alla memoria e, dopo il prelievo, viene automaticamente incrementato per puntare all'istruzione successiva nella sequenza. Nel caso di un salto il PC viene caricato con un nuovo indirizzo specifico
> - IR = contiene l'istruzione che la CPU sta elaborando. L'istruzione prelevata dalla RAM viene copiata nell'IR. Dopodiché il suo contenuto viene letto per decodificare l'operazione e i dati da utilizzare

Interruzioni: Se la macchina funzionasse sempre in questo modo non avrei modo di intervenire. Per questo esistono le interruzioni che permettono agli altri moduli di interrompere il ciclo di fetch-execute. In questo modo si evita di lasciare la CPU inattiva o in attesa di I/O e si può sospendere l'esecuzione di un programma. Le classi di interruzione sono:

- Interruzioni di programma = generate da errori nell'esecuzione di un programma (es. Overflow, eccezioni di indirizzo = un programma non può andare a modificare le informazioni di altri programmi e deve restare nello spazio di memoria a lui assegnato, ecc ...)
- Timer = permettono al sistema operativo di eseguire delle funzioni a intervalli regolari
- I/O = generate da un controllore I/O per segnalare il completamento di un'operazione o un errore
- Errori HW = generati da problemi hardware



Quindi il procedimento è: estrazione istruzione -> esecuzione istruzione -> check delle interruzioni. Se c'è l'interruzione se ne guarda il tipo si compie un salto all'indirizzo al quale inizia il programma per la gestione delle interruzioni (*interrupt handler*) e successivamente si salta ad un'altra istruzione. Una volta che il gestore delle interruzioni ha finito di gestire il problema legato ad un'interruzione, questa viene eseguita.

Se avviene un'interruzione durante la gestione di un'interruzione ci sono almeno 2 possibilità:

1. Disabilito le interruzioni durante la gestione di un'interruzione (può funzionare dato che le interruzioni sono brevi)
2. Definisco uno schema di priorità: in questo caso il sistema permette a un'interruzione di interromperne un'altra in base ad un sistema di priorità. Ciò significa che se l'interruzione in arrivo ha una priorità **più alta** di quella attualmente in esecuzione, il processore mette in pausa la prima, viene salvato il PC e viene eseguita la seconda (quella più urgente). Alla fine della seconda istruzione, il processore recupera i dati relativi al PC e riprende la prima delle due istruzioni che ha causato un'interruzione.

Sviluppo di un sistema operativo

Aspetti principali nello sviluppo dei sistemi operativi:

- Processi
- Gestione della memoria
- Protezione e sicurezza dell'informazione
- Schedulazione e gestione delle risorse
- Struttura del sistema

Approcci e elementi progettuali dei sistemi operativi:

- Architettura micro-kernel
- Multithread
- Multiprocessing simmetrico
- Sistemi operativi distribuiti
- Architettura orientata agli oggetti

Il sistema operativo può realizzare una macchina virtuale. Cioè il sistema operativo crea una macchina che abbia una sua memoria e un suo sistema operativo. Esiste un virtual machine monitor (VMM) che viene eseguito sul sistema operativo e gestisce le comunicazioni tra i singoli sistemi operativi virtuali e il processore

Servizi del sistema operativo:

- Esecuzione programmi
- Gestione di operazioni di I/O
- Filesystem
- Comunicazione con altri sistemi
- Allocazione delle risorse
- Accounting (verifica chi usa il sistema e per quanto tempo)
- Sicurezza e protezione
- Riconoscimento degli errori (nell'esecuzione dei programmi)

Queste funzioni sono utilizzate attraverso le chiamate di sistema = interruzioni richieste dall'utente stesso. Si accede al sistema tramite delle interfacce:

- GUI
- Batch (file di istruzioni che vengono eseguite una dietro l'altra)
- Linea di comando

Servizi per l'utente:

- Interfaccia utente: devo essere in grado di interagire con il sistema operativo (o da interfaccia grafica o da linea di comando o dai file di batch)
- Esecuzione di programmi: caricamento, esecuzione e terminazione di programmi
- Operazioni I/O
- Organizzare il filesystem: creare/eliminare file, leggere/scrivere in un file, cercare informazioni e gestire i permessi (di lettura, scrittura o esecuzione per utente, gruppi e altri)
- Comunicazione fra processi: ogni processo ha il suo spazio e non può invadere lo spazio di altri processi (di solito va bene ma a volte voglio che i processi interagiscano tra di loro). A volte ci può essere uno spazio di memoria condivisa tra i vari processi o in alternativa c'è uno scambio di messaggi (un processo manda un messaggio ad un altro processo attraverso il sistema operativo)
- Rilevazione errori: hardware, di I/O o dei programmi utente

Funzioni che servono ad assicurare l'efficienza del sistema:

- Allocazione delle risorse: alcuni programmi necessitano di risorse di processore/i, memoria, disco o altri dispositivi I/O. L'ordine con cui assegno le risorse determina se il sistema è efficiente o meno
- Accounting: soprattutto nei processi condivisi è bene sapere chi usa il sistema e quanto lo usa
- Protezione e sicurezza: utilizzo di tutte le risorse va riservato a chi ne ha diritto (protezione) e controllo degli accessi al sistema (sicurezza)

Interprete dei comandi (CLI) vs interfaccia grafica (GUI):

CLI	**GUI**
Esegue in sequenza i comandi specificati dall'utente. Viene implementato nel nucleo o programma di sistema (in questo caso ci sono molte versioni (*shells*). I comandi sono implementati nell'interprete o in programmi esterni	È un'interfaccia amichevole (*user friendly*) in cui le icone rappresentano file, directories, programmi e funzioni. Inoltre ci sono menù che permettono di selezionare i comandi e la possibilità di avere touch-screen e comandi vocali
Per compiti ripetititvi	Più intuitiva ma non accede a tutte le funzioni

System call

Le system call sono un'interfaccia per il programmatore ai servizi del SO. Vi si accede attraverso l'API che nasconde i dettagli al programmatore (gestita da libreria di supporto run-time). I parametri sono passati attraverso:

- Registri
- In un blocco o tabella di memoria dell'indirizzo in un registro
- Nello stack da cui sono prelevati (pop) dal sistema operativo

Categorie di chiamate di sistema:

1. **Controllo dei processi:

   - Creazione e arresto
   - Caricamento e esecuzione
   - Terminazione normale e anormale
   - Esame e impostazione degli attributi tramite funzioni apposite
   - Funzioni che mettono in pausa per un tempo indicato
   - Attesa segnalazione evento
   - Assegnazione e rilascio di memoria

2. **Gestione file:**

   - Creazione e cancellazione
   - Apertura e chiusura
   - Lettura, scrittura, posizionamento (posso leggere un carattere a scelta e non per forza uno dopo l'altro)
   - Esame e impostazione degli attributi

3. **Gestione dei dispositivi I/O:**

   - Richiesta (di assegnazione) e rilascio
   - Lettura, scrittura e posizionamento

- Esame e impostazione degli attributi
- Inserimento ed esclusione logica

4. **Gestione delle informazioni:**

- Esame e impostazione dell'ora e della data
- Esame e impostazione dei dati di sistema
- Esame e impostazione degli attributi dei processi, file e dispositivi

5. **Comunicazione:**

- Creazione e chiusura di connessione
- Invio e ricezione di messaggi
- Creazione e accesso a regioni di memoria condivisa (interazione con condivisione)
- Informazioni sullo stato di un trasferimento
- Inserimento ed esclusione dei dispositivi remoti

6. **Protezione:**

- Controllo degli accessi alle risorse
- Visualizzazione e impostazione dei permessi
- Autorizzazione e divieto di accesso all'utente

Un sistema operativo è fatto anche di programmi detti *utilities* che servono a realizzare l'ambiente utile per lo sviluppo e l'esecuzione dei programmi. Ce ne sono diverse categorie:

- Gestione files
- Informazione di stato
- Modifica di file (text editor)
- Supporto alla programmazione (compilatore, debugger, linker,...)
- Caricamento ed esecuzione dei programmi
- Comunicazioni
- Servizi in background
- Programmi applicativi

Progettazione di un sistema operativo

Non è semplice dato che devo tenere conto di che hardware si dispone e di che tipo di richiesta abbiamo da parte dell'utilizzatore. Quindi le specifiche da rispettare sono:

- Decidere hardware
- Decidere che sistema usare
- Obiettivi dell'utilizzatore
- Obiettivi del sistema (es obiettivo dell'utilizzatore = far terminare i programmi nel minor tempo possibile, obiettivo del sistema = non lasciare il processore inutilizzato)
- Separazione delle politiche (se ho un server servo il primo client e non l'ultimo che ha fatto una richiesta) dai meccanismi (es come si accede al disco)

I moderni sistemi operativi sono scritti in linguaggi come C e C++ (non più Assembler) perché ciò mi garantisce:

- Facilità di sviluppo e manutenzione
- Compattezza
- Portabilità

Inoltre sono linguaggi più efficienti di linguaggi interpretati come Python (computer non sono fatti per mandare avanti il sistema operativo, ma per far partire programmi di sistema). Alcune sezioni critiche che richiedono altissima efficienza sono scritte in Assembler.

Struttura del sistema operativo

Sistemi monolitici (Non sono divisi in moduli): MS-DOS o UNIX tradizionale sono sistemi monolitici complessi e scritti tramite molte funzioni. Il kernel di UNIX tradizionale ha un'interfaccia verso l'hardware per la gestione delle chiamate di sistema (questo all'interno del kernel che gestisce segnali I/O, file system, driver, dispositivi di archiviazione e scheduling)

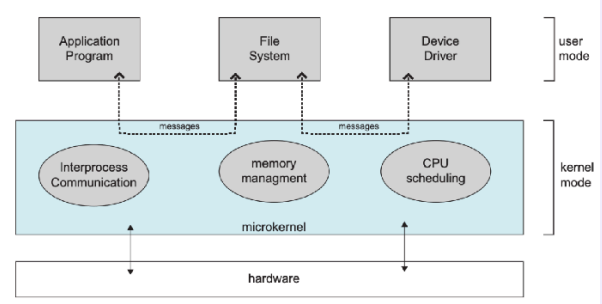
Sistemi stratificati: Ogni layer (strato) è l'implementazione di un oggetto astratto fatto di dati e operazioni per manipolarli. Ogni strato nasconde i dettagli dell'implementazione agli strati superiori.

- *Vantaggi* = semplicità di scrittura, aggiornamento, debugging e portabilità
- *Svantaggi* = difficoltà nel definire gli strati e overhead

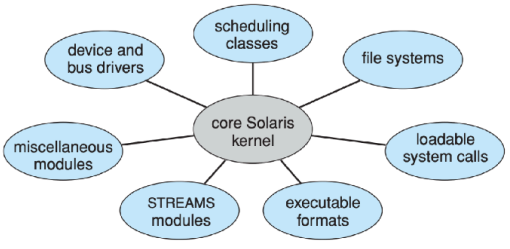
Microkernel: Sono sistemi monolitici che facciano il minimo necessario. Il resto è trasferito a livello-utente (server). Le richieste di servizi avvengano tramite scambio di messaggi tra client e server.

- *Vantaggi* = semplicità di aggiornamento e debugging, portabilità e affidabilità
- *Svantaggi* = overhead (il kernel affida più compiti ai programmi e per questo c'è bisogno di un maggiore tempo di esecuzione)

Example



Non c'è differenza dal punto di vista del sistema tra questi programmi, ma ogni volta che un programma applicativo deve accedere a un file o un dispositivo invierà una richiesta tramite messaggio che sarà svolta un un altro programma che è il gestore del dispositivo e non dal kernel, dato che questo contiene solo le funzioni fondamentali (altre funzionalità sono in moduli, ogni modulo nasconde i dettagli di implementazione agli altri moduli)



Solaris ha questo tipo di architettura: il kernel contiene tutte le funzioni fondamentali e una serie di moduli da collegare quando si vuole per le loro specifiche funzioni

Debugging: Per il fatto che i SO sono sistemi complessi, ci possono essere degli errori e per questo nella loro progettazione e gestione c'è il problema del debugging. Gli errori, però, si possono presentare in situazioni particolari (difficili da individuare), considerando che il SO deve fare il suo compito nel minor tempo possibile. Quindi valutare

le prestazioni di un SO (e per monitorare il comportamento del sistema) richiede una lunga sperimentazione tramite anche la generazione di file di log che registrano il comportamento del sistema e *core dump*

Generazione sistema operativo: Scritto il codice sorgente del sistema va compilato e configurato per il sistema su sui verrà eseguito poi si istalla il sistema avvia il computer e il nuovo sistema operativo. La configurazione può realizzarsi modificando il codice prima della compilazione in base al sistema su cui lo vado a installare, oppure tramite la selezione di tabelle e moduli precompilati da collegare al momento dell'installazione, oppure tramite un sistema guidato da tabelle di dati che guidano il sistema in base all'hardware. Per far partire il sistema l'esecuzione inizia in un predefinito indirizzo della ROM (parte di memoria non modificabile nemmeno a computer spento = *read only memory*) che contiene il *bootstrap loader*. Questo contiene le istruzioni necessarie a caricare il *bootstrap program* che a sua volta carica il kernel e avvia l'esecuzione del sistema

Processi

Definizione: un processo (o job) è un programma (lista di istruzioni con riferimenti a delle locazioni di memoria) in esecuzione. Gli elementi del processo sono:

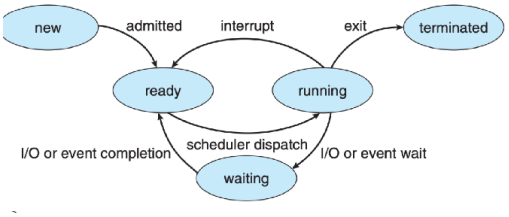
- Program counter (indirizzo istruzione da eseguire) e contenuto dei registri (immagine del processore)
- Stack che contiene variabili locali, indirizzi di return o parametri delle funzioni
- Sezione testo (programma)
- Sezione dati che contiene i valori delle variabili locali
- Heap per l'allocazione dinamica

Programma	Processo
Entità passiva su disco / file eseguibile	Entità che esegue le istruzioni e usa le risorse associate
Sono istruzioni che non cambiano finché non avviene una modifica e non ha memoria allocata né stato di esecuzione	Possiede risorse proprie ed evolve attraverso stati successivi

> **Ricorda**
>
> - Un programma può essere associato a più processi attraverso il multitasking = situazione in cui il SO permette di avviare lo stesso codice eseguibile più volte, assegnando a ciascuna istanza uno spazio in memoria, un PID e delle risorse
> - nella memoria il processo occupa un certo spazio occupato dal codice, i dati, l'heap e lo stack dove vengono eseguite le procedure (la heap si allarga verso l'alto e lo stack verso il basso e bisogna fare attenzione affinché non collidano)

Stati del processo:

- *Nuovo* = in fase di creazione
- *In esecuzione* = il processore sta eseguendo le istruzioni
- *In attesa* = il processo attende un evento
- *Pronto* = il processo attende la disponibilità di un processore
- *Terminato* = il processo ha terminato l'esecuzione



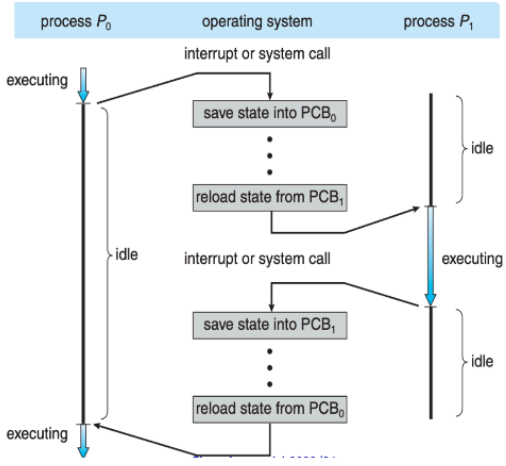
Quando il processo viene creato può trovarsi in stato di attesa di un processore assegnato ad esso che esegua le istruzioni e lo termini. Può capitare che l'esecuzione del processo sia temporaneamente interrotta => non è necessario che le risorse del processore siano ancora assegnate a questo processo => si salva l'immagine del processore e il processo rimane in attesa dell'evento che lo libera dallo stato di attesa. Quando l'evento avviene saranno riassegnate al processo delle risorse con l'immagine salvata prima e si completerà l'esecuzione/terminerà il processo

PCB (descrittore di processo): A ogni processo è associata una struttura dati detta descrittore di processo o PCB (*process control block*) contenente tutte le informazioni relative al processo:

- Stato del processo
- Program counter
- Registri CPU (immagine processore)
- Parametri per lo scheduling della CPU
- Informazioni per la gestione della memoria
- Informazioni di accounting per sistemi multiutente
- Informazioni su I/O

Un processo può avere multiple sequenze di istruzioni (*threads*) che possono essere eseguite in parallelo (anche su core differenti) per la realizzazione di task differenti. In tal caso il PCB include informazioni su ciascun thread.

Salvataggio del PCB:



Supponiamo che venga creato il processo P 0 che è in stato di running. Ad un certo punto si verifica un'interruzione e tutte le informazioni del processo vengono salvate nel PCB 0. Queste saranno ricaricate dal sistema di gestione delle interruzioni dopo che il processore avrà terminato un altro processo P1 (ricaricato da PCB 1 quando magari P1 era stato interrotto precedentemente). A questo punto può accadere una nuova interruzione, si salva lo stato di P1 dentro a PCB 1 e si ricarica lo stato contenuto in PCB 0 (si riprende esecuzione di P 0). Questo ci permette di usare il sistema in maniera efficiente purché le operazioni di salvataggio e ricarica degli stati del processore siano eseguite in maniera efficiente.

Il sistema ha bisogno di conoscere la situazione dei vari processi e per questo i PCB sono inseriti in varie code:

- Coda di job che contiene tutti i processi
- Coda di ready che contiene tutti i processi pronti a partire (quelli tra i quali il processore deve decidere a chi assegnare le risorse)
- Coda dei dispositivi (una per dispositivo) che contiene i processi in attesa di interagire con quel dispositivo (il dispositivo genera una nuova interruzione)

Schedulatori e fasi del processo

Il sistema operativo deve decidere a quali processi assegnare il processore e per fare ciò ci sono diversi schedulatori:

- **Schedulatore a breve termine (CPU scheduler):** Prende i processi dalla coda ready e gli assegna un processore. Siccome deve essere eseguito con grande frequenza, deve essere molto veloce (i processi running vengono anche messi in coda ready proprio perché lo scheduler assegna la CPU a un processo, poi la toglie a quello per assegnarla ad un altro, così da far avanzare più processi contemporaneamente).
- **Schedulatore a lungo termine (*job scheduler*):** Seleziona i processi da inserire in coda ready e per fare ciò controlla il grado di multiprogrammazione, cioè i programmi eseguiti contemporaneamente (regola il numero di processi presenti contemporaneamente in memoria. Se è troppo alto, la CPU si sovraccarica; se è troppo basso, le risorse restano inattive). Non viene eseguito con frequenza, per questo può applicare procedure più complesse. Ci sono ad esempio processi CPU-bound, ovvero processi che non rilasciano mai la CPU a loro assegnata a meno che non siano forzati

> **Example**
>
> - I/O bound processi che interagiscono con input e output
> - In UNIX se entro un certo periodo di tempo il processo non rilascia il processore viene costretto a rilasciarlo e viene rimesso in coda (timebound) È bene avere entrambi i tipi di processi in quanto ci potrebbero essere o processi che non possono avanzare o processi che richiedono troppe risorse se ci fosse una prevalenza dei due

- **Schedulatore a medio termine:** Determina con che velocità avanzano i processi nel sistema. Si occupa di assegnare o revocare la memoria ai processi (*swapping*). Quando i processi non hanno bisogno di memoria, gli viene revocata la RAM e la memoria del processo viene copiata su disco rigido. La memoria liberata viene occupata da un nuovo processo.

La schedulazione realizza delle politiche per l'assegnazione della CPU. Generalmente distingue tra:

- Processi *I/O bound*
- *Processi CPU bound*

Oltre a questo c'è un problema di overhead (risorse accessorie sono ridondanti rispetto a quelle necessarie) nel cambio dei contesto, il quale varia in funzione di:

- Velocità della memoria
- Numero registri del processore
- Istruzioni speciali (riservate al kernel)

Creazione del processo Il sistema operativo fornisce meccanismi per creazione/terminazione dei processi. Generalmente un processo genitore chiede al kernel di creare un processo figlio che a sua volta può creare altri processi (*albero dei processi* che in Linux posso vedere con il comando `pstree`). Ogni processo è identificato da un numero detto *identificatore di processo (PID)* su linux è il primo numero che vedo quando scrivo `ps` nel terminale. Quando un processo termina, viene eliminato dall'albero.

Assegnazione delle risorse ad un processo figlio Avviene in base al tipo di sistema operativo:

- Un processo figlio riceve le risorse dal processo padre
- Risorse ripartite = il padre non può usarle finché il figlio non ha finito di usarle

- Risorse condivise = il padre e il figlio condividono le risorse del padre
- Il processo figlio ottiene le risorse dal sistema operativo

Il processo genitore e il processo figlio possono essere eseguiti in modo concorrente; in alternativa, il genitore può attendere la terminazione del figlio tramite una system call (ad esempio `wait`).
Per quanto riguarda la memoria, il processo figlio inizialmente è una copia del processo genitore (stesso codice, dati e stack), ma può successivamente eseguire un nuovo programma sostituendo il proprio spazio di memoria (ad esempio tramite `exec`).

> **La funzione `fork` è una syscall che duplica il processo chiamante (padre) creando un processo figlio, restituendo valori distinti per differenziarli: 0 al processo figlio, il PID (Process ID) del figlio al processo padre (>0), e -1 in caso di errore.**
>
> - **Nel processo FIGLIO (`fork() == 0`):** La funzione restituisce `0`, indicando che il codice in esecuzione è la copia appena creata.
> - **Nel processo PADRE (`fork() > 0`):** La funzione restituisce un intero positivo, che è il PID del nuovo processo figlio creato.
> - **In caso di ERRORE (`fork() < 0`):** Non viene creato alcun processo figlio e viene restituito un valore negativo (spesso -1) al padre, solitamente a causa della mancanza di memoria o del superamento del limite massimo di processi.

Terminazione del processo Il processo termina con la syscall *exit()*, così:

- Restituisce un valore di stato al processo genitore, via *wait()*
- Le risorse sono deallocate dal sistema operativo

Il genitore può terminare l'esecuzione del figlio con la syscall *abort()* e questo per varie motivazioni:

- Eccesso di risorse allocate
- Compito non più richiesto

La terminazione di un processo figlio fa si che vengano terminati anche i figli del processo terminato (*terminazione a cascata*). Solo dopo il wait() del genitore viene deallocato il PID:

- Il PID però non scompare subito e il processo rimane in stato di zombie e scompare solo dopo che il genitore ha eseguito il wait()
- Il genitore termina senza che vengano terminati i suoi processi figli, in tal caso quei processi si dicono orfani.

Threads

I processi sono caratterizzati da:

- **Risorse allocate:** spazio di indirizzamento virtuale (immagine in memoria), la traccia di esecuzione, RAM, dispositivi I/O, file
- **Scheduling:** traccia di esecuzione (alternata con altri processi), stati e priorità

Ma le due caratteristiche potrebbero essere trattate in maniera indipendente:

- **Processo (o task):** possiede le risorse
- **Thread (o processo leggero):** unità di lavoro all'interno di un processo.

Si parla di multithreading la capacità di un SO di permettere più tracce di esecuzione concorrenti in un singolo processo (quindi ad un processo o più processi vengono assegnati più di un singolo thread ciascuno)

Single-threading	**Multi-threading**
Un processo, un thread (MD-DOS)	Un processo, più thread (Java run-time)
Più processi, un thread per ciascuno (UNIX tradizionale)	Più processi, ciascuno con più thread (Windows, Solaris, UNIX)

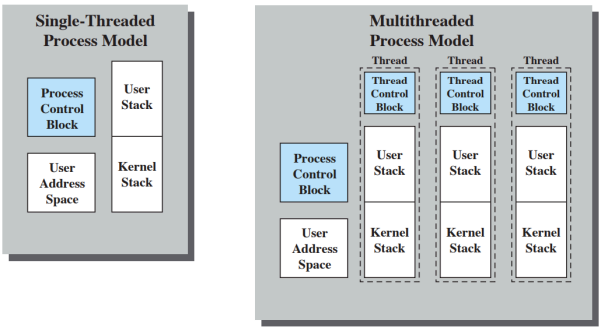
Definizione: in un ambiente multi-thread, il processo è l'unità di allocazione delle risorse e un'unità di protezione (nessun altro può accedere alle risorse assegnate al processo). Associati al processo ci sono:

- Spazio di indirizzamento virtuale (contiene immagine del processo = è l'insieme completo e strutturato dei dati che definiscono lo stato di un programma in esecuzione, includendo il codice, i dati in RAM, lo stack e i registri)
- Accesso protetto al processore, ad altri processi (IPC), files, dispositivi I/O
- Uno o più thread

Associati al thread ci sono:

- Stato = pronto, in attesa, in esecuzione
- Contesto del thread (immagine del processo), viene salvata quando non è in esecuzione
- Stack di esecuzione
- Spazio di memoria statico per le variabili locali
- Accesso a memoria e risorse del processo, condivise con gli altri thread del processo

Questo genera un problema: due thread che accedono agli stessi dati/stessa memoria (magari uno vuole incrementare e l'altro decrementare la stessa variabile)



I benefici del multithreading sono:

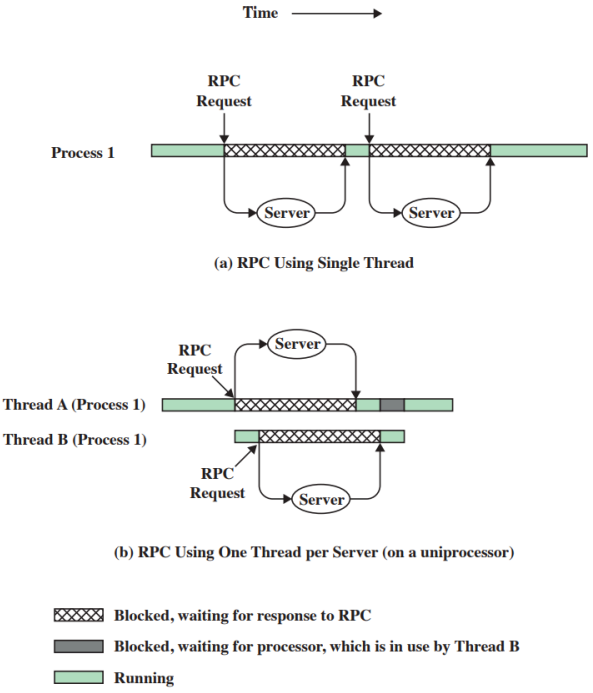
- La creazione di un thread è più rapida della creazione di un processo
- La terminazione di un thread è più rapida della terminazione di un processo (no questione di protezione della memoria)
- Lo switch tra due thread del medesimo processo è più rapida dello switch tra due processi (sempre per via della memoria)
- Efficienza della comunicazione fra programmi (eseguiti da thread dello stesso processo)

In un sistema a utente singolo:

- Esecuzione in foreground e background
- Elaborazione asincrona
- Velocità di esecuzione
- Struttura modulare dei programmi

Osservazione: Si nota che la schedulazione è gestita principalmente nel thread, ma non la sospensione e la terminazione del processo.

Gli stati del thread sono gli stessi dei processi (in esecuzione, pronto e in attesa), così come le operazioni (creazione, block/sospensione, unblock/riattivazione, terminazione).



Sincronizzazione dei thread: I thread di un processo condividono spazio di indirizzamento (porzione di memoria virtuale condivisa con gli altri thread dello stesso processo che include codice, dati e risorse) e altre risorse. Ogni modifica simultanea di un dato da parte di due thread può corrompere il dato stesso. Per questi motivi è necessario sincronizzare l'accesso dei thread alle risorse

Thread a livello utente: La gestione dei thread è fatta dall'applicazione e il kernel non sa dell'esistenza dei thread.

- *Vantaggi*: lo scambio di thread non richiede privilegi della modalità kernel, la schedulazione può essere specifica per l'applicazione e portabilità.
- *Svantaggi:* le chiamate di sistema sono bloccanti per l'intero processo e non ci sono vantaggi dai multiprocessori
- *Contromisure:* programmare le applicazioni in processi multipli e *jacketing* (trasforma una chiamata di sistema bloccante in una non bloccante)

Thread a livello kernel: La gestione dei thread è fatta dal kernel e l'applicazione usa un'API per la gestione dei thread. Il kernel gestisce i PCB dei processi e dei thread e inoltre la schedulazione del processore sulla base dei thread.

- *Vantaggi:* possibilità di eseguire thread differenti su diversi processori, il blocco di un thread non blocca il processo e anche le routine del kernel possono essere gestite in multithreading
- *Svantaggi:* i cambi di modalità (utente/kernel) producono overhead

Approcci misti: Creazione dei thread, schedulazione e sincronizzazione nello spazio utente

- Thread modalità utente mappati su thread modalità kernel (in numero minore o uguale)
- Permette esecuzione parallela dei thread e il blocco di un thread non blocca il processo (come solaris)

Modello	Descrizione
1:1	Ogni thread di esecuzione è un processo unico con il suo spazio e le sue risorse
M: 1	Un processo definisce uno spazio di indirizzo e il possesso di risorse dinamiche. Molti threads possono essere creati ed eseguiti in quel processo
1: M	Un thread può migrare dall'ambiente di un processo all'ambiente di un altro processo. Ciò permette al thread di muoversi facilmente tra sistemi distinti
M: N	Combina gli attributi di 1: M e M: 1

Legge di Amdahl

$$= = 1(1\ ) + n$$

La legge di Amdahl stabilisce che il miglioramento massimo delle prestazioni di un sistema, grazie alla parallelizzazione (uso di più processori), è limitato dalla parte del software che non può essere parallelizzata. In breve: l'algoritmo, non il numero di processori, determina il limite massimo di velocità.

Il vantaggio lo hanno:

- Applicazioni multithread native (pochi processi e molti thread)
- Applicazioni multiprocesso (molti processi single-thread)
- Applicazioni Java (JVM è un processo multithread e assicura schedulazione e gestione della memoria per le applicazioni Java)
- Multiple istanze di un'applicazione

Threads in UNIX:

- Il kernel non distingue tra processi e thread
- Per ogni task (processo o thread) c'è un *task control block* (TCB) con fork()
- Fork() crea una nuova task, con una copia del TCB
- Clone() crea una nuova task, ma nel suo TCB ci sono puntatori ai campi di quello del genitore (in base ai parametri passati a clone()

Mutua esclusione

I processi possono interferire tra di loro in maniera positiva o negativa, negativa ad esempio quando sottrae risorse ad altri processi, in maniera positiva quando due o più processi interagiscono e ci permettono di realizzare un'attività che sfrutta l'uso di diversi processi in un'unica applicazione. La gestione dei processi può avvenire tramite:

- **Multiprogrammazione**: molti processi con un singolo processore, i processi avanzano attendendo che il processore si liberi;
- **Multiprocessing**: molti processi gestiti da un multiprocessore;
- **Processi distribuiti**: molti processi, gestiti da molti processori distribuiti (**cluster**), con memorie distribuite, rende le cose più complicate da gestire, ma più veloci.

I differenti processi hanno bisogno di risorse che tipicamente vengono condivise tra i vari processi, si crea quindi una *concorrenza*:

- Nelle applicazioni multiple, o nelle applicazioni strutturate (con più processi o threads che devono accedere a delle risorse)
- La modifica dei dati disponibili da parte di un processo influenza l'andamento di un altro processo
- Nella struttura del sistema operativo.

Principi della concorrenza

- Nell'uniprocessore multiprogrammato, le esecuzioni dei processi si alternano, interrompendo certi processi e riprendendone altri
- Nel multiprocessore, le esecuzioni dei processi si alternano e si sovrappongono

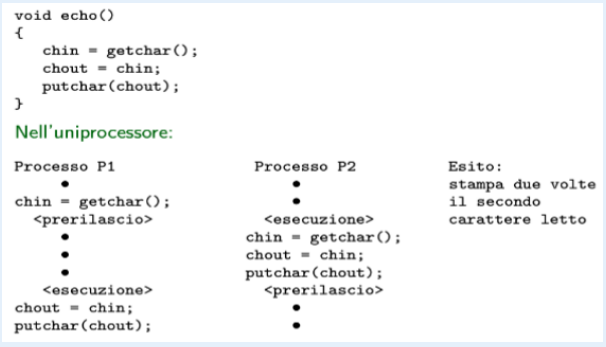
Questi processi detti *concorrenti* potrebbero dover accedere ad una risorsa e non si può prevedere quale processo specifico entrerà per primo nella risorsa, poiché la velocità relativa di esecuzione dei processi non è prevedibile. Dipende da:

- Attività degli altri processi
- Gestione delle interruzioni
- Politiche di schedulazione

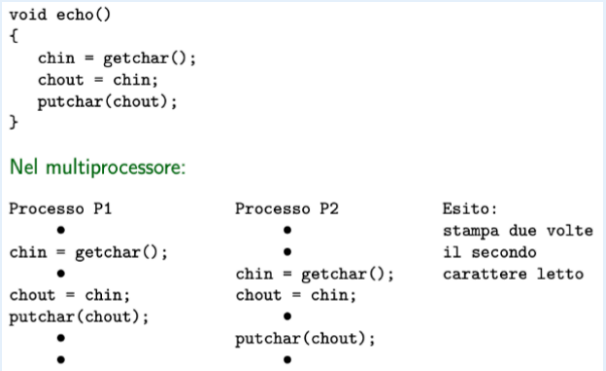
Vengono generate le seguenti difficoltà:

- Condivisione di risorse globali
- Allocazione ottimale delle risorse
- Debugging (può comparire un errore specifico dovuto a una serie di eventi sfortunati che si verifica raramente e sarà difficile da ri-individuare)

> **L'uniprocessore prende il processo P1, esegue chin=getchar(), sospende l'esecuzione di P1 passando a P2, rifà getchar() e cambia chin, stampa chout, poi si ritorna al processo P1, dove chout = chin, ma chin è stato cambiato da P2, perciò verrà stampato nuovamente il secondo carattere. Se un solo processo alla volta potesse eseguire la procedura P2 va in stato di block perchè la risorsa (in questo caso la procedura) non è disponibile poiché usata da un altro processo. La computazione va avanti finché viene ripreso P1, viene conclusa la procedura e si ritorna a P2 con la risorsa ora disponibile, i due caratteri ora sono stampati in ordine. Anche nel multiprocessore ci sarebbe questo problema.**







>

È necessario pertanto proteggere le variabili condivise e si deve controllare il codice che accede alla variabile.

Race condition: si verifica quando più processi o threads leggono e scrivono dati in modo che il risultato finale dipenda dall'ordine di esecuzione delle istruzioni dei processi

> **I processi P1 e P2 assegnano rispettivamente 1 e 2 alla variabile condivisa a. Il valore finale sarà quello del 'secondo arrivato'. I processi P1 e P2 condividono le variabili b e c con valori iniziali b = 1, c = 2. P1 esegue b = b + c, P2 esegue c = b + c. Se arriva primo P1, avremo b = 3; c = 5. Nel caso opposto, b = 4; c = 3**

Compiti del SO: La concorrenza quindi può portare a problemi, è compito del sistema operativo fare in modo che la concorrenza si svolga nella maniera corretta:

- Tenere traccia dei processi (PCB)
- Allocare e deallocare le risorse per i processi attivi (tempo di elaborazione, memoria, files, dispositivi I/O)
- Proteggere dati e le risorse di ogni processo da interferenze involontarie di altri processi
- Il risultato di ogni processo deve essere indipendente dalla susa velocità relativa a quella dei processi concorrenti (salvo l'interazione voluta tra questi processi)

Grado di consapevolezza	Relazione	Influenza che un processo ha su un altro	Potenziali problemi di controllo
Processi non consapevoli degli altri	Competizione	• I risultati di un processo indipendente dall'azione degli altri • I tempi del processo potrebbero subire variazioni.	• Mutua esclusione • Deadlock (risorsa rinnovabile) • Starvation
Processi consapevoli indirettamente degli altri (cioè oggetti condivisi)	Cooperazione tramite condivisione	• I risultati di un processo potrebbero dipendere dale informazioni ottenute da altri • I tempi del processo potrebbero subire variazioni.	• Mutua esclusione • Deadlock (risorsa rinnovabile) • Coerenza dei dati
Processi consapevoli direttamente degli altri (hanno a disposizione strumenti di comunicazione primitivi per loro)	Cooperazione tramite comunicazione	• I risultati di un processo potrebbero dipendere dale informazioni ottenute da altri • I tempi del processo potrebbero subire variazioni.	• Deadlock (risorsa rinnovabile) • Starvation

Competizione per le risorse: Cosa cambia quindi quando abbiamo processi indipendenti e processi cooperanti? La relazione più semplice è la competizione. I processi non sanno nulla (unaware) degli altri processi in corso e il risultato di un processo deve quindi essere indipendente dall'azione degli altri. Il tempo di elaborazione, però, potrebbe essere influenzato poiché una risorsa potrebbe non essere disponibile (come prima quando si blocca l'accesso a una procedura già usata) e si deve attendere che si liberi. I problemi di controllo possibili possono essere:

- Mutua esclusione (risorsa critica e sezione critica): cioè si impedisce l'accesso alla stessa risorsa da parte di più di un processo
- Stallo (*deadlock*): cioè tanti processi bloccati perché richiedono le stesse risorse già usate da altri processi e quindi al momento inaccessibili
- Starvation: un processo ha bisogno di una risorsa, ma gli altri processi la prendono prima di lui e esso non riesce mai ad accedere alla risorsa.

```
/* PROCESS 1 */
void P1 {
     while (true) {
          /* preceding code */;
          entercritical (Ra);
          /*critical section */;
          exitcritical (Ra);
          /* following code */;
     }
}

/* PROCESS 2 */
void P2 {
     while (true) {
          /* preceding code */;
          entercritical (Ra);
          /*critical section */;
          exitcritical (Ra);
          /* following code */;
     }
}
```

Collaborazione tramite la condivisione di risorse: in questo caso i processi interagiscono senza conoscersi, ma usano e modificano i dati condivisi. Il risultato di un processo può dipendere dalle azioni di un altro e anche qui i tempi di elaborazione ne sono influenzati. I problemi sono sempre gli stessi, la mutua esclusione, poiché il meccanismo di controllo deve assicurare l'integrità dei dati (tranne per le operazioni di lettura), la deadlock e la starvation, ma anche un problema di data coherence. I processi modificano dei dati condivisi e se uno dei processi sbaglia a modificare i dati essi diventano incoerenti e portano a errori anche gli altri processi.

Example

P1: a = a + 1, b = b + 1 P2: b = 2 *b, a = 2* b Posso ottenere situazioni come: a = a + 1, b = 2 *b, b = b + 1, a = 2* b $\implies$ due risultati diversi

Collaborazione per condivisione (condivisione per scambio di messaggi): in questo caso i processi collaborano per un obiettivo comune e la comunicazione permette di sincronizzarsi e coordinarsi. Ci sono delle primitive che consentono a un processo di mandare informazioni a un altro processo o di riceverle fornite dal kernel o dal linguaggio di programmazione. In questo caso il risultato è sicuramente influenzato da ciò che il processo riceve, così come il tempo di elaborazione che è limitato a quando il processo riceve l'informazione necessaria. In questo caso non c'è problema di mutua esclusione poiché i processi non condividono dati, si hanno però sempre problemi di deadlock e starvation.

Sezione critica = porzione di codice all'interno di un processo o thread che accede a risorse condivise (variabili, file, dispositivi) e che non deve essere eseguita contemporaneamente da più processi

Requisiti della mutua esclusione: Quindi per la mutua esclusione dobbiamo far sì che:

- Un solo processo alla volta è ammesso alla sezione critica per ciascuna risorsa condivisa
- Un processo che si ferma fuori dalla sua sezione critica non deve interferire con gli altri processi
- Si deve evitare deadlock e stravation
- Se nessun processo è nella sezione critica deve essere concesso a ogni processo di entrare senza ritardo
- Nessuna ipotesi sulla velocità relativa dei processi e sul numero dei processori
- Un processo resta nella sezione critica per un tempo finito

Mutua esclusione con supporto harware: Per assicurare che un processore non possa uscire da una sezione critica di un processo su uniprocessore si possono disabilitare le interruzioni tramite delle primitive definite dal kernel, ciò però rallenta il sistema ed è inapplicabile per multiprocessori:

```
while(true){
     //disabilita interruzioni
     //sezione critica
     //abilita interruzioni
     //resto del programma
}
```

È anche possibile utilizzare una serie di istruzioni speciali:

- Un solo processore alla volta può accedere a una locazione di memoria
- Speciali istruzioni macchina eseguono due operazioni su un dato in maniera atomica

Ci sono degli hardware con delle istruzioni come *compare and swap* in un'unica operazione:

```
in compare_and_swap(int *word, int testval, int newval){
     int oldval;
     oldval = *word;
     if (oldval == testval) *word = newval;
     return oldval
}

/* program mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P( int i) {
     while (true) {
          while (compare_and_swap(bolt, 0, 1) == 1)
               /* do nothing */;
          /* critical section */;
          bolt = 0;
          /* remainder */;
     }
}

void main() {
     bolt = 0;
     parbegin (P(1), P(2), ... ,P(n));
}
```

Un altro tipo di istruzione speciale è la *exchange*, una procedura che scambia il valore di 2 variabili.

```
/* program mutualexclusion */
int const n = /* number of processes */;
```

```
int bolt;
void P( int i) {
     int keyi = 1;
     while (true) {
          do exchange (&keyi, &bolt)
          while (keyi != 0);
               /* critical section */;
               bolt = 0;
               /* remainder */;
     }
}

void main() {
     bolt = 0;
     parbegin (P(1), P(2), ..., P(n));
}
```

Vantaggi:

- Applicabile a qualsiasi numero di processi su uniprocessore o multiprocessore
- Semplicità
- Può supportare multiple sezioni critiche

Svantaggi:

- Attesa attiva (*busy-waiting*)
- Possibilità di stallo
- Possibilità di starvation

Algoritmo di Dekker

L'obiettivo è quello di avere un protocollo che permetta a due processi di assicurare la mutua esclusione:

- Senza supporto hardware (salvo impossibilità di accesso simultaneo al medesimo indirizzo di memoria)
- Senza stallo
- Senza starvation

1. Primo tentativo: Apparentemente questo primo approccio funziona, c'è però un problema, supponiamo che il processo 0 esegua la sua sezione critica e che poi setti turn = 1, ma poi process 0 va in attesa, magari si vuole eseguire la sezione critica solo una volta, ma così le esecuzioni dei due processi si alternano.

```
/* PROCESS 0 /*                /* PROCESS 1 /*
...                            ...
while (turn != 0)              while (turn != 1)
   /* do nothing */ ;             /* do nothing */ ;
/* critical section*/;          /* critical section*/;
turn = 1;                      turn = 0;
...                            ...
```

2. Secondo tentativo: Si usano due flag che dovrebbero segnalare che la sezione critica sta venendo eseguita da uno dei due processi, questo non funziona poiché se i due processi arrivano quasi contemporaneamente con entrambe le flag false, entrano entrambi nella sezione critica poiché pensano che essa sia libera.

```
/* PROCESS 0 */                /* PROCESS 1 */
...                            ...
while (flag[1])                while (flag[0])
  /* do nothing */;               /* do nothing */;
flag[0] = true;                flag[1] = true;
/*critical section*/;          /*critical section*/;
flag[0] = false;               flag[1] = false;
...                            ...
```

3. Terzo tentativo: Scambiamo l'ordine delle operazioni di scrittura e lettura, prima occupo, poi aspetto che l'altra sia liberata, ma anche qui se i due processi partissero quasi insieme le due variabili sarebbero inizializzate entrambe a true, così entrambi i loop while sarebbero infiniti e i due processi non avanzeranno mai, è un esempio di stallo.

```
/* PROCESS 0 */                 /* PROCESS 1 */
...                             ...
flag[0] = true;                 flag[1] = true;
while (flag[1])                 while (flag[0])
  /* do nothing */;                /* do nothing */;
/* critical section*/;          /* critical section*/;
flag[0] = false;                flag[1] = false;
...                             ...
```

4. Quarto tentativo: Se un processo rileva che un altro processo ha flag true blocca se stesso, poi fa un delay che permette all'altro processo di avanzare e di entrare nella sezione critica per poi impostare flag a false.

```
/* PROCESS 0 */                 /* PROCESS 1 */
...                             ...
flag[0] = true;                 flag[1] = true;
while (flag[1]) {               while (flag[0]) {
  flag[0] = false;                flag[1] = false;
  /*delay */;                     /*delay */;
  flag[0] = true;                 flag[1] = true;
}                               }
/*critical section*/;          /*critical section*/;
flag[0] = false;                flag[1] = false;
...                             ...
```

La soluzione corretta:

```
boolean flag [2];
int turn;
void P0() {
     while (true) {
          flag [0] = true;
          while (flag [1]) {
               if (turn == 1) {
                    flag [0] = false;
                    while (turn == 1) /* do nothing */;
                    flag [0] = true;
               }
          }
          /* critical section */;
          turn = 1;
          flag [0] = false;
          /* remainder */;
     }
}
```

Con questa soluzione però paghiamo l'attesa attiva che è un costo non indifferente.
L'algoritmo di Dekker si può scrivere in maniera più elegante con l'algoritmo di Peterson:

```
boolean flag [2];
int turn;
void P0() {
     while (true) {
          flag [0] = true;
          turn = 1;
          while (flag [1] && turn == 1) /* do nothing */;
          /* critical section */;
          flag [0] = false;
          /* remainder */;
     }
}
```

Semafori

I semafori sono i meccanismi utilizzati per sincronizzare l'accesso alle sezioni critiche dei processi (porzione di codice in cui un processo accede a dati o risorse condivise tra più flussi di esecuzione). Sono formati da una variabile con valore intero, una coda e tre operazioni:

- Inizializzazione a un valore non negativo
- *semWait:* decrementa il calore del semaforo. Se il valore diventa negativo, il processo è sospeso e inserito nella coda
- *semSignal:* incrementa il valore del semaforo. Se il valore non diventa positivo, uno dei processi della coda è riattivato

> **Per la situazione di prima potremo utilizzare un semaforo inizializzato a 1, con la prima semWait il valore diventa 0, così tutti gli altri processi che tenteranno di accedere facendo la semWait renderanno il valore -1, sospendendo il processo e inserendolo nella coda, quando il processo iniziale manderà un semSignal si tornerà a un valore positivo e il valore precedentemente in coda entrerà nella sezione critica. In pratica quando la variabile è positiva si indica quanta disponibilità c'è, quanti processi possono accedere, quando è negativo mi dice quanti processi sono in coda in attesa di entrare nella sezione critica.**

Osservazione: il processo non ha modo di sapere in anticipo:

- Se con l'esecuzione di *semWait* sarà sospeso
- Se c'è un processo che attende l'esecuzione di *semSignal*
- Quale fra i processi in attesa sarà riattivato da *semSignal*

```
struct semaphore {
    int count;
    queueType queue;
};

void semWait(semaphore s) {
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process */;
    }
}

void semSignal(semaphore s) {
    s.count++;
    if (s.count<= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```

Semaforo binario: la variabile ora è un valore binario (o 0 o 1), poi ho una coda e tre operazioni:

- Inizializzazione a 0 o 1 (1 significa risorsa libera, 0 risorsa occupata)
- *semWaitB* verifica il valore del semaforo. Se il valore è 1 allora è portato a 0, nel caso contrario, il processo è sospeso e inserito nella coda
- *semSignalB* verifica se ci sono processi in attesa nella coda. In caso affermativo, uno dei processi della coda è riattivato, nel caso opposto, il valore del semaforo è portato a 1

Gestione della coda: (ora non posso dire che ci sono X programmi running e X in coda)

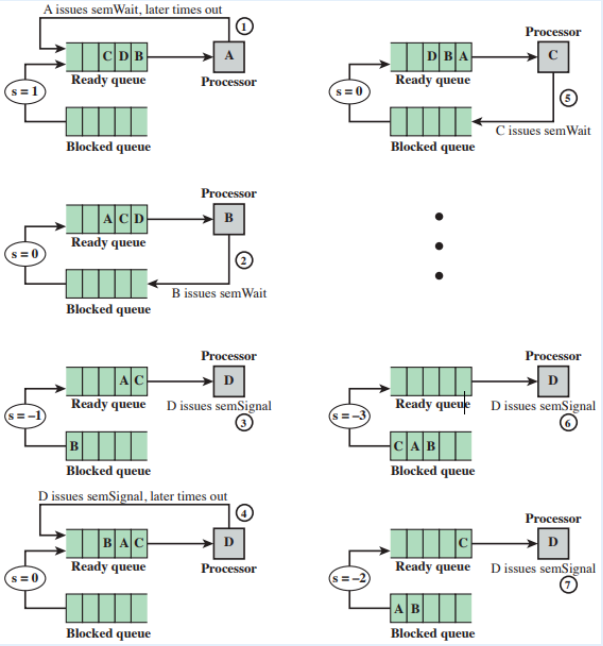
- Deve evitare la starvation
- Generalmente FIFO (*semaforo forte*)

```
struct binary_semaphore {
     enum {zero, one} value;
     queueType queue;
};

void semWaitB(binary_semaphore s) {
     if (s.value == one)
          s.value = zero;
     else {
          /* place this process in s.queue */;
          /* block this process */;
     }
}

void semSignalB(semaphore s) {
     if (s.queue is empty())
          s.value = one;
     else {
          /* remove a process P from s.queue */;
          /* place process P on ready list */;
     }
}
```

come funziona un semaforo



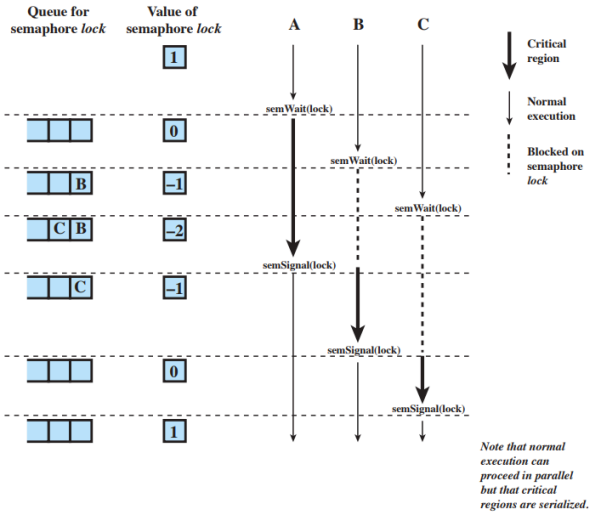

```
/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;

void P(int i) {
      while (true) {
          semWait(s);
          /* critical section */;
          semSignal(s);
          /* remainder */;
```

```
      }
}

void main() {
    parbegin (P(1), P(2), ..., P(n));
}
```

Si usa nella programmazione parallela:



Implementazione: *semWait* e *semSignal* devono essere primitive atomiche

- Con soluzioni hardware o firmware
- Con istruzioni speciali
- Con l'algoritmo di Peterson

L'alternativa alla condivisione di dati è lo scambio di messaggi, un meccanismo molto più sicuro poiché i processi non accedono allo spazio degli altri processi.

Messaggi

Anche il meccanismo dei messaggi consente la sincronizzazione per realizzare la mutua esclusione e la comunicazione per lo scambio di informazioni tra processi.

Perché utilizzare i messaggi invece della condivisione di memoria? In sistemi distribuiti i processi hanno difficoltà a leggere la memoria di altri processi in sistemi diversi, perciò il loro utilizzo è naturale nei sistemi distribuiti, ma anche nei sistemi a memoria condivisa questo meccanismo è utile. Hanno 2 primitive:

- *Send (destination, message):* un processo invia informazioni in forma di messaggio a un altro processo designato come destinatario
- *Receive (source, message):* un processo riceve informazioni indicando il mittente e l'indirizzo in cui salvare il messaggio

Caratteristiche dei sistemi di messaggistica: La sincronizzazione per la send è bloccante se il processo a cui si vuole mandare non attende un messaggio, quindi si blocca l'esecuzione del send, è non bloccante quando il messaggio viene mandato correttamente. Il receive allo stesso modo è bloccante quando si tenta di ricevere un messaggio indiretto senza che sia stato ancora mandato, non bloccante quando il messaggio già c'è, oppure si testa per la presenza di messaggi (ci sono situazioni in cui sia la send che la receive sono bloccanti). Per la send tipicamente l'indirizzamento è diretto: il processo specifica a quale processo inviare il messaggio. La receive può avere un indirizzamento esplicito (dal processo X prelevo il messaggio) o implicito. Quando il processo esegue una receive, se c'è un messaggio per questo processo, prende il messaggio e l'identificativo del processo che l'ha mandato. Il formato dei messaggi può essere di lunghezza fissa o

variabile, inoltre l'organizzazione delle code dei messaggi è FIFO, oppure può esserci una gestione tramite priorità.

Sincronizzazione: il destinatario non può ricevere il messaggio prima che questo sia stato inviato. Quando si esegue la receive:

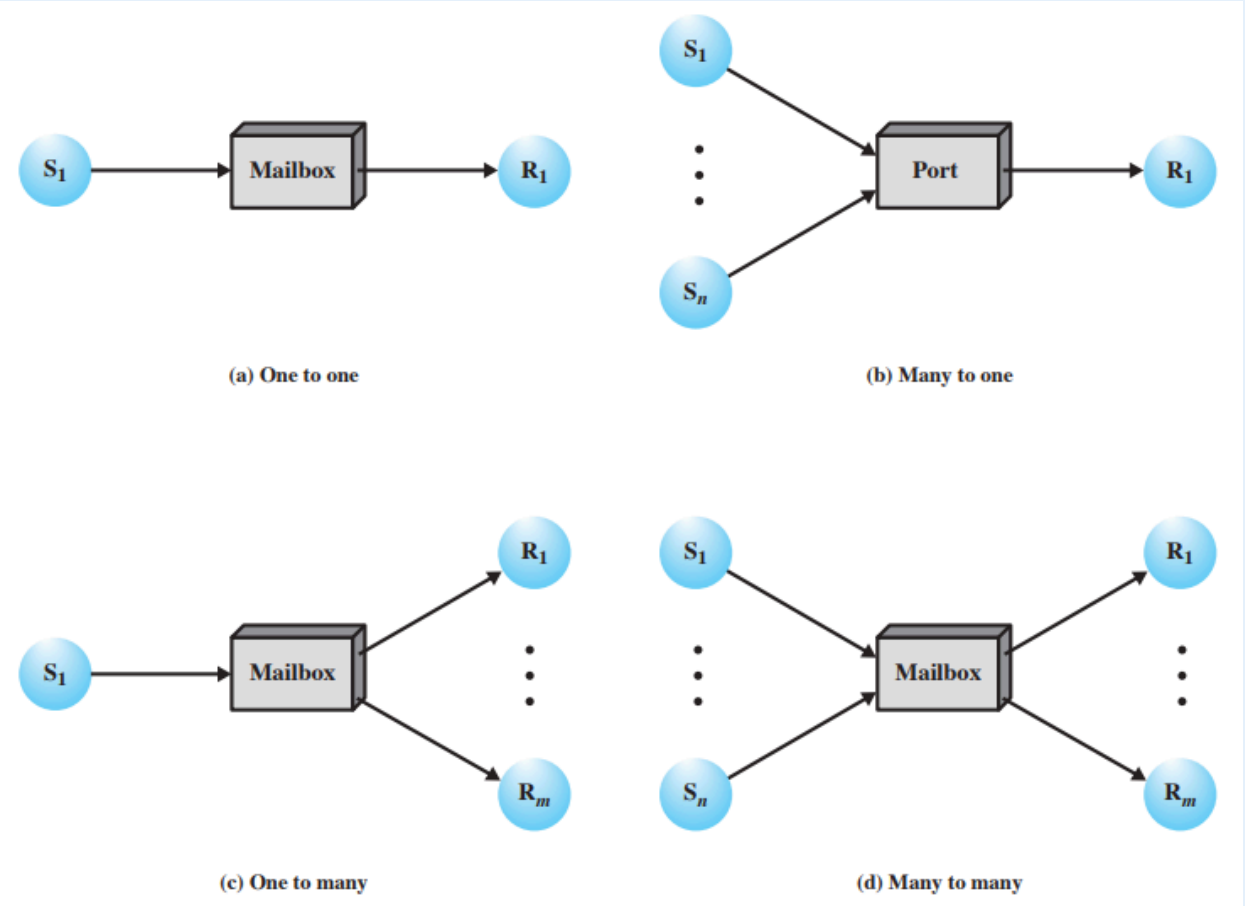
- Se è presente un messaggio, il processo lo riceve e continua l'esecuzione
- In caso contrario ci sono due possibilità:
  1. Si sospende finché il messaggio non è inviato
  2. Si continua l'esecuzione senza più ricevere il messaggio

Tipo di send e receive	**Comportamento**
Send e receive bloccanti	• Mittente e destinatario bloccati fino al completamento dell'operazione • Rendez-vous • Permette la sincronizzazione dei processi
Send non bloccante e receive bloccante	• Il mittente procede ma il destinatario è bloccato fino alla ricezione del messaggio • Combinazione più comune • Il mittente invia più messaggi a tanti destinatari senza perdita di tempo (es. Processo server che fornisce servizi a più processi)
Send e receive NON bloccanti	• Nessuno dei due processi attende

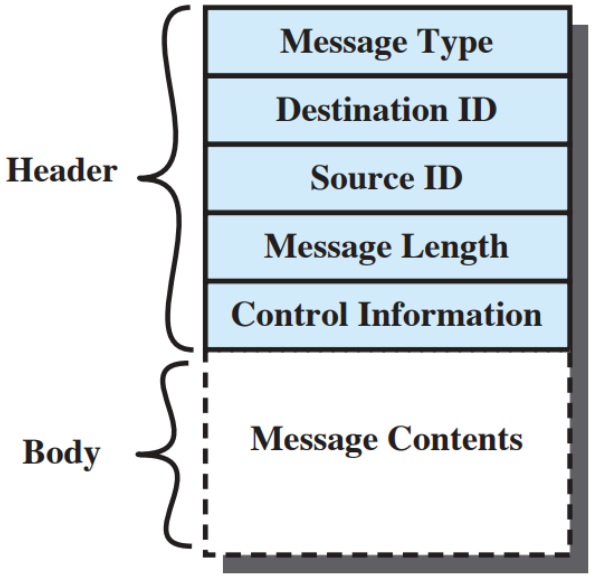
Indirizzamento:

1. *Diretto:*
   - La primitiva send include l'identificatore del processo destinatario
   - La primitiva receive può o specificare il mittente (utile per processi concorrenti cooperanti), oppure può usare l'indirizzamento implicito, quindi ricevere un messaggio in generale e il parametro sender conterrà l'identificatore del mittente.
2. *Indiretto:*
   - Messaggi inviati a una struttura dati condivisa consistente in code che contengono temporaneamente i messaggi (mailbox)
   - Il mittente invia un messaggio a una mailbox
   - Il destinatario preleva un messaggio dalla mailbox
   - Flessibilità

> **Indirizzamento indiretto**



Formato del messaggio:



Gestione delle code: Il meccanismo più semplice è il FIFO, il primo messaggio in coda viene trasmesso, oppure si può fare una coda prioritaria per tipo di messaggio o designata dal mittente, alternativamente il destinatario potrebbe ispezionare la coda e scegliere un messaggio particolare.

Avendo quindi a disposizione una receive bloccante, la mutua esclusione diventa molto più semplice degli algoritmi visti in precedenza. Si crea una mailbox e si inviano ad essa dei messaggi, i processi prima di iniziare la sezione critica eseguono un receive, se c'è il messaggio nella mailbox entrano nella sezione critica e terminata la sezione mandano un messaggio alla mailbox. Se un processo esegue la receive mentre l'altro processo è nella

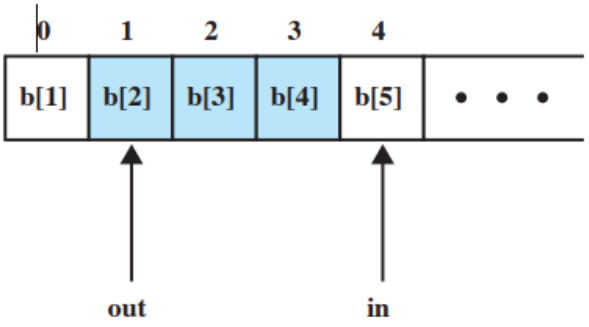
sezione critica troverà la mailbox vuota e perciò si bloccherà finché l'altro processo non uscirà dalla sezione critica e manderà un messaggio nella mailbox.

```
/* program mutualexclusion */
const int n = /* number of process */;
void P(int i) {
     message msg;
     while (true) {
          receive (box, msg);
          /* critical section */;
          send (box, msg);
          /* remainder */;
     }
}

void main() {
     create mailbox (box);
     send (box, null);
     parbegin (P(1), P(2),..., P(n));
}
```

Problema produttore/conduttore: Ci sono processi che consumano e processi che producono. Se ho uno o più produttori che inseriscono dati in un buffer, un consumatore estrae i dati dal buffer (uno alla volta). Un solo produttore o consumatore alla volta può accedere al buffer.

Bisogna quindi assicurare la mutua esclusione ed evitare che un produttore aggiunga dati a un buffer pieno e che un consumatore cerchi di estrarre dati da un buffer vuoto.



Soluzione errata con buffer infinito:

```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0; /*due semafori distinti per evitare che si legga in un buffer
vuoto ???????????*/

void producer() {
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
          /*ogni volta che si entra si fa una wait e ogni volta che si esce si fa una signal*/
     }
}

void consumer() {
     semWaitB(delay);
     while (true) {
```

```
            semWaitB(s);
            take();
            n--;
            semSignalB(s);
            consume();
            if (n==0) semWaitB(delay);
        }
    }
```

Abbiamo un semaforo binario che esegue la wait per realizzare la mutua esclusione. append() scrive l'evento prodotto nel buffer, poi n++ incrementa il contatore del numero di elementi nel buffer. Se n == 1, esegue un signal sull'altro semaforo, il quale prenderà la risorsa, decrementerà il contatore n e segnalerà con delay che il buffer è vuoto. Questo non funziona perché terminato semSignalB(s) il produttore potrebbe aver già prodotto qualcos'altro, quindi blocca l'accesso ad altri processi (n non sarà zero). Il problema è che la semWaitB() non può stare nel while true, ma altrimenti non funzionerebbe.

Soluzione corretta con buffer infinito:

```
/* program producerconsumer */
int n; binary_semaphore s = 1, delay = 0;

void producer() {
    while (true) {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1) semSignalB(delay);
        semSignalB(s);
    }
}

void consumer() {
    int m; /* a local variable */
    semWaitB(delay);
    while (true) {
        semWaitB(s);
        take();
        n--;
        m = n;
        semSignalB(s);
        consume();
        if (m==0) semWaitB(delay);
    }
}
```

La procedura del producer non cambia, il consumer invece crea m, copia in m il contatore n, consuma la risorsa e poi il check viene fatto non con n (che potrebbe essere stato modificato da producer), ma con m che ha salvato il valore di n prima che il consumer abbia consumato la risorsa.

Soluzione con semafori generici e buffer infinito (soluzione più semplice):

```
/* program producerconsumer */
semaphore n = 0, s = 1;

void producer() {
    while (true) {
        produce();
        semWait(s);
        append();
```

```
            semSignal(s);
            semSignal(n);
        }
    }

    void consumer() {
        while (true) {
            semWait(n);
            semWait(s);
            take();
            semSignal(s);
            consume();
        }
    }
```

Soluzione con semafori generici e buffer finito:

```
    /* program boundedbuffer */
    const int sizeofbuffer = /* buffer size */;
    semaphore s = 1, n= 0, e= sizeofbuffer;

    void producer() {
        while (true) {
            produce();
            semWait(e);
            semWait(s);
            append();
            semSignal(s);
            semSignal(n);
        }
    }

    void consumer() {
        while (true) {
            semWait(n);
            semWait(s);
            take();
            semSignal(s);
            semSignal(e);
            consume();
        }
    }
```

Il produttore produce, poi fa wait sul semaforo e, che prenderà valori negativi, quando il buffer sarà pieno, e wait su s per la mutua esclusione sull'accesso al buffer. Poi fa il signal a s per segnalare che ora il buffer è libero e segnala a n, incrementa quindi n di 1 dicendo che ora c'è un elemento in più nel buffer. Il consumer invece fa esattamente il contrario: facendo lo wait su n che se positivo indica che ci sono degli elementi nel buffer. Altrimenti il consumer si mette in coda, se c'è la risorsa prende la risorsa, segnala che ora e si è liberato di un elemento.

Monitor

Quando si hanno programmi complessi si possono utilizzare dei programmi costruiti apposta per la concorrenza. Questi hanno degli oggetti fatti apposta per la gestione dei dati condivisi. Il monitor è un costrutto di alcuni linguaggi di programmazione con funzionalità analoghe ai semafori usato per la programmazione concorrente (disponibile in molti linguaggi e molte librerie). Si tratta di un modulo software usato per gestire:

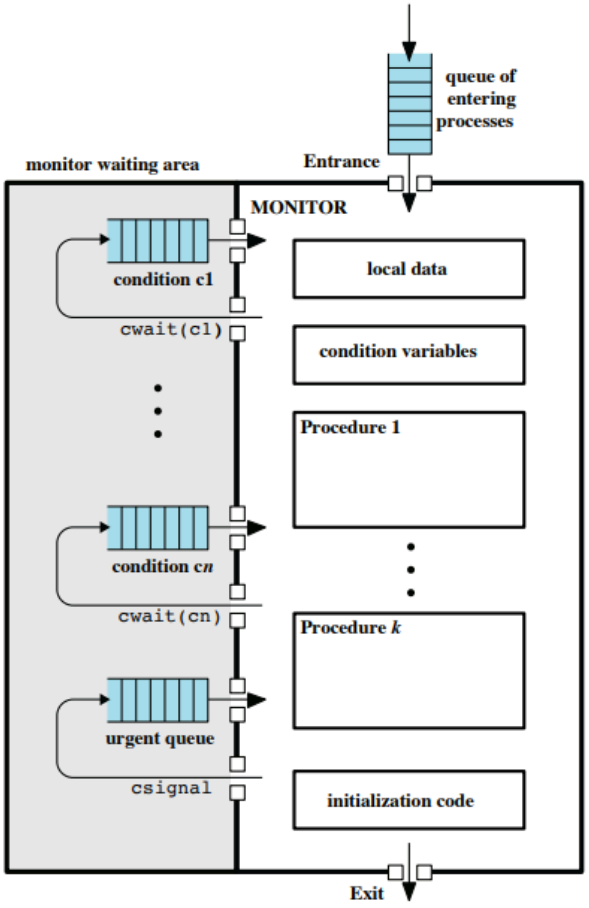
- I dati locali
- Le sequenze di inizializzazione

- Una o più procedure

I dati locali condivisi sono accessibili solo attraverso le procedure del monitor e in esso definisco le operazioni da fare su questi dati condivisi. Il resto dei programmi che usano la concorrenza sono chiamate alle procedure definite nel modulo. I singoli processi quindi non toccano direttamente i dati condivisi, ma devono passare per le procedure del modulo. I dati condivisi quindi sono accessibili solo tramite le procedure del monitor e i codici dei processi possono accedervi solo tramite le procedure. Inoltre un solo processo alla volta può eseguire le procedure del monitor.

All'interno del monitor stanno le variabili di condizione, variabili accessibili solo attraverso due funzioni:

1. *Cwait(c):* sospende l'esecuzione del processo che la ha eseguita sulla condizione *c*
2. *Csignal(c):* riattiva uno dei processi in attesa sulla condizione *c*; se non ce ne sono non fa nulla



Sintesi:

è un costrutto di sincronizzazione usato nella programmazione concorrente per gestire in modo sicuro l'accesso a risorse condivise tra thread o processi. Solo un thread alla volta può entrarci e dentro ci sono:

- variabili condivise
- procedure per accedervi
- variabili di condizione per sincronizzare i thread

In particolare servono per evitare race condition, mutua esclusione e coordinare i thread. Immagina il monitor come una stanza con una sola porta:>

- se un thread entra → gli altri devono aspettare
- quando esce → un altro può entrare

Quindi: l'accesso è mutualmente esclusivo ed è gestito automaticamente dal sistema. All'interno di un monitor trovo variabili condivise, funzioni che accedono ai dati e variabili di condizione che servono per aspettare eventi

Problema del produttore/consumatore:

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N]; /* space for N items*/
int nextin, nextout; /* buffer pointers*/
int count; /* number of items in buffer*/
cond notfull, notempty; /* condition variables for synchronization*/

void append (char x) {
     if (count == N) cwait(notfull); /* buffer is full; avoid overflow*/
     buffer[nextin] = x;
     nextin = (nextin + 1) % N;
     count++; /* one more item in buffer */
     csignal(notempty); /* resume any waiting consumer */
}

void take (char x) {
     if (count == 0) cwait(notempty); /* buffer is empty; avoid underflow*/
     x = buffer[nextout];
     nextout = (nextout + 1) % N;
     count--; /* one fewer item in buffer */
     csignal(notfull); /* resume any waiting producer */
}

{ /* monitor body */
     nextin = 0; nextout = 0; count = 0; /* buffer initially empty */
}
```

Problema dei lettori e scrittori

Alcuni processi leggono soltanto (lettori), altri scrivono soltanto (scrittori). Imponiamo come condizioni affinché tutto vada bene che:

- Qualsiasi numero di lettori può leggere simultaneamente i dati;
- Un solo scrittore alla volta può scrivere i dati;
- Durante la scrittura nessun lettore può leggere.

Precedenza ai lettori: Si ha un contatore, due semafori, il semaforo x per la mutua esclusione, e un semaforo wsem per controllare l'accesso degli scrittori. Se readcount == 1, cioè se c'è un lettore, esegue una wait sul semaforo wsem. Fatto ciò esco dalla mutua esclusione. La x in sostanza serve per la mutua esclusione sulla variabile readcount. Devo incrementare readcount, e per fare questa operazione ho tre istruzioni elementari: leggi readcount, aggiungici 1 e aggiorna. A questo punto il lettore legge e quando ha terminato decrementa readcount (la variabile in mutua esclusione). Quando verifica che lui è l'ultimo lettore, esegue la signal su wsem, quindi abilita o un processo scrittore o un prossimo lettore. Il processo scrittore può scrivere solo quando non ci sono lettori, quindi esegue semwait su wsem, scrive ed esegue la signal. Può scrivere il suo comando solo se è l'unico processo che sta accedendo. C'è un semWait all'interno di una sezione critica, sarebbe buona regola non lasciare dei wait all'interno delle sezioni critiche perché altrimenti se il processo si blocca nessuno può più accedervi, ma qui non fa danni perché la wait blocca il processo quando c'è un processo che sta scrivendo. Quindi finché il processo scrive è vero che tutti i lettori non possono accedere alla sezione critica, ma non potrebbero comunque accedere alla lettura. Precedenza ai lettori (con buffer infinito):

```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;

void reader() {
     while (true) {
          semWait (x);
          readcount++;
          if (readcount == 1) semWait (wsem);
          semSignal (x);
          READUNIT();
          semWait (x);
          readcount--;
          if (readcount == 0) semSignal (wsem);
          semSignal (x);
     }
}

void writer() {
     while (true) {
          semWait (wsem);
          WRITEUNIT();
          semSignal (wsem);
     }
}

void main() { readcount = 0; parbegin (reader, writer); }
```

Precedenza agli scrittori: Ho 3 semafori. Con semsignal(x) esco dalla mutua esclusione per questo tratto, poi semsignal(rsem) e semsignal(z). A questo punto, finita la lettura, segnalo semwait(x) per la mutua esclusione, decremento il contatore, e se il contatore è 0 eseguo una signal su wsem, e ciò significa che tutti i lettori hanno finito di leggere. Alla fine semsignal(x) conclude la mutua esclusione. Passando a void writer(), si ha una semWait(y) sul semaforo y per la mutua esclusione, poi incrementiamo la writecount. Se writecount == 1 significa che questo è l'unico processo che vuole scrivere, c'è una wait su rsem (semaforo dei lettori). Dopodiché si esce dalla sezione critica e si esegue una wait sul semaforo wsem che segnala se si può scrivere o meno. A questo punto si scrive e una volta terminata questa fase si segnala wsem, cioè un altro processo può accedere. Finiti gli scrittori, per cui questo era l'ultimo scrittore, segnala ai processi lettori di partire. Wsem inizialmente è 1, viene decrementato o quando accede un primo lettore o quando accede uno scrittore, quindi wsem sarà a 0 quando almeno un processo sta leggendo o un processo sta scrivendo. Ciò dunque assicura che non vi siano sovrapposizioni tra due processi scrittori o un processo scrittore e uno lettore. Il semaforo x serve per la mutua esclusione sulla variabile readcount. semwait e semsignal devono essere accessibili all'interno della mutua esclusione. Ciò significa che non posso mettere semwait(x) e basta o semwait(z) e basta, perché succederebbe che mentre un processo è fermo sulla semsignal, non sarebbe accessibile la lettura che invece deve essere accessibile perché altrimenti il processo in uscita non potrebbero aggiornarla e quindi far ripartire il processo.

```
/* program readersandwriters */
int readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;

void reader() {
     while (true) {
          semWait (z);
               semWait (rsem);
                    semWait (x);
                         readcount++;
                         if (readcount == 1) semWait (wsem);
                              semSignal (x);
```

```
                    semSignal (rsem);
               semSignal (z);
               READUNIT();
               semWait (x);
                    readcount--;
                    if (readcount == 0) semSignal (wsem);
                         semSignal (x);
     }
}

void writer () {
     while (true) {
          semWait (y);
               writecount++;
               if (writecount == 1)
                    semWait (rsem);
          semSignal (y);
          semWait (wsem);
               WRITEUNIT();
          semSignal (wsem);
          semWait (y);
               writecount--;
               if (writecount == 0)
                    semSignal (rsem);
          semSignal (y);
     }
}

void main() {
     readcount = writecount = 0; parbegin (reader, writer);
}
```

Scambio di messaggio: Un server, chiamato anche controller, gestisce le informazioni che vengono condivise. A sinistra si ha il programma del processo lettore e scrittore, il processo lettore ha una variabile rmsg con un messaggio che deve leggere il lettore e invia questo messaggio a una mailbox chiamata readrequest (un contenitore che conterrà tutte le richieste da parte dello scrittore). Fatto questo si mette in attesa sulla sua porta (mailbox indice i) e quando avrà ricevuto il messaggio di risposta dal controllo leggerà e poi invierà il messaggio lettura terminata in un'altra mailbox denominata finish. Il processo scrittore funziona allo stesso modo, con la differenza che invece di readrequest avrà writerequest. Se c'è una richiesta di lettura viene prelevato il messaggio, decrementato il contatore, e in risposta al messaggio ricevuto si invia un messaggio di ricezione. Quando il contatore arriva a 0 si manda allo scrittore il messaggio OK e ci si mette in attesa del messaggio di terminata scrittura, dopodiché il contatore viene portato a 100. Al termine si ha un while(count <0), finché il count è <0 ci si mette in attesa dei messaggi di lavoro terminato e mano mano si incrementa il contatore. Quindi in questo modo il diritto di accesso alla lettura e alla scrittura la dà un processo ben preciso, ovvero il controllore, e non c'è più la necessità di realizzare la mutua esclusione poiché è il controllore stesso che gestisce il tutto. In questo senso i meccanismi operativi basati su meccanismi client server sono più sicuri, un solo processo che sbaglia le operazioni di mutua esclusione rendono il dato condiviso

```
void reader(int i) {
     message rmsg;
     while (true) {
          rmsg = i;
          send (readrequest, rmsg);
          receive (mbox[i], rmsg);
          READUNIT ();
          rmsg = i;
          send (finished, rmsg);
     }
```

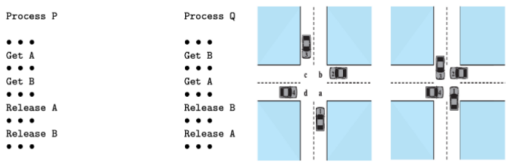
```
}

void writer(int j) {
     message rmsg;
     while(true) {
          rmsg = j;
          send (writerequest, rmsg);
          receive (mbox[j], rmsg);
          WRITEUNIT ();
          rmsg = j;
          send (finished, rmsg);
     }
}

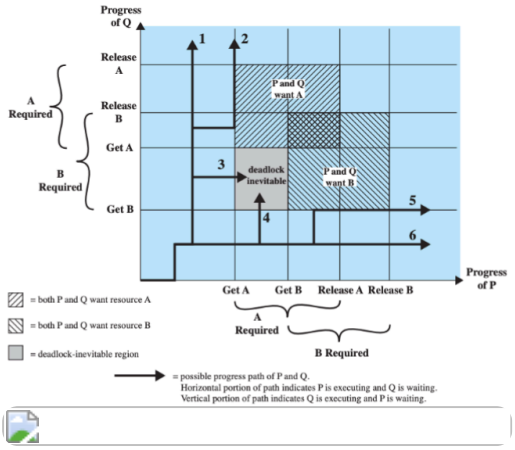
void controller () {
     while (true) {
          if (count > 0) {
               if (!empty (finished)) {
                    receive (finished, msg);
                    count++;
               }
               else if (!empty (writerequest)) {
                    receive (writerequest, msg);
                    writer_id = msg.id;
                    count = count – 100;
               }
               else if (!empty (readrequest)) {
                    receive (readrequest, msg);
                    count--;
                    send (msg.id, "OK");
               }
          }
          if (count == 0) {
               send (writer_id, "OK");
               receive (finished, msg);
               count = 100;
          }
          while (count < 0) {
               receive (finished, msg);
               count++;
          }
     }
}
```

Stallo

Definizione: un insieme di processi è in stallo (*deadlock*) se ogni processo è in attesa di un evento che può essere generato solo da un altro processo dell'insieme, nel caso generale non c'è una soluzione efficiente per la deadlock



Se siamo sfortunati il processo P preleva A mentre il processo Q preleva B, quando poi i due cercano di prelevare rispettivamente B e A entrambe le risorse sono già occupate, ma per essere rilasciate dai processi si deve passare il punto dove si prendono B e A, perciò oltre ai processi che rimarranno in stallo all'infinito, anche le risorse A e B saranno occupate all'infinito creando un gran problema.



La deadlock avviene se una delle due frecce si interseca con una delle aree tratteggiate. Da questa situazione, come scritto la deadlock è inevitabile se si entra nell'area grigia poiché poi le frecce andranno per forza nelle aree tratteggiate

In questa foto invece è sempre possibile evitare la deadlock facendo in modo che il processo P prenda A e la rilasci prima di prendere B e lo stesso faccia il processo Q con la risorsa B prima di prendere la A. Per i processi concorrenti non c'è modo di prevedere chi avanzerà per primo e chi per secondo, se la deadlock succede ci si ritrova in una situazione complicata.

Categorie di risorse:

1. *Risorse riutilizzabili:* utilizzabili da un processo alla volta e non vengono distrutte dal loro utilizzo (una volta finito il processo vengono restituite). Per esempio sono processore, canali I/O, memoria, files, semafori
2. *Risorse consumabili:* possono essere create (prodotte) e distrutte (consumate), come le interruzioni, i segnali, i messaggi, i dati in buffer ecc…

Esempio risorse riutilizzabili:

Due processi competono per l'uso di un file D e un dispositivo T, P1 richiede prima D e lo blocca, P2 richiede prima T e lo blocca, poi però i processi richiedono subito rispettivamente T e D, che però ora sono bloccati dall'altro processo, perciò ora ci troviamo nella situazione di stallo descritta prima.

P1	P2
Request (D)	Request (T)
Lock (D)	Lock (T)
Request (T)	Request (D)
Lock (T)	Lock (D)
Perform function	Perform function
Unlock (D)	Unlock (T)
Unlock (T)	Unlock (D)

Se però si invertisse l'ordine in cui sono richieste T e D in modo tale che entrambi i processi richiedono le risorse T e D nello stesso ordine, così P1 prende subito sia T che D, fa ciò che deve fare e sblocca le risorse a P2 che aveva trovato T subito occupato ed era rimasto perciò in attesa.

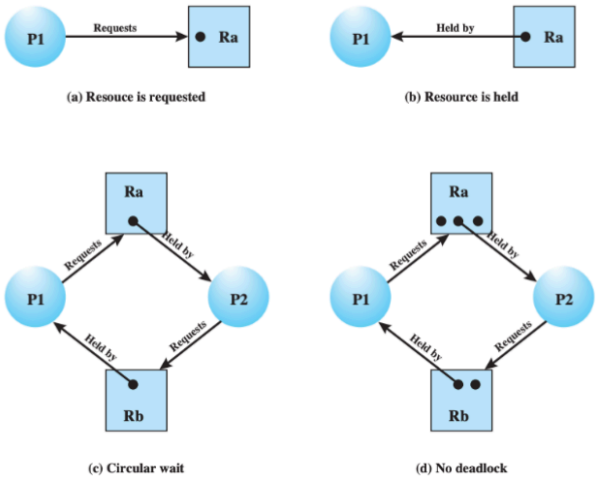
Esempio risorse consumabili:

Due processi si scambiano messaggi:

```
  P1                          P2
• • •                       • • •
Receive(P2)                 Receive(P1)
• • •                       • • •
Send(P2,M1)                 Send(P1,M2)
```

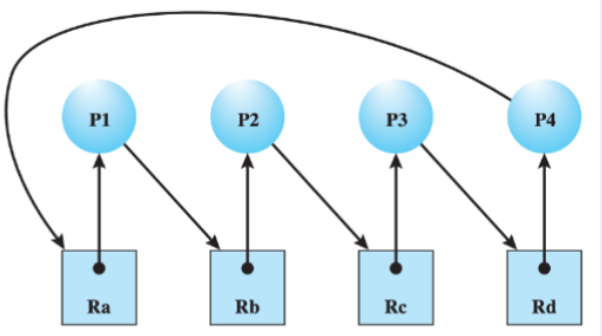
Receive è una funzione bloccante, P1 attende che P2 mandi un messaggio,ma allo stesso tempo P2 attende che P1 mandi un messaggio, quindi attendono messaggi che non arriveranno mai poiché entrambi in attesa.

Grafo di allocazione delle risorse:



Example

L'esempio dell'incrocio delle auto che non sanno chi debba passare per primo può essere rappresentato così:



Condizioni per lo stallo: le condizioni necessarie sono:

1. Mutua esclusione: le risorse possono essere assegnate a un solo processo alla volta
2. Possesso della risorsa e l'attesa che questa si liberi
3. Assenza di pre-rilascio

La condizione determinante per lo stallo è l'attesa circolare (4), cioè che esista una catena chiusa di processi tale che ogni processo possiede una risorsa richiesta dal processo successivo (quindi che si abbia un ciclo nel grafo di Holt, il grafo delle risorse).

Info

- $1, 2, \implies$ stallo POSSIBILE, cioè se si hanno solo le condizioni necessarie
- $1, 2, , \implies$ stallo ESISTENTE, cioè se si ha anche la quarta condizione

Azioni contro lo stallo:

- *Prevenzione:* far si che i processi si comportino in maniera tale da evitare lo stallo
- *Esclusione:* meccanismi che permettono al sistema operativo di controllare se assegnare una risorsa a un dato processo può causare lo stallo, nel qual caso questa assegnazione viene rifiutata o posticipata
- *Rilevamento:* non lo prevediamo, non lo escludiamo, ma quando temiamo che esso si sia verificato cerchiamo di rilevarlo e di prendere provvedimenti

Prevenzione: si progetta il sistema in modo da escludere lo stallo. Ci sono metodi indiretti, che riguardano l'esclusione di una delle tre condizioni necessarie per lo stallo e i metodi diretti, che sono atti a prevenire il presentarsi dell'attesa circolare.

- *Metodi indiretti:*
  - *Mutua esclusione:* impossibile disabilitarla per risorse seriali. Possibile solo in rari casi (e.g.file in lettura dove tutti i processi possono accedere contemporaneamente al file)
  - *Possesso e attesa:* si può imporre ai processi un'unica richiesta globale di tutte le risorse, sospendendolo finché non le ha ottenute tutte simultaneamente (*assegnazione statica*). Questo metodo funziona ma è inefficiente poiché molti processi restano sospesi in attesa delle risorse e possono esserci risorse assegnate e non utilizzate.
  - *Assenza di prerilascio:* si impone ai processi in attesa di una risorsa di rilasciare quelle che possiedono, oppure il sistema pre rilascia il processo che usa la risorsa richiesta e gli impone di rilasciare la risorsa utilizzata (*priorità*). Anche questo però non è sempre realizzabile, lo è solo se lo stato della risorsa può essere salvato e ripristinato.
- *Metodi diretti:* si può evitare l'attesa circolare tramite l'*assegnazione ordinata*:
  - Si definisce un ordine totale sulle categorie di risorse
  - Un processo cui siano state già assegnate delle risorse, può richiedere solo risorse di categorie successive a quelle delle risorse assegnate

Poiché in caso di attesa circolare, un processo ha una risorsa e chiede una risorsa e un altro processo ha la risorsa e chiede , allora uno dei due sta violando il protocollo. Il primo se , il secondo se . Anche se il metodo diretto dell'assegnazione ordinata funziona, esso è inefficiente. Se l'ordinamento è coerente e tiene conto dell'ordine in cui le risorse vengono usate (ad esempio scrivere dati su disco, prima ho bisogno della tastiera e poi ho bisogno del disco, raramente viceversa) è molto meno inefficiente dell'assegnazione statica.

Prevenire/Evitare lo stallo:

- *Prevenire (deadlock prevention):* Limitazioni sulle richieste di risorse
- *Evitare (deadlock avoidance):* Limitazione sulle assegnazioni di risorse

Ho 2 approcci possibili:

1. Rifiuto di esecuzione per i processi le cui richieste potrebbero determinare stallo
2. Rifiuto di allocazione per le richieste di risorse che potrebbero determinare stallo

In entrambi i casi è necessario conoscere le future richieste dei processi

C'è bisogno quindi di un algoritmo detto **algoritmo del banchiere** che usa una serie di informazioni:

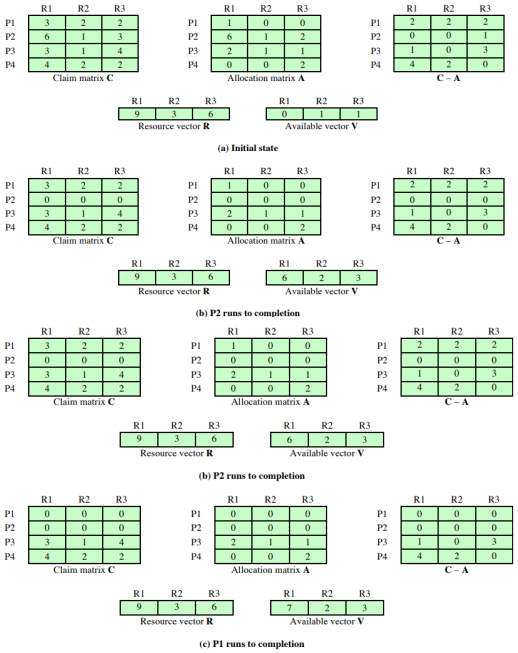
- Il vettore risorse, la quantità totale di ciascuna risorsa nel sistema;
- Il vettore disponibilità, indica quante di quelle risorse sono disponibili;
- Una matrice di richieste, che per ogni riga (che rappresenta un processo), mi dice quante risorse sta usando quel processo (le colonne sono il tipo di risorsa);
- Una matrice allocazione, dove ciascuna riga è sempre un processo che contiene l'allocazione attuale della risorsa della colonna per il processo della riga in cui ci troviamo.

Resource $= \mathbf{R} = (R_1, R_2, \ldots, R_m)$	Total amount of each resource in the system
Available $= \mathbf{V} = (V_1, V_2, \ldots, V_m)$	Total amount of each resource not allocated to any process
Claim $= \mathbf{C} = \begin{pmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{22} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ C_{11} & C_{12} & \cdots & C_{1m} \end{pmatrix}$	C_{ij} = requirement of process i for resource j
Allocation $= \mathbf{A} = \begin{pmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ A_{11} & A_{12} & \cdots & A_{1m} \end{pmatrix}$	A_{ij} = current allocation to process i of resource j

$$R_j = V_j + \sum_{i=1}^{n} A_{ij}\,, \qquad C_{ij} \leq R_j\,, \qquad A_{ij} \leq C_{ij}\,.$$

Le 3 equazioni/disequazioni mostrano le relazioni fra questi dati definiti sopra: se io sommo le risorse attualmente usate in un processo e ci aggiungo le risorse disponibili rimaste inutilizzate ottengo , cioè la quantità disponibile di quella risorsa, la seconda indica che chiaramente un processo non può richiedere più di quanto c'è al massimo di una risorsa, la terza indica chiaramente che in ogni momento un processo deve avere una quantità di risorse di ogni tipo minore o uguale di quella che ha dichiarato necessaria. Questi quindi sono i dati del problema, allora un possibile approccio è il seguente: inizia un nuovo processo $_{n+1}$ solo se per ogni , $\geq {}_{n+1,} + {}_{=1}$, cioè se ci sono risorse sufficienti per soddisfare tutti i processi. Questo però è inefficiente poiché assume che tutte le richieste di tutti i processi debbano essere soddisfatte simultaneamente. È più efficiente l'algoritmo del banchiere.

Algoritmo del banchiere: L'algoritmo tiene conto dello stato del sistema, quindi i vettori sopra definiti e e le matrici e , l'algoritmo controlla se esiste almeno una sequenza di allocazioni che porta a termine tutti i processi e rifiuta una richiesta se porta in uno stato insicuro per evitare lo stallo. Come determino se uno stallo è sicuro o meno?



Abbiamo 4 processi con le loro specifiche matrici A e C, la matrice C-A sono le risorse aggiuntive di cui avrà bisogno ogni processo per terminare, dopo questo nel vettore delle risorse disponibili mi saranno rimaste 0 1 1, la domanda è: con queste risorse rimaste posso terminare un processo già esistente? Si, in questo caso il processo p2 a cui manca solo 1 della risorsa R3 per terminare, perciò l'assegnamo, termina p2 e le risorse allocate a p2 si azzerano e tornano nel vettore V. Con le nuove risorse ottenute posso chiudere anche il processo 1 e così via andrò poi a chiudere anche gli altri processi, lo stato è quindi sicuro.

Rilevazione di uno stato sicuro:

```
W <- V
while c'è un processo P_i non spuntato, tale che C_ij - A_ij <= W_j per ogni j spunta P_j
     for all j
          W_j <- W_j + A_ij
if tutti i processi sono spuntati
     return SICURO
else
     return INSICURO
```

Algoritmo del banchiere:

```
struct state{
     int resource[m];
     int available[m];
     int claim[n][m];
     int alloc[n][m];
}

if (alloc[i,*] + request[*] > claim[i,*])
     <error>; //total request > claim
else if (request[*] > available[*])
     <suspend process>;
else{
     <define newstate by:
     allc[i,*] = alloc[i,*] + request[*];
     available[*] = available[*] - request[*] >;
}

if (safe(newstate))
     <carry out allocation>;
else{
     <restore original state>;
     <suspend process>;
}
```

Lo stato è la struttura dei dati disponibili sui processi e le risorse allocate, il primo if controlla se c'è una richiesta di risorse maggiore a quelle veramente necessarie dichiarate nel claim. Il secondo if controlla che la richiesta non sia maggiore della quantità disponibile di quella risorsa. Passati questi due controlli definisco un nuovo stato cercando di allocare ad un processo le risorse necessarie e se il nuovo stato è safe mando a buon fine l'allocazione, altrimenti torno allo stato originale e sospendo il processo.

Come controllo che uno stato è safe?

```
boolean safe(state S){
     int currentvail[m];
     process rest[<number of processes>];
     currentvail = available;
     rest = {all processes};
     possible = true;
     while(possible){
          <find a process Pk in rest così che claim[k,*] -                alloc[k,*] <= current
vail;
          if(found){ //simula l'esecuzione di Pk
               currentvail = currentvail + alloc[k,*];
               rest = REST - {pK};
          }
          else possible = false;
     }
```

```
    return (rest == null)
}
```

La procedura safe cerca nello stato che gli viene fornito se c'è un processo che può essere completato tramite le risorse disponibili attualmente, se lo trova simula l'esecuzione di e lo toglie dall'insieme dei processi.

Vantaggi e limiti dell'algoritmo del banchiere per evitare lo stallo:

- *Vantaggi:*
  - Non richiede prerilascio e ripristino del processo
  - È meno restrittivo della prevenzione
- *Limitazioni:*
  - I processi devono stabilire in anticipo le risorse necessarie
  - I processi devono essere indipendenti
  - Risorse allocabili in un numero fisso
  - Nessun processo può terminare prima di aver rilasciato le risorse

Rilevare lo stallo: Nei casi in cui si previene o si evita del tutto lo stallo l'accesso alle risorse è limitato a causa delle restrizioni imposte a processi e assegnazioni per evitare lo stallo, quando invece lo stallo si rileva solo quando si verifica (deadlock detection) le risorse sono assegnate senza restrizioni. Sono periodiche le verifiche dell'assenza di attesa circolare. I vantaggi della rilevazione sono:

- Che la rilevazione dell'attesa circolare permette un ripristino precoce
- L'algoritmo è molto semplice

Lo svantaggio della rilevazione è che:

- Le verifiche vanno fatte frequentemente e consumano molto tempo di elaborazione.

Algoritmo di rilevazione dello stallo: i dati del problema sono essenzialmente quelli necessari all'algoritmo del banchiere:

- La matrice di allocazione $=$
- Il vettore delle risorse disponibili $=$
- Una matrice delle richieste $=$

```
W <- V
spunta tuttii processi P_i tali che A_ij = 0 per ogni j
while c'è un processoP_i non spuntato tale che Q_ij <= W_j per ogni j spunta P_i
    for all j
        W_j <- W_j + A_ij
```

Al termine dell'algoritmo gli eventuali processi non spuntati sono in stallo.

Example

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix Q

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix A

R1	R2	R3	R4	R5
2	1	1	2	1

Resource vector

R1	R2	R3	R4	R5
0	0	0	0	1

Available vector

Nell'esempio prima controlliamo se ci sono alcuni processi Pi tali che Aij = 0 in tutte le colonne, in questo caso P4 ha 0 risorse allocate e quindi viene spuntato, poi vediamo che sulla riga di P3, per ogni j gli elementi Qij sono <= Wj, quindi possiamo

spuntare anche P3, le cui risorse allocate vengono liberate e assegnate a W che diventa (0,0,0,1,1), poi però guardando P2 e P1 entrambi non possono essere spuntati, dunque alla fine rimangono P1 e P2 non spuntati e quindi c'è lo stallo.

Approach	Resource Allocation Policy	Different Schemes	Major Advantages	Major Disadvantages
Prevention	Conservative; undercommi-ts resources	Requesting all resources at once	• Works well for processes that perform a single burst of activity • No preemption necessary	• Inefficient • Delays process initiation • Future resource requirements must be known by processes
		Preemption	• Convenient when applied to resources whose state can be saved and restored easily	• Preempts more often than necessary
		Resource ordering	• Feasible to enforce via compile-time checks • Needs no run-time computation since problem is solved in system design	• Disallows incremental resource requests
Avoidance	Midway between that of detection and prevention	Manipulate to find at least one safe path	• No preemption necessary	• Future resource requirements must be known by OS • Processes can be blocked for long periods
Detection	Very liberal; requested resources are granted where possible	Invoke periodically to test for deadlock	• Never delays process initiation • Facilitates online handling	• Inherent preemption losses

Preempts = interruzioni forzate dei processi

Strategie di ripristino:

- Terminare tutti i processi in stallo
- Ripristinare tutti i processi in stallo a uno stato precedente (**checkpoint**), questo però necessita l'uso di meccanismi di salvataggio e riattivazione che rallentano l'esecuzione e che inoltre rischiano di riproporre lo stallo
- Terminare un processo alla volta finché lo stallo non scompare (applicando una politica di costo minimo elencate sotto)
- Revocare una risorsa alla volta finché lo stallo non scompare (sempre applicando una politica di costo minimo)

Alcune politiche seguono:

- Minimo tempo di elaborazione consumato (che usa meno CPU)
- Minimo output prodotto
- Massimo tempo di elaborazione rimanente (stimato)
- Minima quantità di risorse allocate
- Priorità

Strategia integrata (soluzione migliore): Una strategia non deve per forza prendere uno dei metodi e usarlo per tutto, l'idea è di raggruppare le risorse in classi, usare l'assegnazione ordinata per prevenire l'attesa circolare fra classi di risorse e in ogni classe utilizzare il metodo più appropriato.

Example

Classi di risorse:

- spazio di swap
- memoria principale
- risorse dispositivi
- canali di I/O

Assegnate nell'ordine indicato.

- spazio di swap: assegnazione statica o rifiuto di esecuzione
- memoria principale: prevenzione tramite prerilascio

- risorse dispositivi: algoritmo del banchiere o assegnazione ordinata
- canali di I/O: assegnazione ordinata

Sintesi algoritmo del banchiere:

Idea di base: Il sistema si comporta come una banca:

- ogni processo = un cliente
- le risorse = soldi
- ogni processo dichiara quanto al massimo potrà chiedere

Il sistema concede risorse solo se rimane in uno stato sicuro. L'obiettivo è di evitare il deadlock, cioè una situazione in cui: tutti i processi sono bloccati e nessuno può proseguire. Nello specifico se un processo Pi chiede risorse:

1. controllo base: Request[i] <= Need[i] ? Se no → errore (sta chiedendo troppo)
2. controllo disponibilità: Request[i] <= Available ? Se no → aspetta
3. simulazione: Fingi di assegnare: Available -= Request[i], Allocation[i] += Request[i], Need[i] -= Request[i]. Poi esegui **l'algoritmo di sicurezza**.
4. Se stato sicuro → concedi; Se NON sicuro → annulla tutto (rollback)

Gestione della memoria

Compiti della gestione della memoria: Un programma per essere eseguito va caricato in memoria, i principali compiti sono:

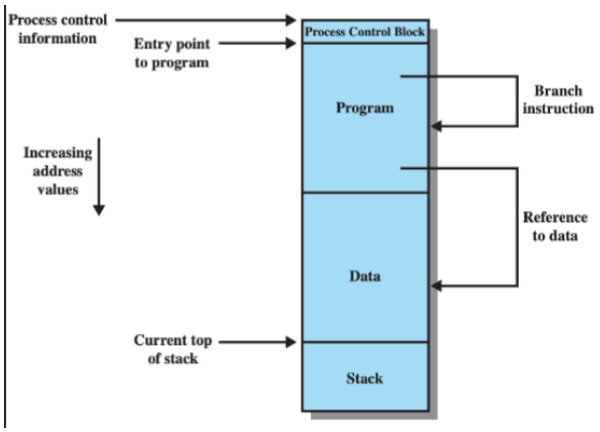
- *Rilocazione:* il nostro programma viene copiato nella memoria centrale, ma a priori non sappiamo in quale parte venga caricato. Per questo, nel nostro eseguibile, gli indirizzi in cui andare a prelevare le istruzioni e i dati non ci possono stare, saranno indirizzi simbolici (logici). Una volta caricati, questi indirizzi simbolici devono essere convertiti in indirizzi fisici
- *Protezione:* il processo preleva l'istruzione, la esegue, scrive i risultati in memoria e passa all'istruzione successiva, quando fa queste operazioni è fondamentale che non esca dalla memoria assegnata al processo.
- Condivisione
- Organizzazione logica
- Organizzazione fisica

Rilocazione: il programmatore non sa quali altri programmi saranno presenti nella memoria principale del calcolatore al momento dell' esecuzione. Deve essere possibile trasferire i programmi della memoria principale (interna al computer come RAM o ROM, la CPU vi accede rapidamente per eseguire programmi attivi) alla memoria secondaria (hard disk, SSD e unità flash, serve per l'archiviazione permanente dei dati e dei programmi a computer spento) e viceversa (**swapping**). Pertanto è impossibile:

- Specificare in che parte della memoria sarà caricato il processo
- Richiedere che a ogni trasferimento sia caricato nella stessa posizione

Il processo può essere rilocato in un'area di memoria differente. L'**entry point** è la prima istruzione del programma (dalla quale il programma inizia), poi c'è il programma, poi c'è una parte riservata ai dati dove si salta quando nel programma ci si riferisce a quei dati. L'istruzione che fa il riferimento conterrà un indirizzo simbolico al dato richiesto. Questo è il modello più semplice di programma, dopodiché questo oggetto sarà trasferito in

memoria principale, dove l'istruzione 0 corrisponderà a un certo indirizzo fisico della RAM, l'indirizzo 1 a quella successiva e così via.



Protezione: I riferimenti di cui parlavamo prima alla memoria servono per leggere e scrivere i dati. Come detto non sappiamo l'effettiva locazione del programma in memoria, quindi solo quando tentiamo di accedere un indirizzo possiamo realmente vedere se quell'indirizzo fa parte del processo, quindi i meccanismi di rilocazione devono anche contenere un supporto di protezione.

Condivisione: in alcuni sistemi è fortemente limitata, in altri è supportata. Ci sono programmi che vengono eseguiti da più processi, quindi in questo caso è utile permettere a questi processi di accedere entrambi alla stessa copia del codice. Il gestore della memoria quindi deve permettere un accesso controllato delle aree di memoria condivisa, senza compromettere la protezione. I meccanismi che supportano la rilocazione supportano anche la condivisione.

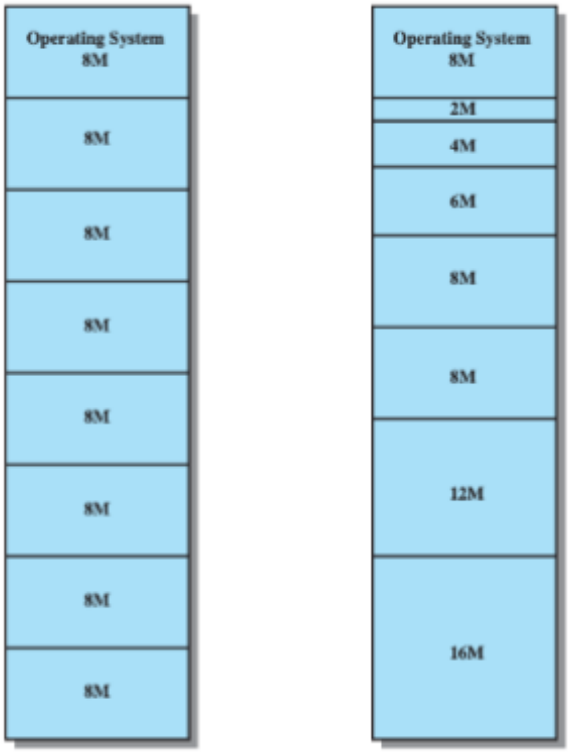
Organizzazione logica: la memoria principale è organizzata linearmente, i programmi sono scritti in moduli. L'**organizzazione logica modulare** permette che i moduli vengano scritti e compilati indipendentemente, possono avere differenti gradi di protezione (sola lettura o sola scrittura) e la condivisione a livello di moduli è più naturale dal punto di vista dell'utilizzatore. Lo strumento che meglio soddisfa questi requisiti è la **segmentazione**

Organizzazione fisica: la differenza sostanziale tra la memoria principale (RAM) del sistema e la memoria secondaria (il disco rigido nel PC), è che la memoria principale è volatile, veloce e costosa, la memoria secondaria invece è permanente, lenta, ma più economica quindi più accessibile a tutti. La memoria disponile per programma e dati potrebbe essere insufficiente e il programmatore non sa quanto spazio sarà disponibile. In altre parole il programma da eseguire deve stare almeno parzialmente nella RAM, perché altrimenti sarebbe troppo lento accedervi istruzione per istruzione. Il problema sta nel fatto che potrebbe non esserci RAM sufficiente nel programma. Un motivo è la multiprogrammazione in quanto potremmo avere tanti processi in avanzamento. Un altro motivo è il fatto che alcuni programmi potrebbero richiedere più spazio di quello che abbiamo disponibile. Una possibilità è quella dell'**overlaying:** la stessa regione è assegnata a vari moduli sotto la responsabilità del programma. Meglio lasciare al sistema la responsabilità di spostare le informazioni fra memoria principale e secondaria.

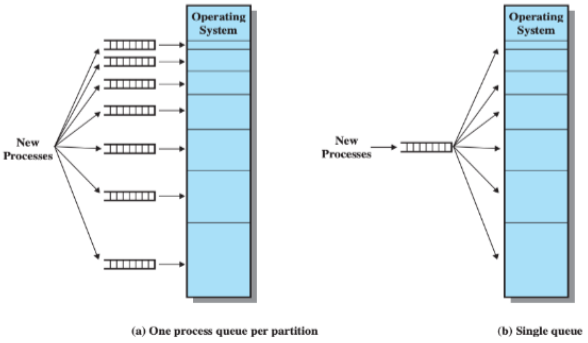
Il gestore della memoria carica i processi nella memoria principale per essere eseguiti:

- Realizza la memoria virtuale
- Si basa su partizionamento, segmentazione e paginazione

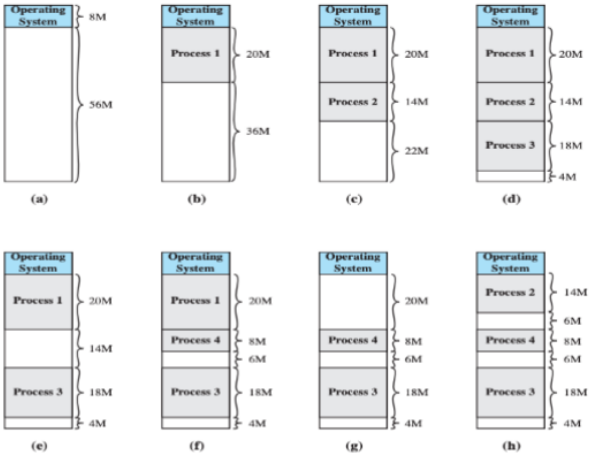
Partizionamento: È un metodo perlopiù obsoleto e che non prevede memoria virtuale, usa le partizioni fisse e le partizioni variabili. Si carica il sistema operativo nella RAM e ciò che resta di RAM viene diviso in partizioni uguali, poi carico il programma e si cercherà una delle partizioni con grandezza minore o uguale a quella richiesta dal programma.



Nell'esempio sulla prima colonna abbiamo tutte partizioni da 8 mega, quindi possiamo far partire solo programmi da 8 mega, nella seconda invece abbiamo quantità differenti. I problemi di questo metodo stanno proprio nel fatto che un programma potrebbe essere troppo lungo per entrare in una partizione e quindi si usa fer forza l'overlay. Inoltre comporta un utilizzo inefficiente della memoria principale poiché ogni programma occupa un'intera partizione e quindi si limita anche il numero di processi attivi al numero di partizioni che abbiamo. Si crea anche una **frammentazione interna**, cioè uno spreco di spazio poiché il programma può richiedere meno memoria di quella della partizione. Per allocare un processo ad una partizione o si crea una coda per ogni processo, oppure metto il processo in una singola coda che guarda tutte le partizioni. L'algoritmo di allocazione è:



Segmentazione: Un'alternativa sono le **partizioni dinamiche**, cioè le partizioni vengono create dinamicamente quando un processo è caricato in memoria. La RAM, quella che non è occupata dal sistema operativo, è un blocco unico che viene allocato man mano.

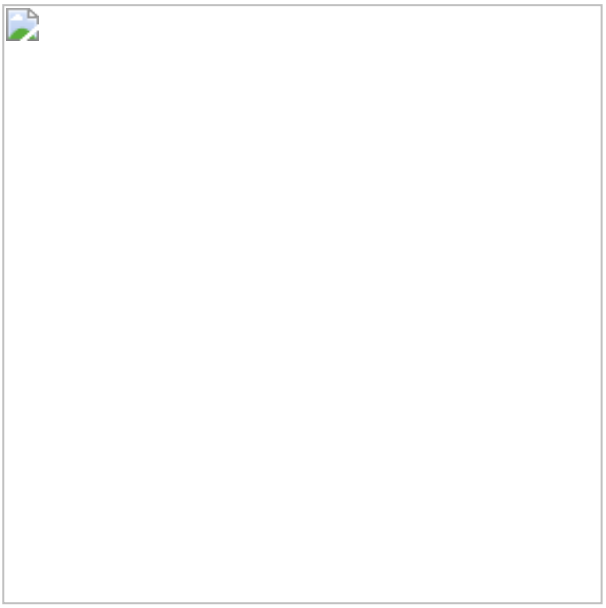


Questo, come si vede nella foto, provoca una **frammentazione esterna** che può essere risolta però con una compattazione della RAM che viene liberata man mano che i processi finiscono.

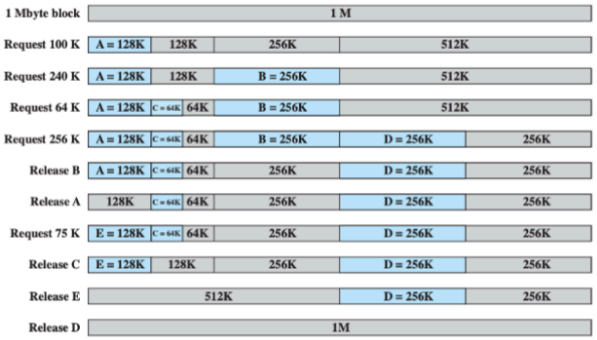
Per il metodo di allocazione si possono avere 3 politiche di scelta del blocco:

- *Best-fit:* sceglie il blocco disponibile che ha la dimensione più simile a quella richiesta
- *First-fit:* sceglie il primo blocco di dimensione sufficiente, a partire dall'inizio della memoria
- *Next-fit:* sceglie il primo blocco sufficiente a partire dall'ultima partizione creata

Il best-fit può sembrare il metodo migliore però con esso si creano un sacco di partizioni molto piccole, la first-fit invece tutto sommato funziona meglio, però con questa si cerca di usare sempre la prima parte della memoria, si potrebbe creare quindi una frammentazione concentrata sulla prima parte della memoria.



Buddy system: È un compromesso tra partizioni fisse e dinamiche ed è più facile da gestire. Si hanno blocchi di dimensione $2, L$, dove 2 è lo spazio allocabile, $2L$ è la dimensione dei blocchi allocabili. Un programma di dimensione con 2^1 2 viene allocato in un blocco libero di dimensione 2, se non ce ne sono, un blocco di dimensione 2^{+1} è spezzato in due buddies di dimensione 2. Non appena due buddies sono contemporaneamente liberi sono riuniti in un unico blocco di dimensione doppia.



Gli indirizzi all'interno del nostro programma possono essere logici, ossia riferimenti a una locazione di memoria indipendente dall'assegnazione della memoria principale al processo, quindi il vero indirizzo del dato non è conosciuto al programmatore. L'indirizzo fisico, invece, è l'effettiva posizione del dato nella memoria principale e l'indirizzo relativo è l'indirizzo logico espresso come posizione relativa a un altro indirizzo noto.

In sostanza:

Il **buddy system** è un algoritmo di gestione della memoria che serve ad allocare e liberare blocchi in modo veloce, riducendo la frammentazione. L'idea base è che la memoria viene vista come un blocco di dimensione 2 (es. 1024, 2048, ecc.). Quando serve memoria:

- si divide il blocco in **due metà uguali**
- e si continua a dividere finché non ottieni una dimensione adatta (con la potenza di 2 più piccola $\geq$ della richiesta)

Ogni split crea due blocchi della stessa dimensione detti buddy (gemelli) che possono essere riuniti. Infatti, quando libero/dealloco un blocco, controllo il suo buddy e se questo è libero unisci i due e ripeto tale processo verso l'alto. Il più grande problema di questo sistema risiede nella frammentazione interna e quindi nella quantità di byte che vanno sprecati quando alloco qualcosa in memoria

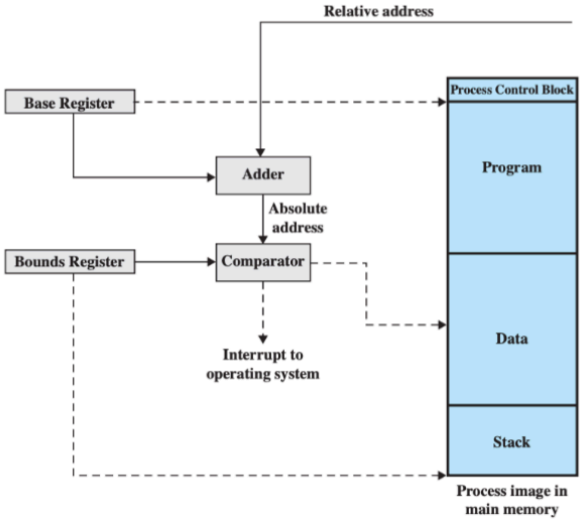
Attenzione:

- *Frammentazione interna:* Succede quando viene assegnata più memoria di quella realmente richiesta. Lo spazio “in più” resta inutilizzato dentro il blocco allocato (spreco **interno** al blocco)
- *Frammentazione esterna:* Succede quando la memoria libera è divisa in tanti pezzi piccoli non contigui. Anche se la memoria totale è sufficiente, non si riesce ad allocare perché non è tutta attaccata (spreco **tra** i blocchi)

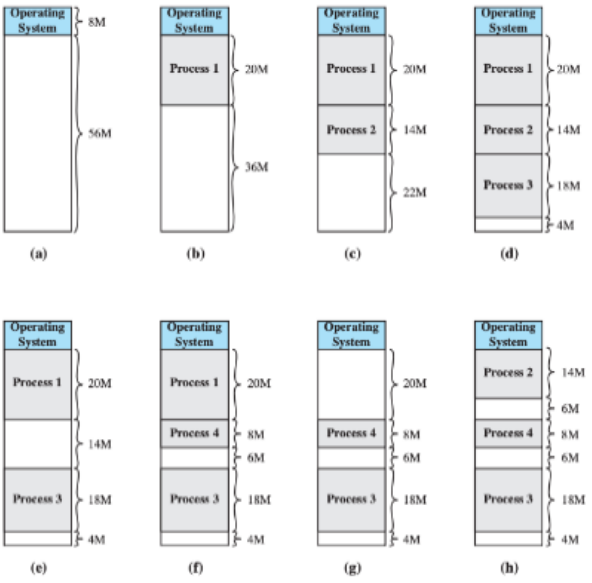
Rilocazione: La rilocazione può essere **statica** se a un processo sarà sempre assegnata la stessa partizione, in questo caso si può usare un caricatore (**loader**) rilocante. Al caricamento tutti i riferimenti a indirizzi relativi sono sostituiti da indirizzi assoluti, determinati dall'indirizzo base del processo caricato. La rilocazione può essere anche **dinamica** se il processo può occupare differenti partizioni nel corso della sua vita (ad esempio partizioni variabili e caricamento dinamico). Si ha quando avviene lo swapping tra memoria principale e secondaria:

- Quando l'immagine del processo è creata, viene caricata in una partizione;
- Quando è trasferita in memoria secondaria e poi ricaricata in memoria, occuperà un'altra partizione;
- In caso di compattamento della memoria, la partizione sarà ancora modificata.

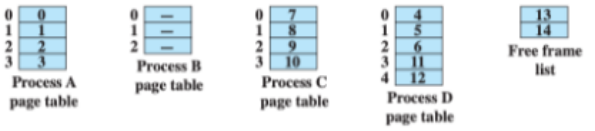
Quindi le locazioni di memoria riferite dal programma variano ogni volta che il processo subisce uno swap. È necessaria la rilocazione dinamica con supporto hardware.

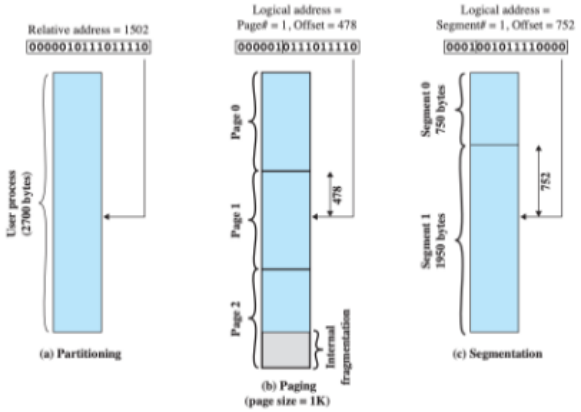


Paginazione: Nella paginazione la memoria principale è partizionata in blocchi di dimensione fissa detti **frame**. I programmi sono divisi in blocchi della stessa dimensione detti **pagine**, queste pagine sono poi caricate in frame grandi quanto la pagina (non necessariamente contigui).

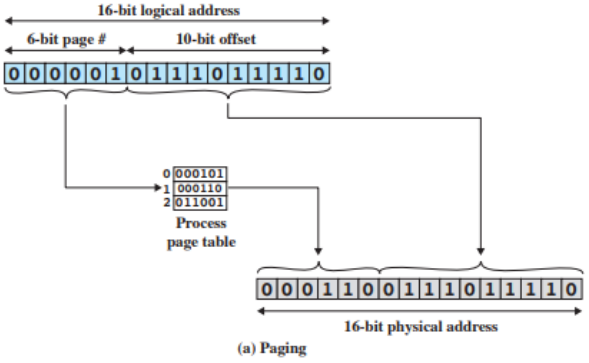


Per ogni processo viene creata una tabella che contiene la posizione del frame per ogni pagina del processo. Il processore accede alla tabella delle pagine del processo corrente per produrre l'indirizzo fisico.





Le 3 pagine sono lunghe ciascuna 1024 byte, l'ultima pagina ha uno spazio “sprecato” dovuto al fatto che il programma occupa 2700 byte, quindi all'ultima pagina se ne allocano 652 e il resto non servirà. L'indirizzo relativo 1502 rappresenta la casella indicata dalla freccia, l'indirizzo logico corrispondente è che, la casella appartiene alla pagina di numero 1, con offset = 478, ossia salterà i primi 1024 byte che appartengono alla prima pagina (la pagina 0) e partirà dal 1024-esimo byte applicandogli l'offset di 478 che ci permette di arrivare all'indirizzo 1502



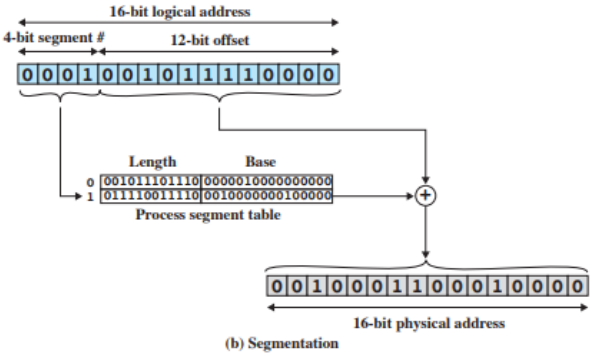
Segmentazione: Il caricamento dei programmi segmentati avviene tramite segmentazione dinamica. A ogni processo servono tante partizioni, una per ogni segmento, quindi a ogni processo assegneremo tanti partizioni quanti sono i segmenti possibili. Così facendo i segmenti sono più piccoli del programma intero, quindi tutti i fenomeni problematici che si avevano quando si aveva bisogno di una partizione molto grande non vi sono più (i segmenti sono piccoli, ce ne sono quindi tanti e la lunghezza massima è limitata). Il caricamento avviene come nella partizione dinamica e riduce la frammentazione esterna. Ciò che è visibile al programmatore è uno strumento per strutturare programma e dati: segmenti codice e segmenti dati, uno per modulo, stack. Il problema scaturisce dal fatto che, perché tutto funzioni, i segmenti non possono essere troppo grandi. La dimensione dei segmenti dipende dal programmatore che deve tenere conto della massima lunghezza dei segmenti (non succede dunque che a frame più piccoli o più grandi deve variare la grandezza del programma). Quindi gli indirizzi (logici) sono fatti dalla coppia:

- indirizzo di segmento
- offset.

Traduzione degli indirizzi:

1. Vi è un indirizzo logico che è fatto da n bit, che è l'indirizzo di segmento, e un'altra parte che è l'offset.
2. Dopodiché nel nostro sistema si ha una tabella dei segmenti (non più delle pagine) per cui ogni processo deve usare l'indice di segmento per trovare l'indirizzo fisico in cui comincia il segmento che mi interessa (indirizzo base)
3. Poi bisogna comparare l'offset nei rimanenti m bits con la lunghezza del segmento (indirizzo limite). Se è più grande allora non è valido (eccezione di indirizzo)
4. L'indirizzo fisico a cui accedere è dato dalla somma dell'indirizzo base e dell'offset

> **Nota bene:**
>
> Non posso ancora accedere all'indirizzo perché vi potrebbe essere un'eccezione di indirizzo. Si applica dunque la protezione di memoria, ovvero confronto il risultato della mia operazione con l'indirizzo limite, se è più grande il sistema operativo segnala un'eccezione. Nella paginazione non c'è questo problema, poiché la dimensione è fissa, il massimo valore che può assumere è la dimensione del frame. Sommo l'offset con l'indirizzo base, il risultato è l'indirizzo fisico. L'accesso alla tabella dei segmenti non è frequente perché tipicamente un programma rimane nel segmento per tempi molto lunghi. L'aggiornamento di questi registri (base e limite) avviene di fatto una volta ogni tanto, non a ogni istruzione. Il discorso è diverso nel caso della paginazione, lì la divisione è puramente casuale.



(b) Segmentation

Memoria virtuale

Il programma per essere eseguito deve stare in memoria, che però è limitata. Per questo i sistemi moderni sfruttano il modello della memoria virtuale. Questo è caratterizzato da:

- Rilocazione, ovvero la traduzione degli indirizzi, che normalmente avviene a tempo di esecuzione (run-time).
- Il processo, inoltre, non ha necessariamente un modello continuo ma può essere paginato oppure segmentato, quindi in realtà il programma è diviso in pezzi. Il processo può avanzare anche solo se in memoria vi sono i pezzi che mi servono (cioè i processi) e non il programma intero (non serve il programma sia caricato in pezzi contigui di memoria).

Dunque il punto chiave è che non è necessario che l'intero programma sia in memoria per poter eseguire il processo. Quindi di volta in volta verrà caricata in memoria solo la parte di processo che ci serve in tale momento.

> **Il principio di località:**
>
> Per tempi d'esecuzione relativamente brevi le istruzioni e i dati necessari sono tutti concentrati in una piccola parte di programma. Osservando il programma per un piccolo periodo, ci si accorge che le istruzioni e i dati sono concentrati in una piccola parte. Effettivamente per periodi relativamente lunghi (si parla di frazioni di secondo) il processo utilizza in realtà aree di programma molto piccole.

Esecuzione:

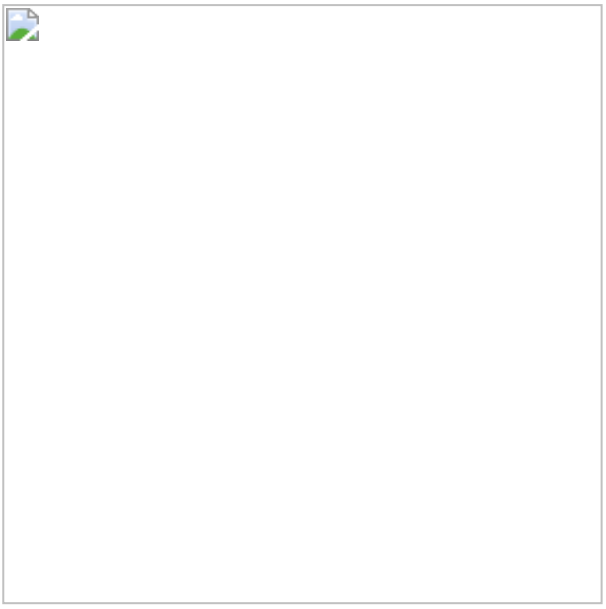
- Il SO carica in memoria (disco rigido) solo poche pagine (o segmenti) del programma con i dati (*resident set)
- Quando è richiesto un indirizzo non presente in memoria principale si genera un'interruzione

- Il SO sospende il processo e il processore è assegnato ad un altro processo e la pagina (o segmento) contenente l'indirizzo logico richiesto è caricato in memoria. Il trasferimento da disco a memoria principale avviene senza spesa di CPU.
- Al termine del caricamento, è generata un'interruzione (dal controllore = *DMA*) in seguito alla quale il SO riattiva il processo

Vantaggi:

- Incremento della multiprogrammazione: poiché solo parte dei processi è caricata in memoria, c'è spazio per più processi. Ciò garantisce l'efficientamento della gestione del processore, perché aumentano i processi pronti.
- Un processo può esse più lungo dell'intera memoria principale: non è più necessario l'overlay, il SO carica solo le parti di programma necessarie di volta in volta e il programmatore non deve conoscere la quantità di memoria disponibile (si può avere in linea di principio un programma che richiede uno spazio molto maggiore di quello disponibile in sistema perché tanto non deve essere caricato interamente in memoria)

Memoria reale	**Memoria virtuale**
Memoria principale (i.e. RAM)	Memoria percepita dal programmatore e dall'utente, realizzata dal So mediante il caricamento dinamico



Ricorda:

Vi sono due modelli di memoria: paginazione e segmentazione.

- Possibilità di caricamento semplice, ovvero segmentazione semplice e possibilità con memoria virtuale: Nella paginazione la memoria viene divisa in frames, nella segmentazione la memoria non è partizionata inizialmente, quindi avviene un partizionamento dinamico al caricamento. Il programma è tagliato in pagine nel caso di paginazione (tutte della stessa dimensione) e ciò è svolto dal sistema operativo, nel caso della segmentazione il programma è diviso in segmenti scelti dal programmatore.
- Spreco di memoria: con la segmentazione c'è frammentazione interna, l'ultima pagina è minore. Con la paginazione c'è frammentazione esterna, certe pagine sono troppo piccole per caricarvi processi. La frammentazione interna rispetto al caricamento in partizioni fisse prevede partizioni fisse, al massimo si possono sprecare un frame -1 byte nella nostra memoria, mentre con la segmentazione avendo segmenti che sono più piccoli di un programma con modello continuo, frammenti piccoli si riescono a utilizzare. Per la paginazione vi è la tabella delle pagine in cui

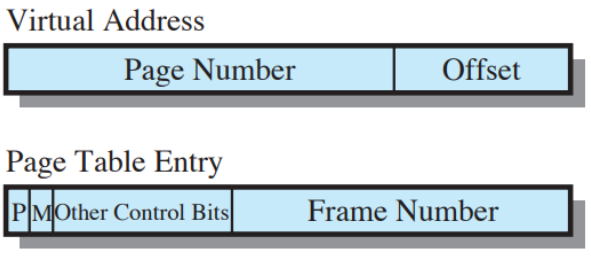
l'informazione principale è il frame occupato da ogni pagina, mentre nella segmentazione si ha una tabella dei segmenti per ciascun processo che mostra l'indirizzo di caricamento e la lunghezza di ciascun segmento. L'indirizzo fisico nel caso della paginazione si ottiene sostituendo il numero di pagine con l'indice di frame, nel caso della segmentazione bisogna calcolare l'indirizzo fisico tramite il numero di segmento e l'offset.

Principio di località: i riferimenti a dati e codice di un processo si concentrano in zone ben precise del programma, quindi si concentrano in poche pagine e in pochi segmenti. Il principio di località assicura che di solito il trashing non avviene. Il trashing è lo stato in cui li sistema spende la maggior parte del suo tempo nel caricamento e scaricamento dalla memoria, anziché nell'esecuzione dei processi:

- I riferimenti a dati e codice in un processo, per brevi periodi di tempo, tendono a raggrupparsi in poche pagine (o segmenti)
- Solo una piccola parte del processo è necessaria per brevi periodi di tempo
- È possibile fare ipotesi intelligenti su quali parti saranno necessarie in futuro (questo evita thrashing) considerando che quando la memoria è piena scarico ciò che non mi servirà in futuro

L'hardware deve supportare la paginazione e la segmentazione. Il SO deve includere il software per gestire il movimento delle pagine o dei segmenti fra memoria secondaria e memoria principale

Paginazione: nel linguaggio comune, il termine memoria virtuale si riferisce alla paginazione dinamica. Nella tabella delle pagine, in paginazione dinamica, vi è un bit di presenze, che è vero se la pagina è caricata in memoria e falso se non lo è, per questo motivo il processore guarda per prima cosa quel primo bit. Poi c'è un bit modificato (*dirty bit*) che serve a indicare se tale pagina, nel momento in cui è stata realizzata, è stata modificata.



Rilocazione (supporto HW)

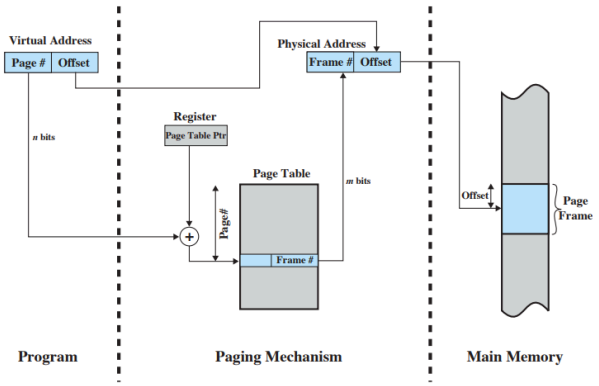
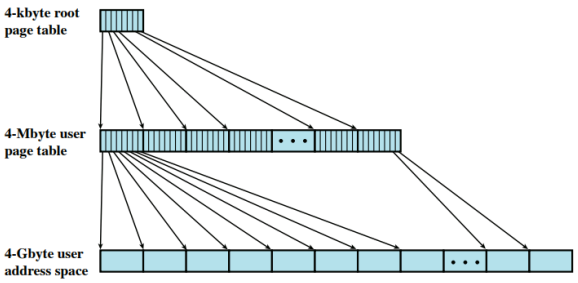
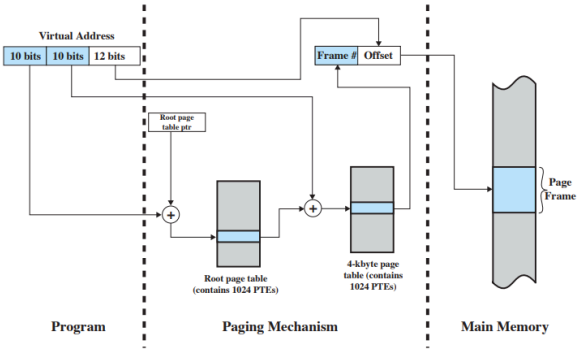


Tabella delle pagine a 2 livelli



Rilocazione a 2 livelli (supporto HW)



L'indirizzo virtuale è quello che sta nel program counter. C'è un registro che contiene il puntatore alla tabella delle pagine in memoria principale. A questo puntatore si somma l'indice di pagina e si ottiene il puntatore al descrittore di pagina, cioè la posizione (l'indirizzo) del descrittore di pagina del record che mi interessa. A questo punto si recupera l'indice di frame e si va a sostituire all'indice di pagina, per cui otterrò un nuovo indirizzo, questa volta fisico, i bit più significativi sono gli indici di frame. Ci sarà un addizionatore dedicato che ci permetterà di trovare l'indirizzo del record che ci interessa e a questo punto si può accedere alla memoria di tale record. Ci manca la verifica che il bit di presenza del caricamento sia true o false. Quando le pagine sono troppe può succedere che questa operazione diventi troppo grande per stare qui dentro, per cui si utilizza una tabella delle pagine a 2 livelli. Dunque ci sarà una tabella che mi dice in quali pagine è caricata la tabella delle pagine. Quindi per accedere alla tabella delle pagine, si dovrà anzitutto interpretare i primi bit, cioè la parte più significativa dell'indirizzo, determinare l'indice che consente di accedere alla voce corrispondente nella tabella delle pagine. Sommiamo i primi 10 bit al registro che contiene il puntatore alla pagina primaria (quindi indicano la pagina primaria), leggiamo la posizione della pagina in cui si trova il dato che ci interessa, poi all'interno di questa tabella i secondi 10 bit mi danno l'offset a cui si trova il dato.

In altre parole:

1. Il **Program Counter** contiene un **indirizzo virtuale**.
2. Questo indirizzo è diviso in:
   - **indice di pagina**
   - **offset**
3. La CPU usa un registro che punta alla **tabella delle pagine**.
4. Procedura:
   - Si prende l'**indice di pagina**
   - Si accede alla voce corrispondente nella tabella delle pagine
   - Si ottiene il **numero di frame** (memoria fisica)
5. Si costruisce l'indirizzo fisico:
   - si sostituisce l'indice di pagina con il **frame**

- si mantiene lo stesso **offset**

Risultato: **indirizzo fisico = frame + offset**

6. Prima di accedere:
   - si controlla il **bit di presenza**
     - `1` → pagina in RAM
     - `0` → page fault

Se la tabella delle pagine è **troppo grande**, non è gestibile direttamente. Per questo esiste la tabella delle pagine a 2 livelli. L'indirizzo virtuale viene diviso in:

- **indice tabella di primo livello**
- **indice tabella di secondo livello**
- **offset**

Procedura:

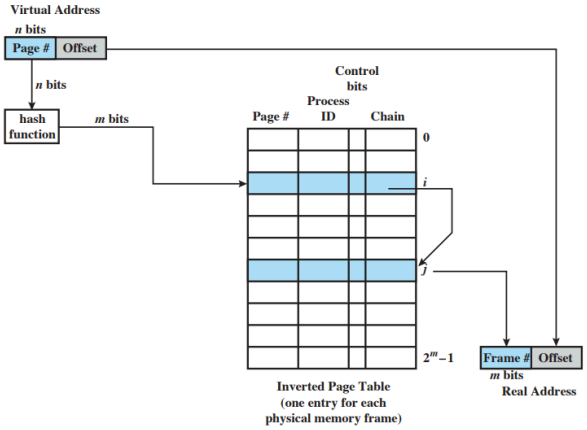
1. I **bit più significativi**:
   - indicano quale entry nella **tabella di primo livello** usare
2. Questa entry contiene:
   - il puntatore a una **tabella di secondo livello**
3. I **bit successivi**:
   - selezionano la voce nella **tabella di secondo livello**
4. Da lì ottieni il **frame**
5. Unisci:
   - **frame + offset** → indirizzo fisico

Tabella delle pagine invertite: poiché tabelle di pagine molto grandi dipendono dal fatto che i programmi sono molto grandi, ma non tutto il programma sta nella memoria principale, a noi basta avere i record delle pagine caricate effettivamente nella memoria principale. Per questo si usa la tabella delle pagine invertita, un array in cui ci sono solo le pagine caricate in memoria. Il numero di pagina è mappato in una tabella hash (il valore di hash punta alla tabella delle pagine invertita, contenente solo le pagine caricate in memoria principale), la tabella è invertita nel senso che le entries sono ordinate in ordine di frame anziché di pagina. Campi della PTE (tabella invertita):

- Numero di pagina
- Identificatore del processo
- Bit di controllo
- Puntatore alla entry successiva (per la hash)

I primi n bit sono inseriti in una funzione di hash che ci consente di cercare il nostro frame nella tabella, trovato il numero di frame gli aggiungiamo poi l'offset contenuto nell'indirizzo.

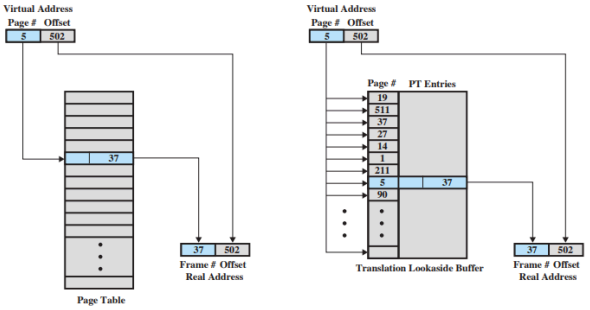
Tabella delle pagine invertita



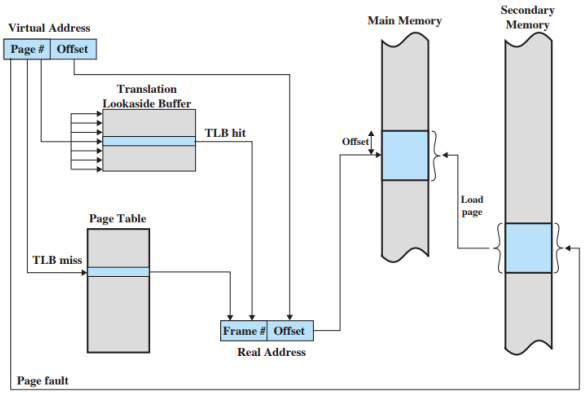
Translation lookaside buffer (TLB): evita un doppio accesso in memoria a ogni riferimento (quando devo tradurre un indirizzo logico ad un indirizzo fisico devo accedere alle tabelle delle pagine, nel caso in cui si ha un indirizzamento doppio si hanno quindi 2 accessi alla memoria):

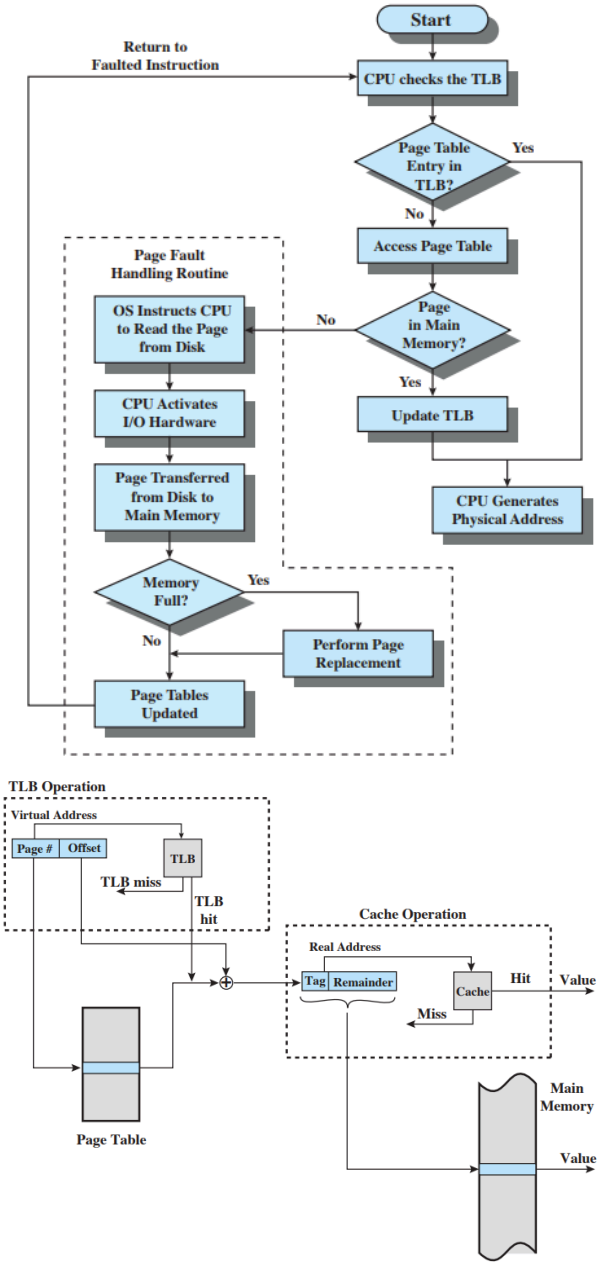
- Per la table entry
- Per il dato

In questa memoria veloce (cache) ci sono le PTE usate di recente. Inoltre c'è una mappa associativa che permette la ricerca simultanea del campo indice di pagina in tutti i PTE



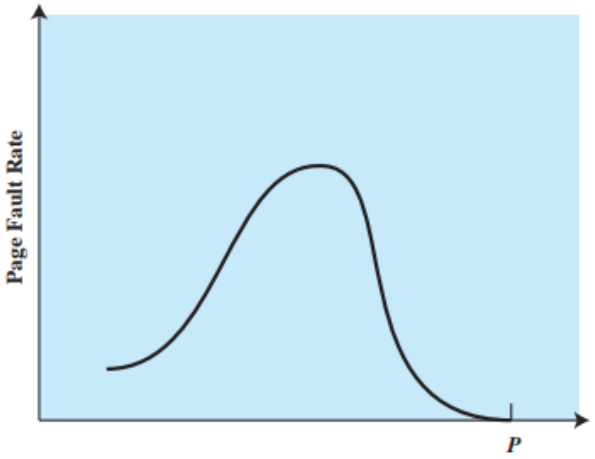
Rilocazione con TLB





Dato quindi l'indirizzo virtuale, prima si controlla il TLB: se l'indirizzo c'è, prendiamo il frame e applichiamo l'offset per trovare l'indirizzo fisico, se non c'è nel TLB si fa il doppio accesso alla memoria, prima nella Page table e poi nella main memory. Se la pagina non esiste il processo viene interrotto con page fault. Questo quindi ci evita di accedere alla memoria nel caso in cui l'indirizzo sia presente nel TLB, facendo tutto nel processore. Se la pagina non è neanche in memoria si inizializza la routine che gestisce l'assenza delle pagine, e la pagina viene trasferita dal disco alla memoria ovviamente se lo spazio c'è. Se la memoria è piena si rimpiazza qualche vecchia pagina che probabilmente non ci serve più (le pagine da sostituire si individuano tramite un algoritmo specifico) con la nuova pagina che prima non era presente in memoria.

Dimensioni delle pagine: pagine troppo grandi aumentano la frammentazione interna, mentre pagine piccole la riducono, ma aumentano la dimensione della tabella delle pagine (potrebbe accadere che neanche la tabella delle pagine può stare in memoria principale), per questo si può usare la gestione dinamica delle tabella delle pagine. Il trasferimento di informazioni da disco a memoria principale avviene a blocchi, quindi le cose vengono più semplici se la pagina è grande quanto un blocco. Le caratteristiche fisiche della memoria secondaria influiscono sulla scelta (trasferimento a blocchi) e la dimensione della pagine influisce sulla frequenza degli errori di pagina



(a) Page Size

La scelta della dimensione è basata su:

- Dimensione della memoria principale e dei programmi.
- Tecniche di programmazione moderne riducono la località
- Il TLB è meno efficiente (specie se di piccola dimensione)
- Pagine di dimensione variabile

Segmentazione: con la segmentazione il programmatore vede la memoria composta da multipli spazi di indirizzamento (segmenti)

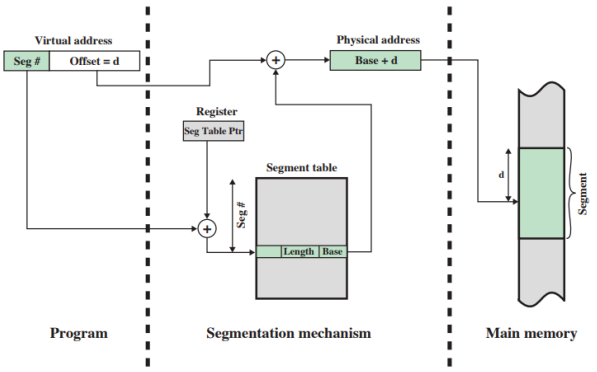
Vantaggi:

- Semplifica la gestione di strutture dati di dimensione variabile
- Permette la modifica e ricompilazione di segmenti di programma, senza modificare il resto
- Facilita la condivisione di dati fra processi
- Facilita la protezione

Tabella dei segmenti: ogni record della tabella dei segmenti contiene:

- L'indirizzo iniziale del corrispondente segmento di memoria
- La lunghezza del segmento
- Flag 'caricato'
- Flag 'modificato'

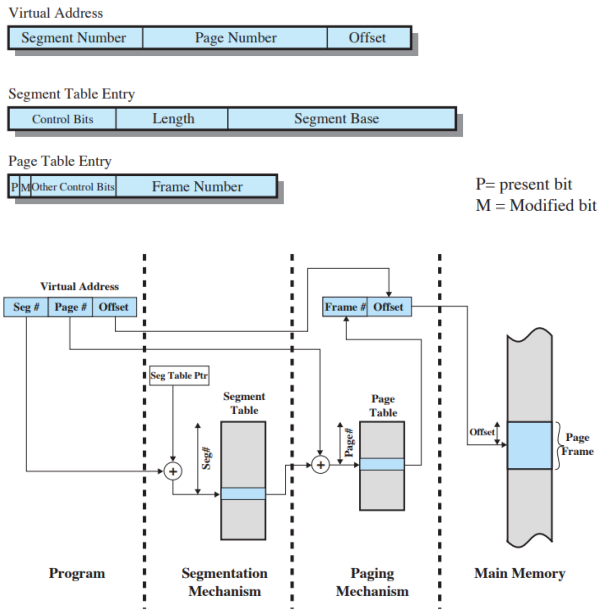
Rilocazione



Segmentazione paginata: nella segmentazione paginata:

- Lo spazio logico è diviso in segmenti
- Ogni segmento è diviso in pagine di dimensione fissa, uguale alla lunghezza dei frame della memoria principale

La segmentazione era visibile al programmatore, la paginazione era trasparente e affidata al sistema operativo. Con la segmentazione paginata l'indirizzo virtuale si divide in numero di segmento, numero di pagina e offset. Inoltre avremo in questo caso due strutture dati, la Segment table e la Page table.



Protezione e condivisione: la segmentazione facilita l'implementazione della protezione e della condivisione:

- Ogni record della tabella dei sementi contiene l'indirizzo base e lunghezza , per controllare accessi erronei
- La condivisone è implementata inserendo un riferimento allo stesso segmento nella tabella dei segmenti di più processi

Memoria virtuale: politiche di gestione

La progettazione del gestore della memoria dipende da tre decisioni fondamentali:

- Usare o meno tecniche di memoria virtuale
- Usare la paginazione o la segmentazione o la segmentazione paginata
- Gli algoritmi utilizzati per i vari aspetti dell gestione della memoria

> **Info**
>
> - Strategia di caricamento, come caricare le pagine in memoria,
> - Strategia di posizionamento, dove caricare le pagine in memoria,
> - Strategia di sostituzione, come decido quali pagine sono sostituibili in caso di memoria piena,
> - Gestione del resident set, può avere dimensione fissa o variabile, l'ambito di sostituzione indica dove carico le nuove pagine dopo averne sostituite alcune vecchie.
> - Strategia di cleaning, a richiesta quando è necessario, oppure quando frame liberi sono vicini allo 0 faccio precleaning.
> - Controllo del caricamento, screening a lungo termo che influisce sul grado di multiprogrammazione

Strategia di caricamento	A richiesta, Prepaginazione
Strategia di posizionamento	
Strategia di sostituzione	Algoritmi di base (ottimo, LRU, FIFO, orologio), Buffer per pagine
Gestione del resident set	Dimensione del resident set (fissa, variabile), Ambito di sostituzione (globale, locale)
Strategia di cleaning	a richiesta, precleaning
Controlllo del caricamento	Grado di multiprogrammazione

Strategie di caricamento: Determinano quando una pagina deve essere caricata in memoria.

1. *Paginazione a richiesta:* la pagina viene caricata, quando c'è un riferimento a quella pagina ed essa non è presente in memoria. I problemi possono essere legati a molti errori di pagina all'avvio del processo (poiché nessuna pagina è presente all'inizio, errore di pagina = page fault = pagina non presente in memoria). Il principio di località suggerisce che al crescere delle pagine caricate, gli errori di pagina si riducono a un livello molto basso.
2. *Pre-paginazione:* sono caricate altre pagine oltre a quelle che hanno determinato l'errore di pagina. Sfrutta le caratteristiche dei dispositivi di memoria secondaria, ad esempio le pagine memorizzate in aree contigue di memoria secondaria sono caricate più velocemente in serie piuttosto che una alla volta. Tuttavia possono essere caricate pagine non riferite.

Strategie di posizionamento: Determina in quale parte della memoria principale la pagina o il segmento deve essere caricato. È importante solo per la segmentazione non paginata, può usare i criteri *first fit, best fit e next fit*. Il posizionamento influisce sulle prestazioni anche con la paginazione nei sistemi distribuiti e i sistemi distribuiti NUMA.

Strategie di sostituzione: seleziona la pagina in memoria principale da sostituire quando è necessario caricare una nuova pagina. L'obiettivo è selezionare la pagina che più difficilmente sarà riferita nel futuro prossimo. Bisogna fare attenzione in quanto più complessa è la politica di sostituzione, maggiore l'overhead (software e hardware) per implementarla. In alternativa si usano algoritmi non troppo complessi, ma che comunque danno un buon risultato.

Ci sono però dei frame che non possono essere sostituiti anche se inutilizzati, sono i frame **bloccati (locked)**:

- La pagina caricata in quel frame non può essere sostituita
- Il kernel del SO e le sue strutture di controllo principali sono caricate in frame bloccati
- Buffer I/O e aree critiche per iterazioni real-time possono essere caricate in frame bloccati (temporaneamente)
- Il blocco si ottiene associando un 'lock bit' a ogni frame (nella tabella dei frame o nella tabella delle pagine)

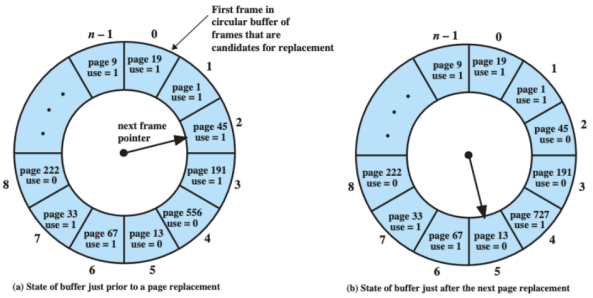
Last recently used (LRU): L'algoritmo ottimo di sostituzione sarebbe quello che sostituisce la pagina che, in futuro, sarà riferita per ultima. Ciò però è ovviamente irrealizzabile, ma può funzionare da benchmark, quindi come confronto di prestazioni. **LRU (Least Recently Used)** è un algoritmo che seleziona la pagina che da più tempo non è riferita, in quanto è ragionevole pensare che questa pagina anche in futuro non sarà riferita (principio di località). Il problema sta nel fatto che è difficile da implementare: dovrei marcare ogni pagina col tempo di ultimo riferimento oppure organizzare le pagine in una coda FIFO, ordinate quindi dalla più giovane alla più vecchia. Questo comporta un grande overhead.

First-in-first-out (FIFO): Un'alternativa è la politica FIFO. Le pagine vengono organizzate in un buffer circolare (coda FIFO) e vengono rimosse in ordine di caricamento, si sostituisce quindi la pagina che è stata più tempo in memoria. Questo algoritmo è facile da implementare, ma il difetto sta nel fatto che ci sono parti di codice eseguite in continuazione, quindi prima o poi potrei sostituire parti di codice che sono in memoria da molto tempo, ma vengono usate spesso.

Strategia dell'orologio: associa una flag a ogni frame (bit di utilizzo). Questo viene settato a 1 quando la pagina è carica e ogni volta che la pagina è riferita. L'algoritmo di sostituzione scandisce circolarmente i frame

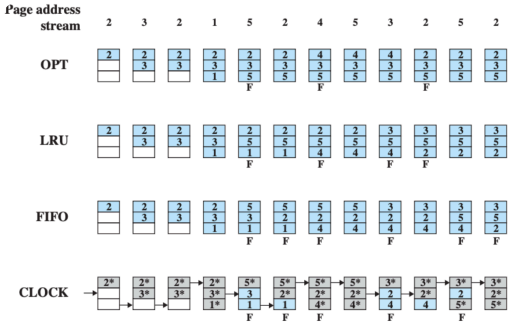
- Se il bit di utilizzo è 1, viene settato a 0
- Se il bit è 0, il frame vine selezionato per lo scaricamento

Una strategia più complessa distingue tra frame modificati e non modificati



(a) State of buffer just prior to a page replacement

(b) State of buffer just after the next page replacement



In questa foto dunque si vede un confronto tra i diversi algoritmi visti finora. Nell'algoritmo ottimo (OPT) dovremmo andare a leggere nel futuro quale delle pagine 2,3,1 non verrà più utilizzata o comunque verrà utilizzata più tardi delle altre: 1 non sarà più usata dunque sarà la pag.1 ad essere scaricata per prima. Nell'algoritmo LRU guardo quella usata meno recentemente: in questo caso la pagina 3. Nel FIFO tolgo la prima pagina caricata (quella che vi è da più tempo).

Buffer per pagine: Migliora la gestione della memoria virtuale (migliora le prestazioni della paginazione e permette l'uso di politiche di sostituzione più semplici). L'idea è quella di utilizzare il buffer per pagine, per cui le pagine scaricate vengono inserite in due liste: pagine libere e pagine modificate. Le pagine libere possono essere immediatamente sovrascritte. Mentre queste pagine sono dentro le liste, di fatto sono ancora presenti dentro al sistema, per cui se arriva una richiesta le pagine possono essere recuperate rapidamente senza accedere al disco, migliorando le prestazioni. Questo meccanismo è utile perché evita di ricaricare pagine che erano state appena rimosse, riducendo così il numero di page-fault e di operazioni di I/O costose. Le pagine modificate sono salvate a gruppi.

Resident set: Il SO decide quante pagine di ogni processo tenere in memoria principale. Sarebbe utile infatti decidere a priori per ogni processo quante pagine tenere in memoria, ciò è il resident set. Come selezioniamo la dimensione del resident set? Avere tanti frame significa avere meno errori di pagina. Se però vogliamo tanti processi (avendo una RAM limitata), allora tanti frame potrebbero non decrementare gli errori di pagina. Ci sono due possibilità:

- *Allocazione fissa:* per cui il processo ha un numero fisso di frame in cui essere eseguito, e a ogni errore di pagina si scarica una pagina del processo che lo ha determinato;
- *Allocazione variabile:* per cui il numero dei frame allocati a un processo può variare nel tempo. Quando il sistema si accorge che un processo fa tanti errori di pagina, decide di aumentargli il resident set. Se invece non fa errori di pagina, riduce il resident set per dar spazio agli altri processi.

Ambito della strategia di sostituzione: Si hanno varie possibilità di scelta per determinare come modificare il resident set:

1. *Sostituzione in ambito locale:* ogni processo sceglie la pagina da sostituire esclusivamente tra le proprie pagine residenti (cioè tra le pagine residenti del processo che ha determinato l'errore di pagina). In questo modo, solo il processo che ha causato l'errore di pagina viene sospeso, e si ha un rallentamento localizzato al singolo processo coinvolto.
2. *Sostituzione in ambito globale:* si considerano tutte le pagine non bloccate presenti nella memoria principale (entrambi attivati da un errore di pagina), indipendentemente dal processo a cui appartengono. In questo caso, il sistema ha maggiore libertà nella scelta delle pagine da sostituire, ma il rischio è che processi non direttamente coinvolti vengano penalizzati. A fronte di ciò, se il numero di processi attivi è limitato, si riduce anche la competizione per la memoria, migliorando l'efficienza complessiva del sistema.
3. *Allocazione fissa, ambito locale:* in questo caso è necessario decidere all'inizio quanto spazio assegnare al processo. Un'allocazione troppo piccola causa un alto numero di errori di pagina, mentre un'allocazione troppo grande porta a pochi processi in memoria. Di conseguenza si ha un rallentamento dell'uso del processore
4. *Allocazione variabile, ambito globale:* l'implementazione di questa soluzione è semplice e il SO mantiene una lista dei frame liberi. Un frame libero è aggiunto all'insieme residente, in caso di errore di pagina. In assenza di frame liberi, una pagina di un qualunque processo può essere scaricata
5. *Allocazione variabile, ambito locale:* quando un nuovo processo è caricato, gli vengono allocati un certo numero di frames. Quando poi c'è un errore di pagina, viene sostituita una pagina dell'insieme residente del processo che ha determinato l'errore. Periodicamente, si rivaluta lo spazio assegnato, incrementandolo o decrementandolo, per migliorare l'efficienza.

Working set:

Definizione: Il **working set** $(,)$ è l'insieme delle pagine riferite nell'intervallo di tempo virtuale $,$ (ovvero un intervallo espresso non in un tempo reale ma in numero di riferimenti a memoria). Esso rappresenta le pagine attivamente utilizzate da un processo in un determinato periodo, e serve a stimare quanta memoria è effettivamente necessaria affinché il processo possa continuare a eseguire senza incorrere frequentemente in errori di pagina. Un processo con un working set ampio dovrebbe avere un resident set ampio. Il pallino (nella foto) indica che il working set non si è modificato rispetto al passo precedente. Man mano che si aumenta la dimensione del delta, i casi in cui il working set non si modifica aumentano.La decisione di incrementare o decrementare la dimensione dell'insieme residente si basa sulla valutazione delle future richieste dei processi attivi e richiede:

- Criteri per la determinazione della dimensione dell'insieme residente
- Tempistica delle variazioni

Sequence of Page References	Window Size, Δ			
	2	3	4	5
24	24	24	24	24
15	24 15	24 15	24 15	24 15
18	15 18	24 15 18	24 15 18	24 15 18
23	18 23	15 18 23	24 15 18 23	24 15 18 23
24	23 24	18 23 24	•	•
17	24 17	23 24 17	18 23 24 17	15 18 23 24 17
18	17 18	24 17 18	•	18 23 24 17
24	18 24	•	24 17 18	•
18	•	18 24	•	24 17 18
17	18 17	24 18 17	•	•
17	17	18 17	•	•
15	17 15	17 15	18 17 15	24 18 17 15
24	15 24	17 15 24	17 15 24	•
17	24 17	•	•	17 15 24
24	•	24 17	•	•
18	24 18	17 24 18	17 24 18	15 17 24 18

Una strategia possibile è quella di controllare periodicamente il working set di ogni processo e rimuovere dall'insieme residente le pagine che non sono nel working set. Un processo può essere eseguito solo se il suo working set è interamente in memoria. L'implementazione di questa soluzione è difficile. Un'alternativa è agire sulla frequenza degli errori di pagina: quando gli errori di pagina diventano troppo ravvicinati, si aggiunge una pagina all'insieme resident, altrimenti si scaricano tutte le pagine non utilizzate dall'ultimo errore di pagina, e tutte le altre diventano candidate alla prossima

Page fault frequency: quando c'è un errore di pagina:

- Se il tempo virtuale dall'errore di pagina precedente è inferiore a una soglia F, si aggiunge una pagina all'insieme residente
- Altrimenti si scaricano tutte le pagine con il bit di utilizzo 0 e si azzera il bit di utilizzo delle rimanenti. Questa soluzione provoca cattive prestazioni nelle transizioni a nuove località

Variable-Interval Sampled Working Set (VSWS): Valuta il working set a intervalli di campionamento basati sul tempo virtuale. Viene diretto da tre parametri:

- Durata minima dell'intervallo di campionamento
- Durata massima dell'intervallo di campionamento
- Numero degli errori di pagina consentiti tra gli intervalli di campionamento

Politica di cleaning:

- *Cleaning su richiesta:* La pagina è salvata in memoria secondaria solo quando deve essere sostituita
- *Precleaning:* Permette il salvataggio di pagine a lotti

Politica di caricamento: Determina il numero dei processi residenti in memoria principale.

- Grado di multiprogrammazione
- Pochi processi = molte occasioni con tutti i processi bloccati
- Troppi processi = trashing

Per ridurre il grado di multiprogrammazione può essere necessario scaricare uno o più processi. Per fare ciò esistono 6 possibili politiche:

1. processo con priorità minima
2. processo che provoca errori di pagina
3. ultimo processo attivato
4. processo con l'insieme residente più piccolo
5. processo più grande
6. processo col massimo tempo di esecuzione rimanente

Scheduling

Lungo termine	Inserimento nella lista dei processi da eseguire
Medio termine	Aggiunta ai processi caricati (parzialmente o totalmente) in memoria principale
Breve termine	Selezione, tra i processi pronti, del processo da eseguire
Scheduling I/O	Seleziona il processo in attesa di I/O da assegnare a un dispositivo disponibile

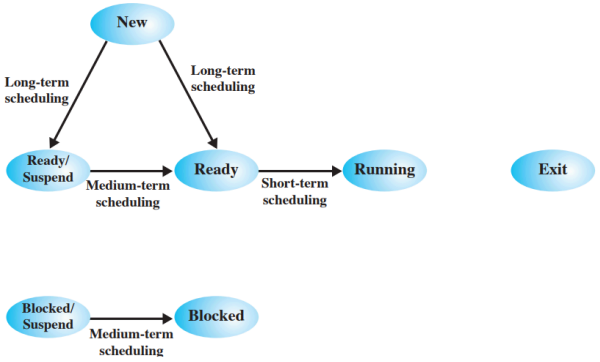
Lo scopo è di assegnare al processore i processi da eseguire in modo da realizzare gli obiettivi del sistema:

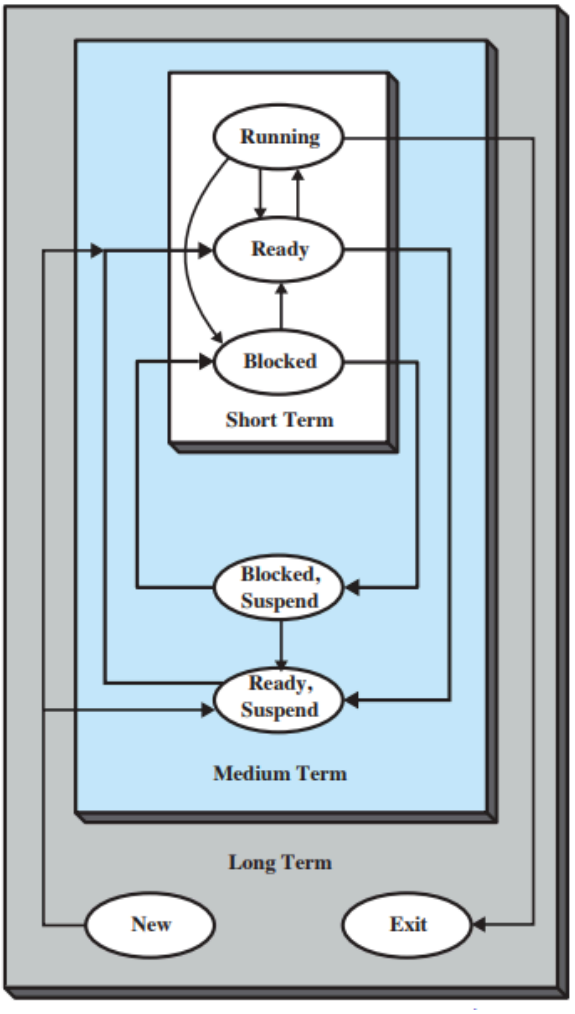
- *Tempo di risposta del processore:* ci sono processi interattivi, in cui l'utente interagisce con il processo e quello che vorremmo da questi processi è una risposta immediata

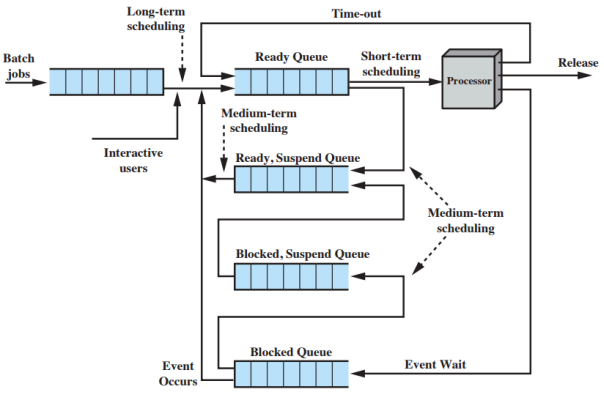
- *Throughput:* quanti lavori riusciamo a fare nel minor periodo di tempo
- *Efficienza del processore:* si deve sempre utilizzare quanto più processore possibile

> **Ricorda:**
>
> Un processo passa attraverso nuovi stati, da processo New a processo Ready agisce la schedulazione a lungo termine, che agisce anche per i processi Ready/Suspend, poi lo scheduling a medio termine deciderà quale di questi processi potrà passare in stato di Ready, quello short-term decide quale dei processi verrà messo in esecuzione. Lo scheduling a medio termine gestisce anche i processi Bloccati o sospesi.

Tipi di scheduling







> **Ricorda:**
>
> Tutti gli stati possibili di un processo:
>
> - Ready;
> - Running;
> - Blocked;
> - Blocked/Suspend;
> - Ready/Suspend;
> - New;
> - Exit.

Schedulatore a lungo termine: determina quali programmi sono ammessi nel sistema per l'esecuzione e controlla il grado di multiprogrammazione:

- Più processi creati = più piccole frazioni del tempo di esecuzione assegnate ai processi
- Per fornire servizi soddisfacenti all'insieme corrente di processi, è opportuno limitarne il numero

Per creare un processo della coda dei nuovi job, lo schedulatore deve decidere:

- Quando il SO può accettare uno o più nuovi processi
- Quali job trasformare in processi, e la scelta può adottare una politica ad esempio:
  - First come, first served
  - Priorità
  - Tempo di esecuzione atteso
  - Requisiti I/O

Schedulatore a medio termine: Parte della funzione di swapping. Le decisioni di caricamento/scaricamento sono correlate a:

- controllo del grado di multiprogrammazione
- requisiti di memoria dei processi

Dispatcher (schedulatore a breve termine): esegue la schedulazione a breve termine ed è eseguito con grande frequenza. Decide quale tra i processi pronti deve essere eseguito e viene attivato da:

- *Interruzione di clock:* il tempo assegnato a un processo è stato raggiunto e quindi il processo deve lasciare il processore per altri processi
- *Interruzioni I/O:* il processo deve interagire con un dispositivo quindi sarà interrotto per diverso tempo
- *Chiamate di sistema:* un processo esegue una system call, si sospende
- Segnali (e.g.semafori)

Lo scheduling a breve termine è ciò che ci interessa di più. È la chiave di volta di un sistema operativo poiché, essendo chiamata molto frequentemente, deve usare algoritmi rapidi e semplici e non si possono adottare politiche molto complesse. D'altra parte però se non funziona viene danneggiato tutto il sistema. Si devono usare quindi dei criteri che vengono stabiliti dal progettista che permettano di allocare la CPU in modo da ottimizzare alcuni aspetti del comportamento del sistema.

Criteri per lo scheduling a breve termine:

Obiettivo: allocare la CPU in modo da ottimizzare alcuni aspetti del comportamento del sistema

Ci sono dei criteri per fare ciò:

1. *Criteri orientati all'utente:* relativi alla percezione del comportamento del sistema da parte degli utenti (o dei processi):
   - E.g.tempo di risposta nei sistemi interattivi
   - Importante in tutti i sistemi
2. *Criteri orientati al sistema:* relativi all'efficienza dell'utilizzazione del processore
   - e.g., frequenza del completamento dei processi
   - trascurabili nei sistemi a singolo utente
3. *Criteri relativi alle prestazioni:* sono quantitativi e facili da misurare (e.g.tempo di risposta)
4. *Criteri non relativi alle prestazioni:* sono qualitativi e difficili da misurare (e.g.predicibilità)

Criteri orientati all'utente e relativi alle prestazioni:

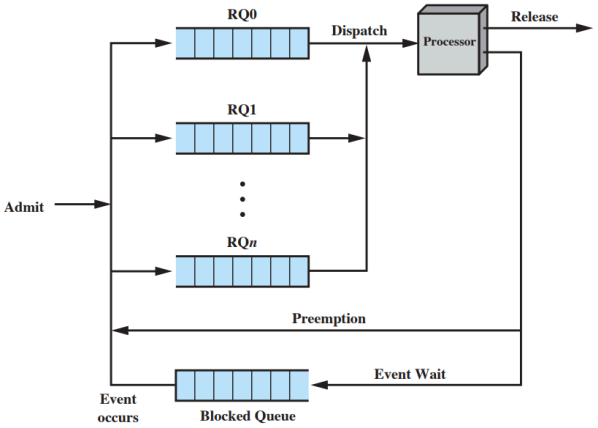
- *Tempo di risposta:* Per un processo interattivo, questo è il tempo che trascorre dall'invio di una richiesta fino a quando la risposta non comincia ad essere ricevuta. Spesso un processo può cominciare a produrre qualche output per l'utente mentre continua l'esecuzione. Perciò, dal punto di vista dell'utente, questa è una misura migliore del tempo di turnaround. La disciplina di scheduling dovrebbe tentare di raggiungere bassi tempi di risposta e di massimizzare il numero di utenti interattivi che hanno un tempo di risposta accettabile.
- *Tempo di turnaround:* È l'intervallo di tempo tra l'invio di un processo e il suo completamento. Comprende il tempo di esecuzione e il tempo speso in attesa delle risorse, compreso il processore. Questa è una misura appropriata per i *job batch* (programma o gruppo di task che viene eseguito automaticamente, richiedendo un'interazione umana minima una volta partito).
- *Scadenze:* Quando si può specificare il termine di completamento di un processo, la disciplina di scheduling può subordinare altri obiettivi oltre a quello di massimizzare la percentuale di scadenze raggiunta.
- *Prevedibilità:* Un dato job può essere eseguito nello stesso tempo e allo stesso costo, indipendentemente dal carico del sistema. Un'ampia variazione nel tempo di risposta o nel tempo di turnaround è fastidiosa per l'utente: potrebbe segnalare un‘ampia oscillazione nel carico di lavoro del sistema oppure la necessità di adattare il sistema per evitare l'instabilità

Criteri orientati al sistema e relativi alle prestazioni:

- *Throughput:* La strategia di scheduling dovrebbe tentare di massimizzare il numero di processi completati per unità di tempo. Così si misura la quantità di lavoro eseguito che dipende chiaramente dalla lunghezza media di un processo, ma che è anche influenzata dalla strategia di scheduling che può avere effetti sull'utilizzazione.
- *Utilizzazione del processore:* Questa è la percentuale del tempo in cui il processore è occupato. Per un sistema costoso e condiviso, questo è un criterio significativo, mentre

in sistemi a singolo utente e in alcuni altri sistemi, come i sistemi a tempo reale, questo criterio è meno importante.

- *Fairness:* In assenza di indicazioni da parte dell'utente, oppure del sistema, i processi devono essere trattati allo stesso modo, e nessun processo deve subire starvation.
- *Priorità:* Quando si assegna una priorità ai processi, la strategia di scheduling deve favorire i processi a più alta priorità.
- *Bilanciamento delle risorse:* La strategia di scheduling deve tenere impegnate le risorse del sistema, favorendo i processi che sotto utilizzano le risorse più impegnate. Questo criterio coinvolge anche lo scheduling a medio e lungo temine.



Funzione di selezione: Determina quale processo, tra quelli pronti, è selezionato per l'esecuzione. Può essere basata su

- priorità,
- esigenza di risorse,
- caratteristiche di esecuzione

Nel terzo caso, si tiene conto dei parametri:

- = tempo di attesa
- = tempo di esecuzione
- = tempo stimato di esecuzione (compreso)

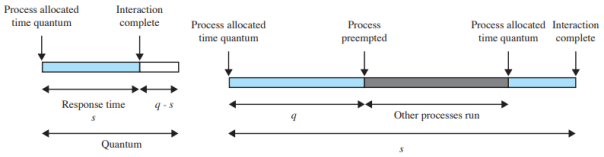
Modo di decisione: Specifica il momento in cui è eseguita la decisione:

- *Senza prerilascio:* Il processo in esecuzione continua fino al termine o fino a un'interruzione per I/O
- *Con prerilascio:* Il processo in esecuzione può essere interrotto dal SO e posto in stato di pronto. Il prerilascio può avvenire:
  - quando arriva un nuovo processo
  - quando un'interruzione riattiva un processo sospeso
  - periodicamente, in base a un clock

First come first served (FCFS): si tratta della gestione più semplice (FIFO). Ogni processo pronto è inserito nella coda pronti e quando il processo in esecuzione termina o è sospeso, viene selezionato per l'esecuzione il processo che è stato più a lungo nella coda pronti. Favorisce i processi lunghi rispetto a quelli brevi e i processi processor-bound rispetto a quelli I/O-bound

Round robin: Politica molto usata soprattutto su sistemi a multi-utente. C'è un clock che definisce il *time-slice* (fetta di tempo assegnata a ciascun processo) quando ad un processo viene assegnato il processore. Se il processo si sospende, tornerà all'ultimo posto della coda e se non si sospende, entro il time-slice, il processo viene pre-rilasciato. Il principale aspetto progettuale è la durata del quanto di tempo, ogni processo interattivo dovrebbe riuscire a fare i propri compiti nel quanto. È molto efficiente nei sistemi time-sharing e orientati alle transazioni. Favorisce i processi processor-bound rispetto a

quelli I/O bound. L'alternativa è quella del VRR (virtual round robin), dove i processori che non utilizzano tutto il quanto di tempo sono premiati alla successiva esecuzione.



Shortest process next (SPN): Seleziona il processo col minimo tempo stimato di esecuzione e senza prerilascio. Un processo breve può saltare in testa alla coda pronti. Può causare starvation per processi lunghi ed è inadatto nei sistemi time-sharing (per l'assenza di prerilascio). È difficile conoscere o stimare il tempo di esecuzione:

- Media aritmetica della durata delle esecuzioni precedenti:

$$_{n+1} = 1n \quad _{n} \quad _{=1} = _{n} + (n \ 1)_{n1}n$$

- Media esponenziale:

$$_{n+1} = \quad _{n} \quad _{=1}(1 \ )^{n} + (1 \ )^{n}_{1} = _{n} + (1 \ )_{n}$$

Se il tempo di esecuzione supera la stima del programmatore, il programma può essere terminato.

Shortest remaining time (SRT): si tratta di un SPN con prerilascio. Seleziona il processo col minimo tempo stimato di esecuzione e potrebbe causare starvation per processi lunghi. Migliora il tempo di turnaround rispetto a SPN, perché i job brevi hanno precedenza immediata sui job lunghi. Evita i prerilasci addizionali di Round Robin e non favorisce i lavori lunghi come FCFS. L'overhead è causato dalla stima del tempo di esecuzione

Highest Response Ratio Next (HRRN):

$$= +$$

con $=$tempo di attesa e $=$tempo stimato di esecuzione. Cerca di eliminare il problema della starvation tenendo conto anche del tempo di attesa. Seleziona il processo con il più alto rapporto di risposta (tempo di turnaround normalizzato), questa è più alta per i processi brevi poiché se è piccola la frazione è molto alta. Processi che sono in attesa da molto tempo avranno una molto grande e quindi finiranno per essere eseguiti. Quindi i processi brevi sono comunque favoriti, ma si evita la starvation.

Confronto:

	FCFS	Round robin	SPN	SRT	HRRN	Feedback
Selection function	$\max w$	constant	$\min s$	$\min(s-e)$	$\max \frac{w+s}{s}$	...
Decision mode	Non-preemptive	Preemptive (at time quantum)	Non-preemptive	Preemptive (at arrival)	Non-preemptive	Preemptive (at time quantum)
ThroughPut	Not emphasized	May be low if quantum is too small	High	High	High	Not emphasized
Response time	May be high, especially if there is a large variance in process execution times	Provides good response time for short processes	Provides good response time for short processes	Provides good response time	Provides good response time	Not emphasized
Overhead	Minimum	Minimum	Can be high	Can be high	Can be high	Can be high
Effect on processes	Penalizes short processes; penalizes I/O bound processes	Fair treatment	Penalizes long processes	Penalizes long processes	Good balance	May favor I/O bound processes
Starvation	No	No	Possible	Possible	No	Possible

Schedulazione fair-share: quando ho sistemi multi-thread dobbiamo tenere conto di due aspetti, a quali processi assegnare la CPU, ma anche a quale utilizzatore assegnare il processore. Si usano politiche di schedulazione basate su insiemi di processi. Ogni

utilizzatore ha assegnata una 'frazione' del processore. L'obiettivo è monitorare l'uso per dare meno risorse a chi ne ha avute più della frazione spettante e più risorse a chi ne ha avute di meno. Si può assegnare una priorità determinata con:

$$() = (\ 1)2, () = (\ 1)2, () = a + ()2 + ()$$

ove:

- $()$ = uso del processore del processo nell'intervallo
- $()$ = uso del processore del gruppo nell'intervallo
- $()$ = priorità del processo all'inizio dell'intervallo
- a = priorità iniziale del processo
- = peso assegnato al gruppo $(\ = 1)$

Schedulazione UNIX tradizionale: Ottimizzato per sistemi time-sharing, ha un buon tempo di risposta per i processi interattivi, evitando starvation dei processi in background. Permette feedback multilivello con round-robin in ogni coda di priorità e prerilascio dopo 1 sec. Priorità basata su tipo di processo e storia dell'esecuzione

$$() = (\ 1)2, () = a + ()2 + n$$

ove:

- $()$ = uso del processore del processo nell'intervallo
- $()$ = priorità del processo all'inizio dell'intervallo
- a = priorità iniziale del processo
- na = fattore di aggiustamento controllabile dall'utilizzatore

Schedulazione UNIX tradizionale: I pesi di CPU e nice sono limitati in modo da tenere separate le seguenti fasce di processi:

- Swapper
- Controllo dei dispositivi I/O a blocchi
- Manipolazione di file
- Controllo dei dispositivi I/O a caratteri
- Processi utente

Input/output

Categorie di dispositivi I/O:

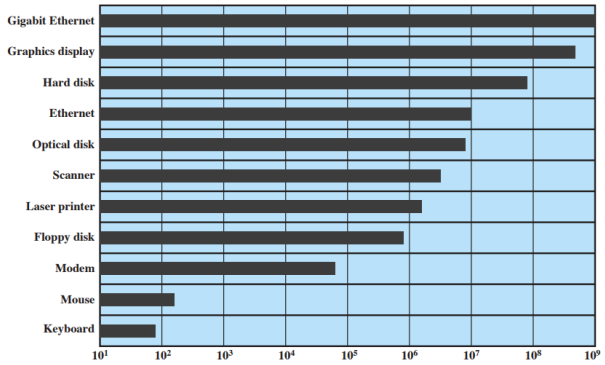
- Leggibili dall'uomo:
  - Per comunicare con l'utilizzatore
  - Stampanti, terminali, video display, tastiera, mouse
- Leggibili dalla macchina:
  - Per comunicare con l'apparecchiatura elettronica
  - Drive di dischi e nastri, chiavette USB, sensori, attuatori
- Dispositivi di comunicazione:
  - Per comunicare con dispositivi remoti
  - Drive di linea digitale, modem

**Caratteristiche dei dispositivi I/O

- Velocità del trasferimento: possono esserci grandi differenze tra un dispositivo e un altro
- Applicazioni: l'uso di un particolare dispositivo influenza il software
- Complessità di controllo: differenti dispositivi richiedono differente software. La complessità è parzialmente filtrato dalla complessità del modulo I/O che controlla il dispositivo

- Unità di trasferimento: dispositivi a caratteri (trasferiscono un byte alla volta) o a blocchi (trasferimenti a blocchi dell'ordine dei kilobyte)
- Rappresentazione dei dati: dispositivi differenti usano codifiche differenti
- Condizioni di errore: i tipi di errore e il modo in cui sono segnalati differiscono da un dispositivo a un altro

Velocità di trasferimento:



Funzioni I/O: ci sono 2 modi per gestire l'input/output

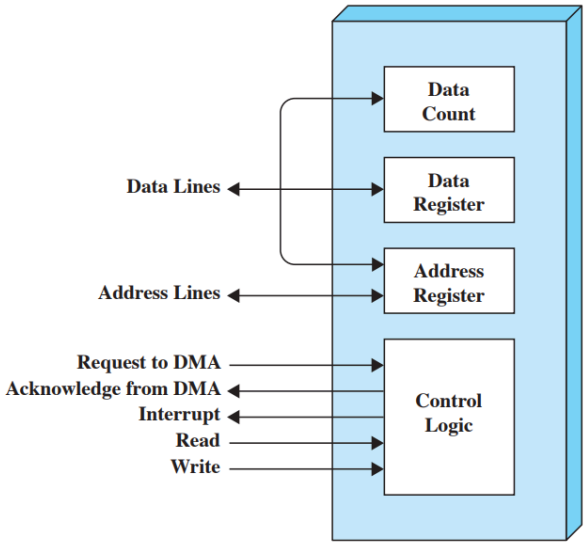
- *I/O programmato:* il processore genera un comando di I/O per conto di un processo che rimane in attesa attiva durante il completamento dell'operazione
- *I/O guidato dalle interruzioni:* il processore genera un comando di I/O per conto di un processo:
  - Non-bloccante: il processore continua a eseguire istruzioni del processo che ha eseguito il comando di I/O
  - Bloccante: il controllo passa al SO che sospende il processo corrente e ne schedula un altro

DMA: un modulo DMA controlla lo scambio di dati tra memoria principale e dispositivo I/O

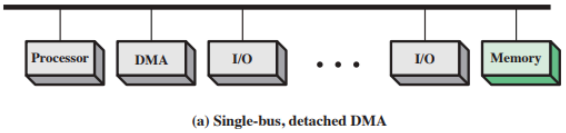
Evoluzione della funzione I/O:

1. Il processore controlla direttamente il dispositivo periferico
2. Viene aggiunto un controllore o modulo I/O
3. Come al passo precedente, con l'introduzione delle interruzioni
4. Col DMA, il modulo I/O, diventa un processore separato con un suo instruction set specializzato (canale di I/O)
5. Il modulo I/O viene dotato di memoria locale e diventa un computer vero e proprio (processore di I/O)

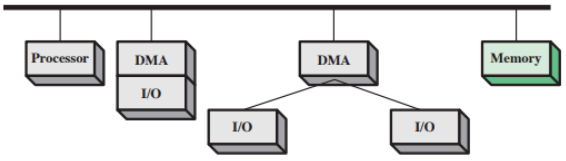
Via via, il processore è meno coinvolto nella gestione dell'I/O



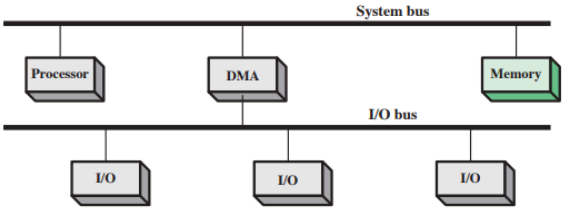
Configurazione DMA: Per organizzare i DMA ho diverse possibilità. La più semplice è la (a): un singolo bus a cui è connesso tutto, la memoria, il processore, il DMA e i dispositivi I/O. Il vantaggio è la semplicità, ma il difetto sta nel fatto che sulla stessa linea devono passare i dati dal dispositivo al DMA, dal DMA alla memoria e i dati fra lo scambio di dati e processore. Per questo può essere preferibile il modello (b) in cui c'è ancora un singolo bus, ma i dispositivi sono ancora connessi, ognuno con il proprio DMA. La terza possibilità (c) è creare due bus, uno a cui sono collegati tutti i dispositivi I/O (che è collegato al DMA) e l'altro bus, collegato agli elementi del sistema.



(a) Single-bus, detached DMA



(b) Single-bus, Integrated DMA-I/O



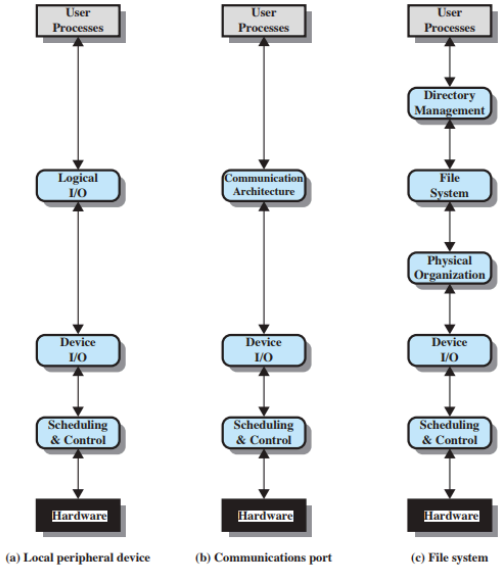
(c) I/O bus

Obiettivi di progettazione:

- *Efficienza:* è il maggiore obiettivo della progettazione (soprattutto per I/O dal disco):
  - Le operazioni di I/O costituiscono un collo di bottiglia
  - I dispositivi I/O sono molto lenti rispetto a memoria principale e processore (soprattutto per I/O dal disco)
- *Generalità:* ricerca uniformità nella gestione dei dispositivi:
  - Nell'interfaccia verso i processi
  - Nella gestione dei dispositivi da parte del SO e le relative operazioni

È difficile da realizzare per le diversità tecniche dei dispositivi. La soluzione è l'approccio gerarchico, per nasconde i dettagli dei dispositivi nelle routine di basso livello

Struttura logica:

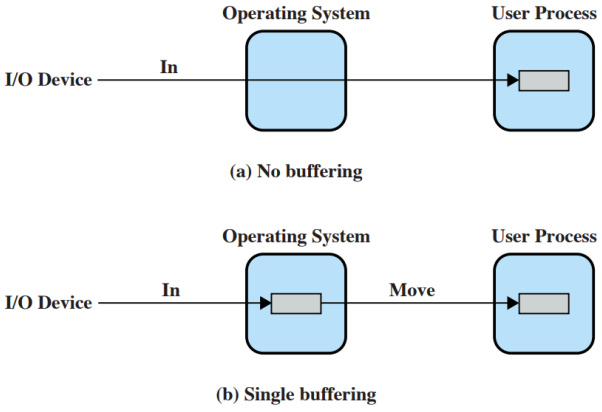


- I/O logico: tratta il dispositivo come una risorsa logica (open, close read, write)
- I/O sul dispositivo: converte le operazioni richieste in sequenze di istruzioni
- Scheduling e controllo: accomodamento, scheduling, controllo, interruzioni
- Gestione di directory e file e organizzazione fisica: realizzazione dell'astrazione dei file

Buffering: per evitare inefficienze e overhead, può essere utile eseguire trasferimenti di input in anticipo sulla richiesta e trasferimenti di output in ritardo rispetto alla richiesta

- *Dispositivi a blocchi:* dischi, chiavi USB. L'informazione viene memorizzata in blocchi di dimensione fissa e il trasferimento viene eseguito un blocco alla volta. Il riferimento ai dati avviene per numero di blocco
- *Dispositivi a flusso:* terminali, stampanti, porte di comunicazione, mouse, ecc. L'informazione non è strutturata in blocchi, invece il trasferimento dei dati avviene attraverso un flusso di bytes

Buffer singolo (dispositivo a blocchi): Col buffering i dati vengono quindi trasferiti prima in quest'area di memoria principale e poi vengono lette dal buffer al processo.



Il buffer singolo per un dispositivo a blocchi è il sistema di supporto più semplice. Quando un processo esegue una richiesta di I/O, il SO assegna un buffer all'operazione (nello spazio del sistema). Nelle operazioni di input, i dati sono trasferiti nel buffer. Input anticipato:

- In previsione che il blocco sia richiesto
- Terminato il trasferimento il blocco viene trasferita dal buffer allo spazio utente e ne viene immediatamente richiesto un altro

Velocizza il sistema:

- Il processo può elaborare un blocco di dati mentre il blocco successivo viene caricato

- Il SO può scaricare il processo (o la pagina) perché il trasferimento avviene nello spazio del sistema

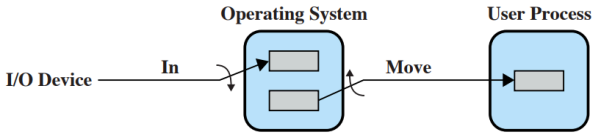
Gli svantaggi sono:

- Maggior complessità nella programmazione del So
- Influenza la gestione dello swapping

Buffer singolo (dispositivo a flusso):

- Una linea alla volta: i terminali in modo scroll, stampanti a linee. L'input è diviso in linee che terminano con 'invio':
  - Il processo è sospeso in attesa della linea di input
  - La linea di output è trasferita nel buffer il processo può continuare
- Un byte alla volta: Terminali in modo 'form', sensori, controller

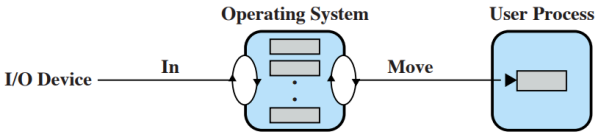
Doppio buffer:



(c) Double buffering

Alternativamente si possono usare due buffer per l'operazione di I/O, uno in cui il processo trasferisce i dati e l'altro che il SO svuota o riempie. Questo ci permette di agire ancora più velocemente poiché non si deve attendere che il buffer si sia svuotato.

Buffer circolare:



(d) Circular buffering

Più di due buffer sono assegnati all'operazione di I/O caricati e scaricati circolarmente

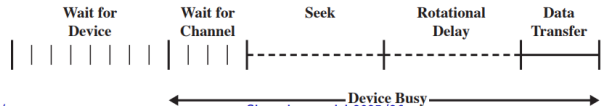
Parametri di prestazione del disco: i dettagli delle operazioni di I/O dal disco dipendono da:

- Sistema di elaborazione
- Sistema operativo
- Natura del canale I/O e hardware di controllo del disco

Il disco è diviso in tracce, cerchi concentrici che contengono il materiale ferro-magnetico usato per memorizzare le cose, su ogni disco c'è una testina che scorre sulla traccia, per certi dischi c'è una testina per ogni traccia. La velocità di rotazione del disco è costante. La testina di lettura/scrittura deve essere posizionata all'inizio del settore richiesto nella particolare traccia richiesta:

- Nei dischi a teste fisse, la traccia si ottiene selezionando elettronicamente la relativa testina
- Nei dischi a teste mobili, è necessario spostare la testina sulla relativa traccia (tempo di ricerca)

Il tempo necessario al settore per raggiungere la testina è detto *ritardo rotazionale*. La somma di tempo di tempo di ricerca e ritardo rotazionale costituisce il *tempo di accesso*, dopo il quale siamo all'inizio del settore che ci interessa e la testina può leggere o scrivere i dati che ci interessano.



Tempo di accesso:

- Tempo di ricerca: è il tempo necessario a muovere il braccio del disco sulla traccia richiesta. Consiste di: tempo di avvio e tempo necessario per attraversare i cilindri (comprensivo di *settling time*). Più piccolo è il disco, minore è il tempo di ricerca (oggi < 10 ms)
- Ritardo di rotazione: Il tempo necessario al settore per raggiungere la testina. Più il disco gira velocemente, minor tempo impiegherà il settore ad arrivare (a 3600 rpm, in media 83. Ms; a 15000 rpm, in media 2 ms)
- tempo di trasferimento:

  $$=$$

  con:
- = numero di bit da trasferire
- = numero di bit per traccia
- = velocità di rotazione in rps

Il tempo di accesso medio è dato dalla formula:

$$_{a} = +12+$$

ove:

- = numero di bit da trasferire
- = numero di bit per traccia
- = velocità di rotazione in rps
- = tempo di ricerca medio

Example

Un disco con tempo di ricerca medio 4 ms, velocità rotazionale 7500 rpm, 500 settori per traccia, ciascuno da 512 byte. Un file occupa tutti i settori su 5 tracce adiacenti (organizzazione sequenziale). Il tempo per la lettura della prima traccia sarà:

tempo di ricerca medio	4 ms
ritardo rotazionale	4 ms
lettura di 500 settori	8 ms
	16 ms

Per le tracce successive, il tempo di ricerca si riduce quasi a 0, per cui il tempo totale necessario è: (16 + 12 + 12 + 12)ms = 64 ms

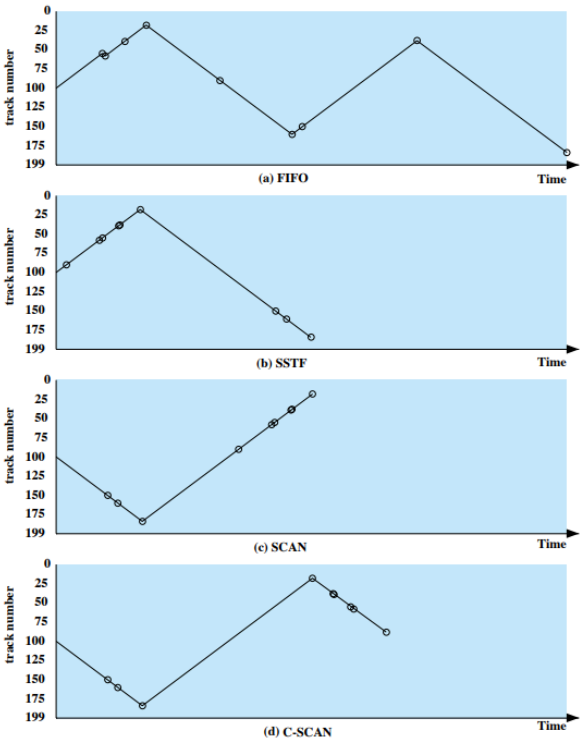
Se i settori sono distribuiti a caso sul disco, il tempo di accesso a ogni settore è:

tempo di ricerca medio	4 ms
ritardo rotazionale	4 ms
lettura di un settore	0.016 ms
	8.016 ms

Per cui il tempo totale necessario risulta, mediamente, $00\ .01 = 20.0$ secondi

Politiche di schedulazione del disco: l'obiettivo principale è quello di ridurre il tempo medio di ricerca

- **FIFO:** Le richieste sono processate nel loro ordine di arrivo. È semplice da implementare ed è equa per tutti i processi. In presenza di molti processi approssima il tempo impiegato dalla schedulazione random
- **Priorità:** La gestione a priorità è eseguita al di fuori del software di controllo del disco. L'obiettivo non è quindi ottimizzare l'utilizzo del disco, ma ottimizzare il comportamento globale. La priorità verrebbe data ai lavori brevi o ai processi interattivi. Si hanno buoni tempi di risposta per i processi a cui la priorità è data, ma lunghe attese per i lavori che in generale hanno bassa priorità (un problema potrebbe essere la starvation dei lavori con bassa priorità). Ad esempio i lavori batch, in un sistema condiviso, l'utente può spezzare il lavoro in piccoli segmenti per “ingannare” il sistema. È inoltre inadatta per i sistemi di basi di dati.
- **SSTF:** precedenza alla richiesta con tempo di servizio minore (*shortest service time first*). Seleziona quindi la richiesta che richiede il minimo movimento del braccio del disco dalla posizione corrente. Questo minimizza il tempo di richiesta, ma anche qui c'è la possibilità di starvation per lavori che richiedono tempo di servizio più alto
- **SCAN:** Detto anche l'algoritmo dell'ascensore, come per gli ascensori c'è una fase di salita e una di discesa. Nella fase di salita il disco si ferma su un cilindro e fa tutti i lavori su quel cilindro, poi passa al prossimo cilindro superiore se c'è richiesta di altri lavori su cilindri superiori a quello precedente. Quando a salire non ci sono altre richieste comincia la fase di discesa. Il braccio quindi si muove sempre in una direzione e soddisfa tutte le richieste su ciascuna traccia finché non raggiunge l'ultima traccia, poi inverte la direzione. Si ha una buona performance e si ha una soluzione al problema dell'attesa indefinita poiché prima o poi tutte le richieste vengono completate, vengono favoriti i settori centrali poiché vengono incontrati sia in discesa che in salita.
- **C-SCAN:** Restringe la scansione a un'unica direzione. Raggiunta l'ultima traccia, il braccio ritorna alla prima traccia e ricomincia la scansione nella medesima direzione
- **N-Step-SCAN:** Evita attese troppo lunghe quando nuovi lavori arrivano sulla medesima traccia. Le richieste sono inserite in sotto-code di lunghezza N che sono processate una alla volta con SCAN. Le nuove richieste non possono essere inserite nella coda attualmente processata
- **FSCAN:** Si hanno due sottocode, all'inizio della scansione una coda è vuota, durante la scansione sono eseguite le richieste della coda inizialmente non vuota e tutte le nuove richieste sono inserite nell'altra.



(a) FIFO (starting at track 100)		(b) SSTF (starting at track 100)		(c) SCAN (starting at track 100 in the direction of increasing track number)		(d) C-SCAN (starting at track 100 in the direction of increasing track number)	
Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed
55	45	90	10	150	50	150	50
58	3	58	32	160	10	160	10
39	19	55	3	184	24	184	24
18	21	39	16	90	94	18	166
90	72	38	1	58	32	38	20
160	70	18	20	55	3	39	1
150	10	150	132	39	16	55	16
38	112	160	10	38	1	58	3
184	146	184	24	18	20	90	32
Average seek length	55.3	Average seek length	27.5	Average seek length	27.8	Average seek length	35.8

Name	Description	Remarks
	Selection according to requestor	
RSS	Random scheduling	For analysis and simulation
FIFO	First-in-first-out	Fairest of them all
PRI	Priority by process	Control outside of disk queue management
LIFO	Last in first out	Maximize locality and resource utilization
	Selection according to requested item	
SSTF	Shortest-service-time first	High utilization, small queues
SCAN	Back and forth over disk	Better service distribution
C-SCAN	One way with fast return	Lower service variability
N-step-SCAN	SCAN of N records at a time	Service guarantee
FSCAN	N-step-SCAN with N = queue size at beginning of SCAN cycle	Load sensitive

Gestione dei file

Tutti i dati sono contenuti nei file, una collezione di dati che il sistema ci permette di interpretare grazie ad una struttura logica. È considerata fra le parti più importanti del Sistema Operativo lato utente

Proprietà:

- *Persistenza:* i file sono memorizzati su disco o altro supporto, non scompaiono quando l'utilizzatore termina il suo lavoro (con lo spegnimento del PC)
- *Condivisione:* Hanno un nome e dei permessi di accesso associati che ne permettono la condivisione (interessante soprattutto nei sistemi condivisi perché utenti differenti possono essere autorizzati o meno a fare uso dei vari file)
- *Struttura:* Sono organizzati in strutture gerarchiche (o più complesse) per riflettere le relazioni tra file

File system: Per poter organizzare i nostri file abbiamo bisogno di una parte del sistema operativo detto “gestore del file system”. Questo fornisce una serie di operazioni che permettono di operare sui file e anche di gestire gli attributi associati ai file (per esempio i permessi). Le principali operazione che il file system ci deve assicurare: creare i file, cancellazione dei file, apertura (per scrivere o leggere il file), chiusura, lettura (di vecchie informazioni) e scrittura (di nuove informazioni)

Sistema di gestione dei file: Fornisce servizi agli utenti e alle applicazioni che permettono l'utilizzo dei file

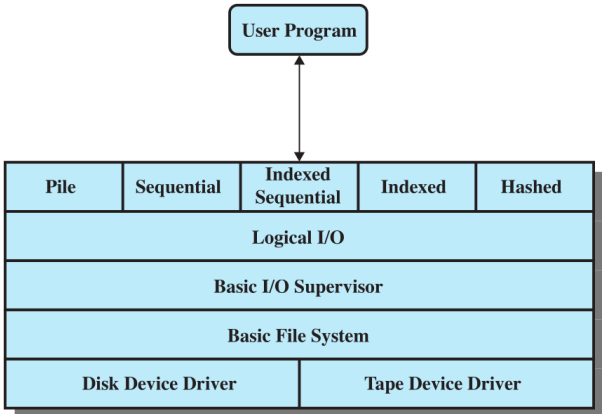
Obiettivi:

- Incontrare le esigenze degli utenti (i principali metodi di accesso ai file usando windows sono quelli che utilizzano l'interfaccia grafica, molto utile per gli utenti meno esperti)
- Permettono di modificare i dati in maniera veloce.
- Garantire la validità dei dati (le operazioni che coinvolgono i dispositivi sono quelle che danno i maggiori problemi, a causa delle parti meccaniche, fili… quindi bisogna verificare che le operazioni siano state portate a termine senza errori oppure segnalare gli errori).
- Provvedere all'input/output dei dispositivi (disco fisso, disco ottico, nastro, penna USB…)
- Minimizzare il rischio di perdita o distruzione dei dati (un backup del contenuto del disco è sempre utile).
- Fornire un'interfaccia standardizzata, uniforme.
- Nei sistemi multiutente deve essere previsto il supporto input/output.

Esigenze degli utenti:

- Creare, cancellare, leggere, scrivere, modificare i file
- Accesso controllato ai file di altri utenti
- Controllo del tipo di accesso consentito agli altri utenti
- Possibilità di muovere dati tra i file
- Backup e ripristino dei file in caso di danneggiamento
- Accesso ai file per nome, anziché per numero identificatore

Architettura del sistema di gestione dei file:



Il sistema di gestione dei file viene organizzato a strati. Partendo dal livello più basso, troviamo l'hardware dei dispositivi di memorizzazione (dischi, nastri, chiavette, CD, ecc.). I **Driver dei Dispositivi** comunicano direttamente con queste periferiche e gestiscono le operazioni di input/output, sia il loro avvio che il loro completamento.

Base del file system:

- Interfaccia primaria con l'ambiente esterno (livello dell'I/O fisico)

- Si occupa dei blocchi di dati scambiati con il disco (o i nastri)
- Controlla il posizionamento dei blocchi nella memoria secondaria
- Organizza i buffer in memoria principale
- Non conosce né il contenuto dei dati né la struttura del file

Supervisore dell'I/O base:

- Responsabile di inizio e terminazione di tutte le operazioni di I/O su file
- Gestisce le strutture di controllo per i dispositivi I/O, la schedulazione dei dispositivi e lo stato dei file.
- Seleziona il dispositivo su cui eseguire I/O
- Si occupa della schedulazione del disco e degli altri dispositivi per ottimizzare le prestazioni
- Assegna i buffer I/O e alloca la memoria secondaria

Gestore logico di I/O: permette di vedere un file come una collezione di record logici di lunghezza fissa o variabile

- Abilita utenti e applicazioni ad accedere ai record
- Fornisce le funzioni di I/O ai record di tipo generale
- Realizza le basi di dati relative ai file

Metodi di accesso: si tratta del livello più prossimo all'utente e alle applicazioni. Il sistema operativo deve fornire questi diversi metodi per accedere ai file, in base alla loro organizzazione logica. Questo strato realizza un'interfaccia uniforme e standardizzata. Differenti metodi di accesso sono dovuti alle diverse strutture logiche con cui i file possono essere organizzati.

Directory: Il file system utilizza le directory per organizzare i file. Per l'utente i file hanno un nome e un utilizzatore associato. Immaginiamo di avere dei cassetti, all'interno dei quali ci sono altri cassetti e poi ci sono i file. Noi abbiamo la possibilità di navigare fra le directory. Operazioni possibili sulla directory:

- Ricerca
- Crea file
- Cancella file
- Listare il contenuto della directory
- Aggiornare la directory

Basic Information	
File Name	Name as chosen by creator (user or program). Must be unique within a specific directory.
File Type	For example: text, binary, load module, etc.
File Organization	For systems that support different organizations
Address Information	
Volume	Indicates device on which file is stored
Starting Address	Starting physical address on secondary storage (e.g., cylinder, track, and block number on disk)
Size Used	Current size of the file in bytes, words, or blocks
Size Allocated	The maximum size of the file
Access Control Information	
Owner	User who is assigned control of this file. The owner may be able to grant/deny access to other users and to change these privileges.
Access Information	A simple version of this element would include the user's name and password for each authorized user.
Permitted Actions	Controls reading, writing, executing, transmitting over a network
Usage Information	
Date Created	When file was first placed in directory
Identity of Creator	Usually but not necessarily the current owner
Date Last Read Access	Date of the last time a record was read
Identity of Last Reader	User who did the reading
Date Last Modified	Date of the last update, insertion, or deletion
Identity of Last Modifier	User who did the modifying
Date of Last Backup	Date of the last time the file was backed up on another storage medium
Current Usage	Information about current activity on the file, such as process or processes that have the file open, whether it is locked by a process, and whether the file has been updated in main memory but not yet on disk

Schema a due livelli:

- Una directory principale e una directory per ogni utente

- La directory principale contiene una entry per ogni directory utente con l'indirizzo e i diritti di accesso
- Ogni directory utente è una lista di files dell'utente
- Nomi unici solo all'interno di ciascuna directory
- Il SO impone le restrizioni di accesso sulle directory

Ogni directory utente può contenere sottodirectory e file (schema ad albero)

Organizzazione dei file:

File a mucchio: meccanismo più semplice per organizzare i dati

- dati registrati in ordine di arrivo
- record con campi differenti o in ordine differente
- accesso ai record per ricerca esaustiva

Ogni nuovo record viene attaccato in coda al file. Sono però difficili da leggere, infatti per trovare un record bisogna leggere tutto il file.

File sequenziale:

- record in formato fisso (ogni record ha gli stessi campi)
- record identificati dal campo chiave
- utile per le applicazioni batch (ricerche lente)
- Il file delle transazione contiene i nuovi record (aggiunti successivamente)

File sequenziale indicizzato:

- aggiunge un indice per accesso random (accesso ai record più rapido)
- modifiche con il file di overflow (contiene i nuovi record accessibili mediante puntatori dal file indicizzato)

File indicizzato:

- record accessibili attraverso gli indici: un indice totale e un indice per ogni campo. Vanno aggiornati a ogni modifica di record
- ammessi record a lunghezza variabile
- veloce accesso ai record

File diretto o a hash:

- mediante hashing della chiave si può accedere direttamente al blocco del record
- nessuna organizzazione sequenziale dei record

Allocazione dei file: In memoria secondaria, un file consiste di una collezione di blocchi. Il SO gestisce l'allocazione dei blocchi di file. L'approccio scelto per l'allocazione dei file influenza la gestione dello spazio libero. Lo spazio è allocato in una o più porzioni (insiemi di blocchi contigui)

FAT: Struttura dati che tiene traccia dei blocchi allocati al file. Ci sono 2 possibilità per allocare:

- *La preallocazione:* creo il mio file, dichiaro quanti blocchi sono necessari, il sistema seleziona i blocchi che mi servono facendo si che siano contigui e quindi che vi si possa accedere rapidamente. Problema: la stima dello spazio necessario è difficile, si tende a sovrastimare. Mi permette comunque di allocare gli spazi in maniera intelligente perché lo faccio prima
- *L'allocazione dinamica* alloca lo spazio al file in base alle necessità

Dimensione delle porzioni: Due esigenze in contrasto:

- Contiguità dello spazio migliora le prestazioni specie per SO orientati alle transazioni
- Un gran numero di porzioni piccole aumenta la dimensione delle tabelle per gestire i dati di allocazione
- Porzioni di taglia fissa facilita la riallocazione dello spazio

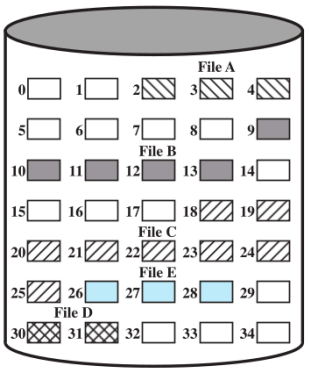
Porzioni contigue grandi e variabili:

- Prestazioni efficienti
- La dimensione variabile evita sprechi di spazio
- Tabelle di allocazione piccole

Blocchi:

- Maggiore flessibilità
- Possono richiedere tabelle di allocazione grandi
- La contiguità non è un obiettivo (blocchi allocati quando richiesti)
- I blocchi vengono assegnati dinamicamente (è inutile - preallocare se perdiamo la contiguità)

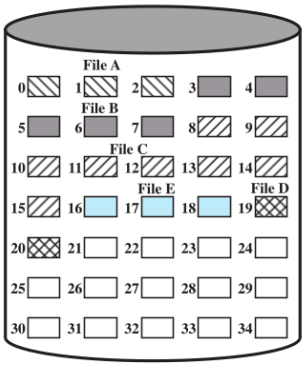
Allocazione contigua: potremmo dover ricorrere alla compattazione per avere dei blocchi contigui liberi.



File Allocation Table

File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

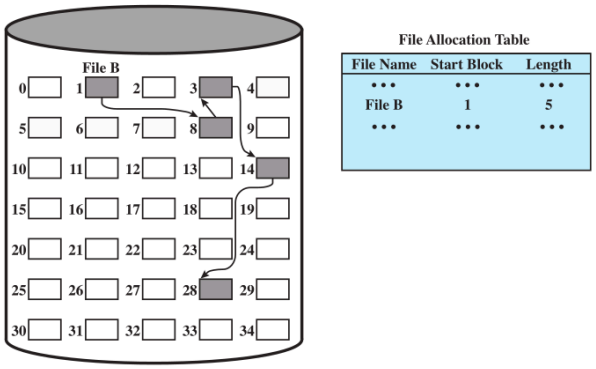
Allocazione contigua (dopo compattazione):



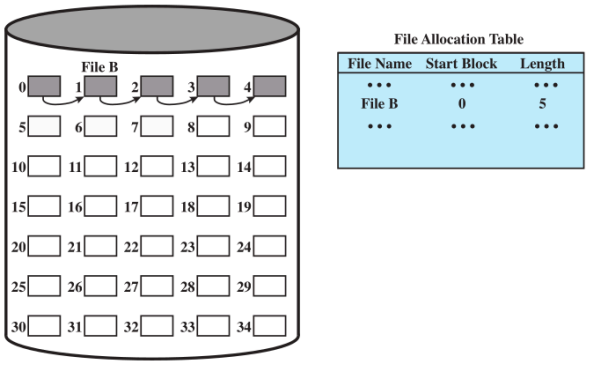
File Allocation Table

File Name	Start Block	Length
File A	0	3
File B	3	5
File C	8	8
File D	19	2
File E	16	3

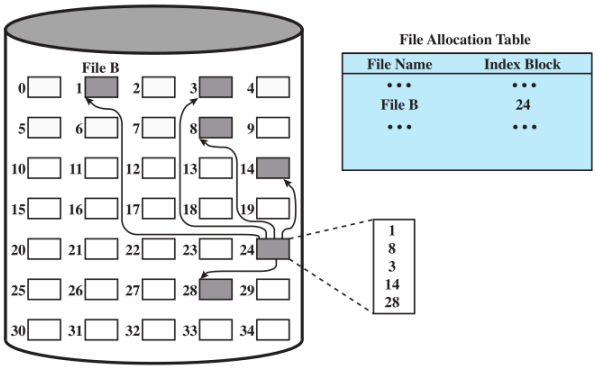
Allocazione a catena: si perde la continuità, ogni blocco contiene un puntatore al blocco successivo: non abbiamo problemi di compattazione.



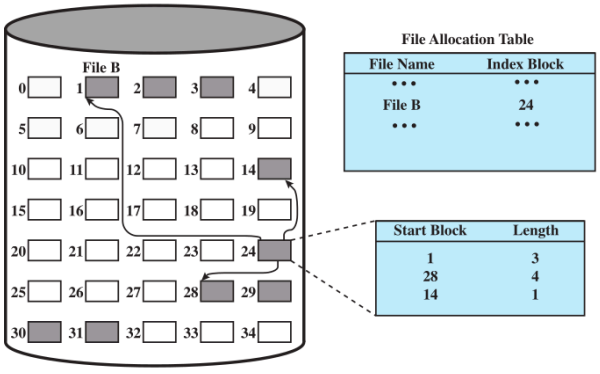
Allocazione a catena (dopo compattazione):



Allocazione indicizzata con porzioni a blocchi: il primo blocco contiene tutti i puntatori verso gli blocchi. È così molto rapido per accedere magari all'ultimo blocco.



Allocazione indicizzata con porzioni a lunghezza variabile: la differenza è che qua nei limiti del possibile i blocchi solo allocati in porzioni di lunghezza variabile. Nel primo blocco ci sono i puntatori ai primi blocchi della porzione. Il vantaggio è che almeno parzialmente i blocchi sono contigui.



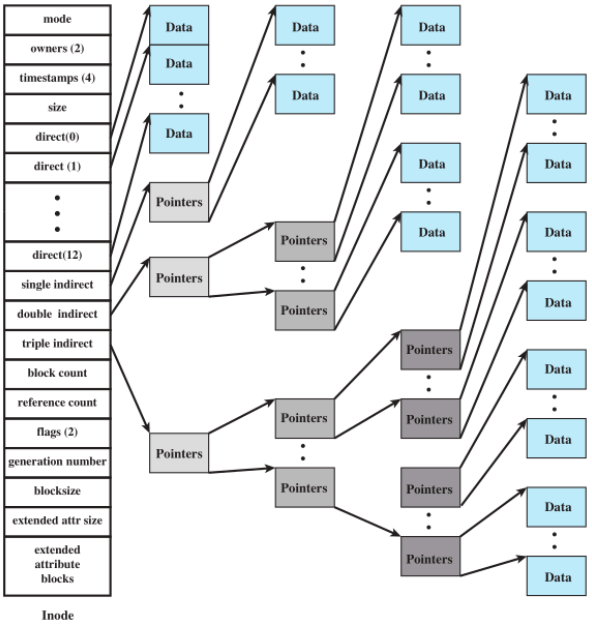
Gestione degli spazi liberi: È necessario conoscere i blocchi disponibili: per questo con la *tabella di allocazione del disco* troviamo i blocchi ancora disponibili. Ho vari meccanismi:

- *Bitmap:* Un vettore con un bit per ogni blocco 0 = libero, 1 = allocato. La dimensione è minima
- *Porzioni libere concatenate:* In ogni porzione libera, la lunghezza è un puntatore alla porzione libera successiva. Non c'è spreco di spazio. I puntatori vanno aggiornati a ogni allocazione o rilascio
- *Indicizzazione:* Spazio libero trattato come un file con la sua FAT. Per contenere le dimensioni, indice basato su porzioni a lunghezza variabile

Gestione file in UNIX: esistono 6 tipi di file:

1. *ordinari:* contiene dati arbitrari in 0 o più blocchi
2. *directory:* contiene una lista di filename più i puntatori ai relativi i-node
3. *speciali:* usati per accedere ai dispositivi periferici
4. *pipe con nome:* per la comunicazione tra processi
5. *link:* nome alternativo per un file esistente
6. *link simbolico:* un file dati contenente il nome di un file al quale è collegato

I-node: tutti i tipi di file UNIX sono amministrati dal Sistema Operativo attraverso gli i-node (index node). L'i-node contiene le informazioni chiave di cui il sistema ha bisogno per un particolare file. Più nomi di file possono essere associati a un singolo i-node. Un i-node è associato a un unico file e ogni file è controllato da un unico i-node



Allocazione dei file:

- Allocazione a blocchi
- allocazione dinamica
- Indicizzazione mediante puntatori diretti (nell'i-node) e tre puntatori indiretti (singoli, doppi, tripli)

Struttura del volume: Il file system UNIX risiede su un singolo disco logico. Contiene:

- *Boot block:* codice per avviare il sistema
- *Superblock:* informazioni e attributi del file system
- *Tabella degli i-node*
- *Blocchi dati:* spazio disponibile per i file dati e le directory